



中国科学院大学
University of Chinese Academy of Sciences

博士学位论文

Linux 内核地址空间布局持续随机化若干关键技术研究

作者姓名: 李润宽

指导教师: 赵晓芳 正高级工程师

中国科学院计算技术研究所

学位类别: 工学博士

学科专业: 计算机应用技术

培养单位: 中国科学院计算技术研究所

2026 年 12 月

Research on Key Technologies of Continuous Address Space
Layout Re-randomization for the Linux Kernel

A dissertation submitted to
University of Chinese Academy of Sciences
in partial fulfillment of the requirement
for the degree of
Doctor of Philosophy
in Computer Applied Technology

By

LI RUNkuan

Supervisor: Professor ZHAO XIAOfang
Institute of Computing Technology, Chinese Academy of Sciences

Institute of Computing Technology, Chinese Academy of Sciences

December, 2026

中国科学院大学 学位论文原创性声明

本人郑重声明：所呈交的学位论文是本人在导师的指导下独立进行研究工作所取得的成果。承诺除文中已经注明引用的内容外，本论文不包含任何其他个人或集体享有著作权的研究成果，未在以往任何学位申请中全部或部分提交。对本论文所涉及的研究工作做出贡献的其他个人或集体，均已在文中以明确方式标明或致谢。本人完全意识到本声明的法律结果由本人承担。

作者签名：

日 期：

中国科学院大学 学位论文授权使用声明

本人完全了解并同意遵守中国科学院大学有关收集、保存和使用学位论文的规定，即中国科学院大学有权按照学术研究公开原则和保护知识产权的原则，保留并向国家指定或中国科学院指定机构送交学位论文的电子版和印刷版文件，且电子版与印刷版内容应完全相同，允许该论文被检索、查阅和借阅，公布本学位论文的全部或部分内容，可以采用扫描、影印、缩印等复制手段以及其他法律许可的方式保存、汇编本学位论文。

涉密及延迟公开的学位论文在解密或延迟期后适用本声明。

作者签名：

日 期：

导师签名：

日 期：

摘要

Linux 内核以 C 语言编写，内存安全漏洞持续被引入与发现，代码重用攻击因而长期是内核面临的主要威胁。持续随机化在运行期反复改变代码布局，使已泄露的地址迅速失效，是应对信息泄露类攻击的有效路线。然而现有的持续随机化工作或面向用户态进程，或以内核模块为保护对象，都依赖一条明确的模块边界来限定随机化范围；而内核中相当一部分代码在任何配置下都无法编译为模块，这部分 **built-in** 代码不具备这样的边界，其代码索引的更新规模、执行流的安全暂停、以及随机化范围之外代码的保护，都缺乏可用的现成解法。

本文面向 Linux 内核 **built-in** 代码，提出并实现了一套持续随机化系统。该系统以跳板函数作为随机化区域的唯一出入口，在其之上组织三项关键技术，分别回答持续随机化的三个核心问题。

第一项工作解决“如何随机化”。本文提出基于跳板函数的低开销持续随机化执行机制：在保持内核原有代码模型不变的前提下，由编译器插件按函数改写被选中函数内部的引用，使函数体可以整体搬移；跳板在承担地址间接的同时充当执行流的观测点，由进出计数精确刻画随机化区域内的活跃执行流，从而实现不保留旧副本的切换。实验表明，一次随机化需改写的代码索引由与调用点数量同阶降至与函数数量同阶，一次跨区域调用往返的常驻开销为 **18.02 ns**，其中执行流追踪占绝大部分。

第二项工作解决“何时随机化”。本文把只执行内存由阻断机制改用于检测机制，以对代码探测行为的直接观测替代时钟或行为代理指标作为触发条件，并给出基于管理态保护键与基于硬件调试观察点的两条实现路径。实验表明，正常负载下的触发频率为每秒 **0.02** 至 **0.44** 次，从检测到随机化完成的延迟在基础负载下 **99** 分位不超过 **110 μ s**；受控自测的竞态实验表明，泄露的地址在攻击者完成解引用之前即已失效，且每一次读取本身都会再触发一次随机化，使重复尝试无法累积。

第三项工作解决随机化范围之外代码的保护。本文利用随机化区域内目标地址不可预测这一性质，提出以随机化打断指针认证上下文传播链的协同机制：以类型上下文恒定签名配合归属跟踪在运行时识别高扇入函数并将其移入随机化区域，并在跳板处施加分区验证。实验表明，**91%** 的间接调用点合法目标数不超过 **5**，最大等价类为 **57**，而同平台上基于函数类型哈希的方案在同一口径下最大等价类为 **141**。

综上，本文围绕 Linux 内核 **built-in** 代码的持续随机化，沿着“如何随机化、何时随机化、范围之外如何保护”的递进脉络，系统研究了随机化的执行机制、触发时机与非随机化区域的控制流约束，形成了从单次随机化的代价控制到全内核安全覆盖的一组方法与系统实现。本文的研究表明，**built-in** 代码长期落在持续随机化的保护范围之外，其原因不只在于工程量大，更在于这部分代码缺少一

条可依托的模块边界，代码索引因而无处集中、执行流无处观测、随机化范围无处划定。把跳板确立为随机化区域的唯一出入口之后，三者获得了共同的着力点：索引集中到跳转桩，执行流在跳板处被计数，区域边界由跳板给出。在此之上，以对代码探测行为的直接观测替代固定周期作为触发条件，并以随机化打断指针认证的上下文传播链，可以在可接受的开销内把保护覆盖到 **built-in** 代码，为内核代码重用攻击的防御提供一条可部署、可复现、边界清晰的路线。

关键词：内核安全，地址空间布局随机化，持续随机化，只执行内存，控制流完整性，指针认证

Abstract

The Linux kernel is written in C, memory-safety defects continue to be introduced and discovered, and code-reuse attacks therefore remain a principal threat to the kernel. Continuous re-randomization repeatedly changes the code layout at run time so that a leaked address quickly becomes stale, and is an effective answer to information-disclosure attacks. Existing work on continuous re-randomization, however, targets either user-space processes or kernel modules, and in both cases relies on an explicit module boundary to delimit the randomization scope. A substantial part of the kernel cannot be compiled as a module under any configuration; this built-in code has no such boundary, and for it the scale of code-pointer updates, the safe suspension of execution flows, and the protection of code outside the randomization scope all lack a usable off-the-shelf solution.

This dissertation presents a continuous re-randomization system for built-in Linux kernel code. The system makes trampoline functions the sole entry and exit of the randomized region, and organizes three key techniques on top of them, each answering one of the three core problems of continuous re-randomization.

The first technique addresses how to re-randomize. A low-overhead execution mechanism based on trampolines is proposed: keeping the kernel code model unchanged, a compiler plugin rewrites, per function, the references inside the selected functions so that a function body can be relocated as a whole; the trampoline serves at the same time as an observation point for execution flows, so that entry and exit counting characterizes exactly the flows currently executing inside the randomized region and a copy-free switch becomes possible. Measurements show that the number of code pointers to be rewritten per re-randomization drops from the order of the number of call sites to the order of the number of functions, and that the steady-state cost of one cross-region call round trip is 18.02 ns, of which execution-flow tracking accounts for the large majority.

The second technique addresses when to re-randomize. Execute-only memory is repurposed from a blocking mechanism into a detection mechanism, so that the trigger condition becomes a direct observation of code-probing behaviour rather than a clock or a behavioural proxy; two implementation paths are given, one based on supervisor memory protection keys and one based on hardware debug watchpoints. Measurements show a trigger rate of 0.02 to 0.44 per second under normal workloads and a 99th-percentile detection-to-completion latency of at most 110 μ s under baseline workloads; a controlled race experiment shows that a leaked address has already become stale before the attacker can dereference it, and that every read itself triggers a further re-randomization, so repeated attempts do not accumulate.

The third technique addresses the protection of code outside the randomization scope. Exploiting the fact that target addresses inside the randomized region are unpredictable, a cooperative mechanism is proposed that uses randomization to break the context-propagation chain of pointer authentication: high-fan-in functions are identified at run time by constant type-context signing together with ownership tracking, are moved into the randomized region, and partitioned verification is applied at the trampolines. Measurements show that 91% of indirect call sites have at most five legitimate targets and that the largest equivalence class is 57, whereas a scheme based on function type hashes yields a largest equivalence class of 141 on the same platform under the same criterion.

In summary, this dissertation studies continuous re-randomization of built-in Linux kernel code along the progressive line of how to re-randomize, when to re-randomize, and how to protect what lies outside the scope, covering the execution mechanism of re-randomization, the choice of trigger moment, and the control-flow constraints on the non-randomized region, and yielding a set of methods and a system implementation that reach from the cost of a single re-randomization to security coverage of the whole kernel. The work shows that built-in code has remained outside the reach of continuous re-randomization not merely because of engineering effort, but more fundamentally because this code has no module boundary to build on: code pointers have nowhere to be collected, execution flows nowhere to be observed, and the randomization scope nowhere to be delimited. Once trampolines are established as the sole entry and exit of the randomized region, the three obtain a common footing: code pointers are collected into the jump stubs, execution flows are counted at the trampolines, and the region boundary is given by the trampolines themselves. On that basis, replacing a fixed period with direct observation of code-probing behaviour as the trigger condition, and using randomization to break the context-propagation chain of pointer authentication, brings protection to built-in code at an acceptable cost and offers a deployable, reproducible route with clearly stated boundaries for defending the kernel against code-reuse attacks.

Key Words: Kernel Security, Address Space Layout Randomization, Continuous Re-randomization, Execute-Only Memory, Control-Flow Integrity, Pointer Authentication

目 录

第 1 章 绪论	1
1.1 从内核内存安全缺陷到持续随机化	1
1.2 研究问题与面临的挑战	4
1.2.1 共性挑战: built-in 代码缺乏明确的随机化边界	6
1.2.2 挑战一: 代码索引更新与执行流协调	7
1.2.3 挑战二: 随机化的触发时机与作用对象	8
1.2.4 挑战三: 非随机化区域中指针认证上下文的传播	8
1.3 研究内容与主要贡献	9
1.3.1 研究内容一: 基于跳板函数的低开销持续随机化	9
1.3.2 研究内容二: 基于代码索引泄露的触发式随机化	11
1.3.3 研究内容三: 基于随机化辅助的 ARM 指针认证机制	11
1.3.4 三项工作的关系	12
1.4 本文的安全性论证方式	12
1.5 论文组织结构	13
第 2 章 相关研究综述与威胁模型	15
2.1 内存安全漏洞攻防方法的演进	15
2.2 内核地址空间布局随机化相关工作	18
2.2.1 一次性随机化	19
2.2.2 持续随机化	19
2.2.3 部分随机化与触发式随机化	19
2.3 持续随机化的三个核心问题	21
2.3.1 代码索引更新	21
2.3.2 执行流的安全暂停	22
2.3.3 随机化的触发策略	22
2.3.4 非随机化区域的处理	23
2.4 只执行内存技术	24
2.4.1 实现路径	24
2.4.2 内核场景下的两个困难	25
2.5 硬件辅助的控制流完整性	26
2.5.1 落地点标记类	26
2.5.2 签名验证类	27
2.5.3 类型哈希类	27

2.5.4 小结	28
2.6 本文的威胁模型	28
2.6.1 攻击者能力	28
2.6.2 四类攻击向量	28
2.6.3 信任基础	29
2.6.4 明确的排除项	30
2.7 安全不变式	31
2.7.1 不变式的定义	31
2.7.2 不变式与章节、证据层级的对应	31
2.8 本章小结	32
第 3 章 基于跳板函数的低开销持续随机化方法	33
3.1 引言	33
3.2 研究背景与动机	33
3.2.1 Built-in 代码的代码索引更新	33
3.2.2 Built-in 代码的执行流暂停	34
3.2.3 研究动机与挑战	34
3.3 系统设计	35
3.3.1 总体架构	35
3.3.2 跳板函数的设计	36
3.3.3 函数粒度位置无关代码	41
3.3.4 执行流追踪与安全暂停	45
3.4 系统实现	47
3.4.1 LLVM 编译器扩展	47
3.4.2 运行时随机化的实现	48
3.4.3 可随机化的边界	50
3.5 实验评估	50
3.5.1 实验配置	50
3.5.2 随机化延迟	51
3.5.3 随机化内核核心子系统的开销	54
3.5.4 与已有持续随机化工作的同条件对比	55
3.5.5 随机化范围与跳板调用占比的关系	56
3.6 安全性分析	58
3.7 本章小结	58

第 4 章 基于代码索引泄露的触发式随机化	61
4.1 引言	61
4.2 研究背景与动机	61
4.2.1 跳板机制在开销上的局限	61
4.2.2 随机化的触发策略	62
4.2.3 保护整个代码区域的代价	62
4.2.4 研究动机与挑战	63
4.3 系统设计	63
4.3.1 总体架构	63
4.3.2 只执行代码区域的设计	63
4.3.3 信息泄露识别	64
4.3.4 两级跳板与跳板激活	66
4.4 系统实现	68
4.4.1 x86 平台：基于管理态保护键的只执行内存	68
4.4.2 ARM 平台：基于硬件调试观察点的实现	69
4.4.3 两级跳板的代码生成	70
4.5 实验评估	71
4.5.1 实验配置	71
4.5.2 两条路径的单次检测开销	72
4.5.3 触发频率与触发延迟	72
4.5.4 泄露地址的有效期与利用竞态	74
4.5.5 未激活与激活状态的单次调用开销	75
4.6 安全性分析	76
4.7 本章小结	76
第 5 章 基于随机化辅助的 ARM 指针认证机制	79
5.1 引言	79
5.2 研究背景与动机	79
5.2.1 部分随机化与非随机化区域	79
5.2.2 指针认证与上下文传播	80
5.2.3 研究动机与挑战	81
5.3 系统设计	81
5.3.1 总体架构与核心思想	81
5.3.2 以随机化打断上下文传播链	82
5.3.3 跳板处的分区验证	83
5.4 系统实现	85

5.4.1 编译期：指针认证插桩	85
5.4.2 运行期：高扇入函数的跟踪、绑定与迁移	86
5.4.3 跨区域出口的双重检查	86
5.5 实验评估	88
5.5.1 实验配置	88
5.5.2 控制流完整性检查的精度与开销	88
5.5.3 高扇入函数识别的收敛与准确性	91
5.5.4 其他部分随机化方法迁移	92
5.6 安全性分析	96
5.7 本章小结	97
第 6 章 总结与展望	99
6.1 论文总结	99
6.2 主要创新点	100
6.3 局限性与未来工作	100
6.4 结语	102
参考文献	103
致谢	111
作者简历及攻读学位期间发表的学术论文与其他相关学术成果 ..	113

图目录

图 1-1	内存错误利用模型及不同防御策略的作用阶段	2
图 1-2	本文的研究思路与三章分工	10
图 2-1	JIT-ROP 信息泄露攻击流程	18
图 2-2	只执行内存的三条实现路径	24
图 2-3	三类硬件 CFI 机制的等价类划分	26
图 2-4	四类攻击向量、三条安全不变式与三章工作的对应关系	30
图 3-1	本章机制的总体架构	35
图 3-2	一次跨区域调用的控制流	40
图 3-3	两个平台的绝对地址物化形式	43
图 3-4	返回地址的位置无关表示	44
图 3-5	执行流追踪与无副本随机化的时序	46
图 3-6	随机化抵达至开始的延迟累积分布	52
图 3-7	四种方法从随机化抵达到旧代码释放的时间线	54
图 3-8	与已有持续随机化工作的同条件对比	56
图 3-9	随机化范围与经过跳板的调用占比	57
图 4-1	本章机制的总体架构	64
图 4-2	跳转桩读取事件的处置与失效窗口	65
图 4-3	两级跳板与跳板激活	68
图 4-4	只执行保护施加的映射与跳转桩的改写路径	71
图 4-5	泄露地址利用竞态的成功率随读取至解引用时延的变化	75
图 5-1	指针认证的签名与认证过程	80
图 5-2	指针认证的上下文传播问题	81
图 5-3	本章机制的总体架构	82
图 5-4	高扇入函数的识别与绑定	83
图 5-5	跳板处的分区验证	85
图 5-6	每个间接调用点合法目标数的分布	90
图 5-7	高扇入函数识别数量随启动进程的变化	92
图 5-8	Chaos 原文四组负载在三档随机化间隔下施加本章机制前后的吞吐下降	94
图 5-9	施加本章机制前后的 CPU 开销与单次随机化的迁移时间	95

表目录

表 1-1	Linux 内核 CVE 的缺陷类型分布	2
-------	----------------------------	---

表 1-2	Linux 内核 CVE 所涉代码的可模块化性.....	5
表 2-1	Linux 内核镜像中可利用 gadget 的分布	16
表 2-2	持续随机化代表性工作的横向对比	20
表 2-3	安全不变式的维持与论证方式	32
表 3-1	本章使用的随机化范围	36
表 3-2	图 3-2 中每一步之后的执行流状态	39
表 3-3	一次跨区域调用往返的指令序列与周期分析	41
表 3-4	单次随机化的时间分解	49
表 3-5	不可随机化函数的分类	50
表 3-6	实验平台与配置	51
表 3-7	随机化抵达至开始的延迟	52
表 3-8	随机化开始至完成、完成至释放与抵达至释放的时长	53
表 3-9	四种方法在三个关键时刻上的时效对比	53
表 3-10	随机化四个核心子系统的端到端开销	55
表 3-11	与已有持续随机化工作的同条件对比	55
表 3-12	跨区域调用频度与常驻开销随随机化范围的变化	57
表 4-1	两级跳板各段的指令序列与周期分析	67
表 4-2	两个平台单次检测开销的分解	72
表 4-3	各类负载下的触发频率与读取来源	73
表 4-4	检测到随机化完成的延迟分布	73
表 4-5	泄露地址利用竞态的结果	74
表 4-6	未激活与激活状态在真实规模下的单次调用开销	76
表 5-1	合法目标数分布对比	88
表 5-2	类型哈希机制下最大的几个等价类	89
表 5-3	三档上下文策略在三个快照时刻的精度	91
表 5-4	高扇入函数识别的收敛	91
表 5-5	两种随机化原语的对照	93
表 5-6	施加本章机制前后的效果	94
表 5-7	向各方法移植所需的改造	96

符号与缩略语说明

缩略语

缩写 全称与含义

ASLR	Address Space Layout Randomization, 地址空间布局随机化
KASLR	Kernel ASLR, 内核地址空间布局随机化
ROP	Return-Oriented Programming, 返回导向编程
JOP	Jump-Oriented Programming, 跳转导向编程
COOP	Counterfeit Object-Oriented Programming, 伪造对象编程
JIT-ROP	Just-In-Time ROP, 即时代码重用攻击
DOP	Data-Oriented Programming, 数据导向编程
DEP	Data Execution Prevention, 数据执行阻止
CFI	Control-Flow Integrity, 控制流完整性
XOM	Execute-Only Memory, 只执行内存
PIC	Position-Independent Code, 位置无关代码
PIE	Position-Independent Executable, 位置无关可执行文件
GOT	Global Offset Table, 全局偏移表
PLT	Procedure Linkage Table, 过程链接表
PKS	Protection Keys for Supervisor, Intel 管理态内存保护键
PA / PAC	Pointer Authentication (Code), ARM 指针认证 (认证码)
BTI	Branch Target Identification, ARM 分支目标识别
IBT	Indirect Branch Tracking, Intel 间接分支跟踪
CET	Control-flow Enforcement Technology, Intel 控制流强制技术
SMEP	Supervisor Mode Execution Prevention, 管理模式执行阻止
SMAP	Supervisor Mode Access Prevention, 管理模式访问阻止
PXN	Privileged eXecute Never, ARM 特权级不可执行
TLB	Translation Lookaside Buffer, 地址翻译缓存
IPI	Inter-Processor Interrupt, 处理器间中断
RCU	Read-Copy-Update, 读拷贝更新
ORC	Oops Rewind Capability, 内核栈展开表格式
CVE	Common Vulnerabilities and Exposures, 公共漏洞披露编号
LLVM	一套模块化的编译器基础设施
IR	Intermediate Representation, 中间表示

本文使用的记号

记号	含义
----	----

<code>fixed_in</code>	进站跳板：由非随机化区域进入随机化区域的唯一入口
-----------------------	--------------------------

<code>fixed_out</code>	出站跳板：由随机化区域离开的唯一出口
------------------------	--------------------

<code>target_addr</code>	跳板中保存函数体当前地址的槽，随机化时唯一需要改写的位置
--------------------------	------------------------------

<code>I1 / I2 / I3</code>	第 2.7 节定义的一条安全不变式：机密性、时效性、完整性
---------------------------	-------------------------------

<code>V1 – V4</code>	第 2.6.2 节定义的四类攻击向量
----------------------	--------------------

<code>D1 – D3</code>	各章第 3 节给出的设计需求
----------------------	----------------

第 1 章 绪论

1.1 从内核内存安全缺陷到持续随机化

内核的内存安全缺陷难以被完全消除，运行时防护因此不可或缺。内核运行于最高特权级，管理页表与物理内存，一旦被攻陷，攻击者即可取得系统级控制权。2024 年 1 月公开的 CVE-2024-1086 是一个典型例子：该释放后使用缺陷位于网络过滤子系统 `nf_tables`，进入主线约十年才被发现，公开两个月后利用代码出现，随后被美国网络安全和基础设施安全局（CISA）列入已知被利用漏洞目录，并被确认用于勒索软件攻击。

这并非孤例：Linux 内核社区自 2024 年 2 月起成为 CVE 编号机构，截至 2026 年 9 月已为 10 705 条缺陷分配编号。表 1-1 按缺陷类型统计了这批 CVE。本文把缺陷分为三类：时间类缺陷指针指向的对象已经失效，包括使用释放内存、空指针解引用、内存释放缺失与多重释放；空间类缺陷指针越出了对象的边界，包括越界读、越界写、整型溢出与数组越界；其余归入非内存类，包括条件竞争、锁使用不当与初始化错误。前两类合称内存安全缺陷，合计占全部 CVE 的 81.3%，其中时间类 46.6%、空间类 34.7%。

这内存的时间错误与空间错误正是后续攻击步骤的主要入口：时间类对应图 1-1 中的指针悬空，空间类对应指针越界。缺陷持续出现的根源在于内核以不提供内存安全保证的 C 语言写成；Rust 正在进入内核，但只覆盖新增代码^[1]，千万行存量 C 代码短期内无法替换。动态检测工具能在测试阶段发现其中一部分缺陷，但插桩开销使其难以在生产内核中长期开启^[2]。由缺陷到利用原语的转化在内核上已相当成熟^[3-7]。因此，本文假定攻击者已经取得内存读写能力，考察如何阻止其进一步获取代码地址并劫持控制流。

内存错误本身并不直接构成危害，危害来自攻击者在其之上构造的利用链。图 1-1 给出内存安全的攻击模型：先使指针越界或悬空，再经该指针实施非法读写，进而篡改代码、代码指针、数据变量或输出数据，结果分为代码破坏、控制流劫持、纯数据攻击和信息泄露四类。防御技术沿这条链演进。NX 与 DEP 以页表权限区分可写页与可执行页，切断了由篡改代码到执行注入代码的直接路径；攻击者转而复用程序中已有的指令片段，以返回地址、函数指针或跳转表为落脚点把 gadget 串联成链，ROP^[8]、JOP 与 COOP 分别以返回指令、间接跳转与虚函数调用组织片段。这类攻击的共同前提是确定代码的位置。地址空间布局随机化（ASLR）正作用于这一前提：Linux 3.14 引入的 KASLR 随机化内核代码段的基址，FG-KASLR^[9] 进一步提出在启动时按函数重排。随机化无须针对每一种 gadget 组织方式分别定义规则，即可同时提高多类代码重用攻击的难度。

本文只防御由代码位置泄露通往控制流劫持的这条路径。图 1-1 中的着色部分即本文涉及的攻击路径：代码指针篡改、代码及代码索引读取，以及由此形成的控制流劫持。数据指针篡改与纯数据攻击不在直接防御范围之内：前者被视为

表 1-1 Linux 内核 CVE 的缺陷类型分布

Table 1-1 Distribution of Defect Types in Linux Kernel CVEs (2024.1—2026.9)

类别	缺陷类型	2024	2025	2026*	合计	占比/%
内存时间漏洞	使用释放内存	552	490	1065	2107	19.7
	空指针解引用	357	349	347	1053	9.8
	内存释放缺失	450	347	820	1617	15.1
	多重释放	61	55	98	214	2.0
	小计	1420	1241	2330	4991	46.6
内存空间漏洞	越界读	372	321	621	1314	12.3
	越界写	318	286	553	1157	10.8
	整型溢出	189	163	316	668	6.2
	数组越界	168	145	260	573	5.4
	小计	1047	915	1750	3712	34.7
非内存安全漏洞	条件竞争	268	231	402	901	8.4
	锁使用不当	186	159	278	623	5.8
	初始化错误	143	122	213	478	4.5
	小计	597	512	893	2002	18.7
内存安全漏洞合计		2467	2156	4080	8703	81.3
CVE 总数		3064	2668	4973	10705	100.0

注：*2026 年为部分年份，数据截至 2026 年 8 月 8 日的仓库快照，与表 1-2 同一份快照。

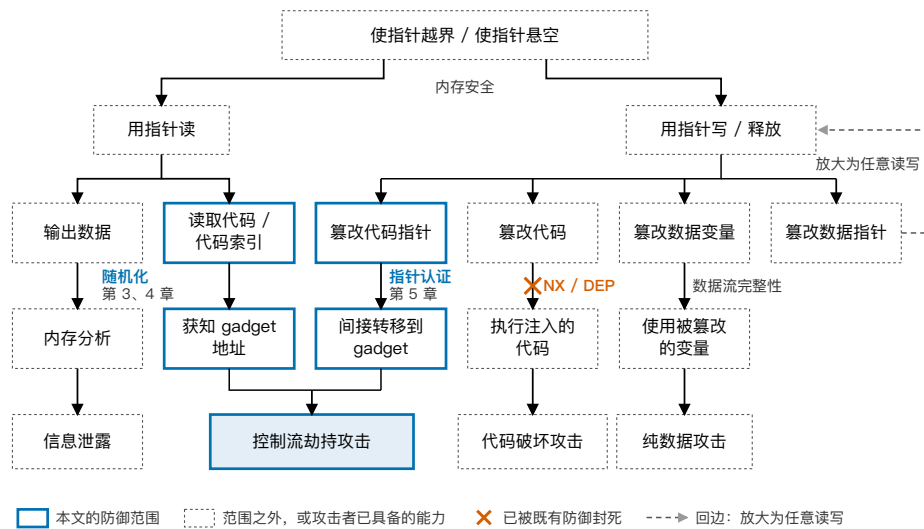


图 1-1 内存错误利用模型及不同防御策略的作用阶段

Figure 1-1 Memory-Error Exploitation Model and the Stages at Which Defenses Act

攻击者已具备的读写原语；后者无须获知 gadget 地址，也不产生异常的控制流，例如直接改写进程凭据结构中的 uid 字段，需要数据流完整性或内存安全强制机制来防护，二者可与本文方法配合使用。第 2.6 节给出完整的威胁模型与防御边界。

内核部署防御机制受两条用户态没有的约束。其一，内核缺少同一软件栈内更高特权级的强制执行者：用户态的权限边界由内核配置页表并由处理器实施，而内核自己掌管页表，SMEP、SMAP、PXN、PAN 与代码段只读等机制的有效性都以其控制状态不被篡改为前提；要形成独立于内核的强制边界，只能借助不

由普通页表直接控制的部件，例如以每处理器核寄存器保存权限状态的管理态保护键、只对数据访问触发的处理器调试观察点，或更高特权级的地址翻译。其二，内核能容忍的开销与兼容性代价更低：它位于系统调用、中断与异常的关键路径上，附加开销会传导至整个系统，且必须控制中断延迟；同时内核包含大量遗留代码、手写汇编与运行时代码修改，依赖全局静态分析或大范围改变代码语义的方案难以覆盖所有执行路径。

在这两条约束下，随机化是三类代表性防御路线中最适合内核的一种。控制流完整性先确定允许的转移关系，再在间接转移处验证目标，其效果取决于控制流图的精度^[10]；C 语言的函数指针与运行时注册使精确分析困难，硬件辅助的 CET-IBT、BTI 与 kCFI^[11-13] 只能验证落地点标记或类型签名这类可快速判定的属性，带相同标记或原型的目标仍构成较大的等价类^[14,15]；指针认证虽能以上下文区分调用点，实际系统中却出现数千个调用点共享一个上下文的情况^[16]。内存隔离先要确定保护对象，再描述合法访问规则，这条研究线已从进程内隔离发展到内核分区化^[17-22]，但内核中可被利用的对象随攻击方法变化，合法访问路径又随运行状态改变，为一般对象构建完整规则需要很强的语义分析能力。随机化主要依赖代码及其地址引用关系，这些信息大多可由编译器、链接器与重定位记录提供，对程序语义的依赖远小于前两者；装载时随机化不在每次转移或访问处检查，运行期开销也较低。

随机化依赖代码布局在运行期间保密，而信息泄露正是针对这一前提。攻击者可以利用越界读直接读出内存，可以根据访问是否引发崩溃探测地址有效性，也可以借助微体系结构侧信道；针对内核的去随机化侧信道已形成一条持续的研究线，从地址翻译缓存时序^[23]、DrK^[24] 与分支预测器冲突^[25]，到页表隔离部署之后仍然有效的预取侧信道^[26,27]、内核数据结构侧信道^[28] 与瞬态执行攻击^[29]，随机化一侧也有抵抗侧信道的尝试^[30]。JIT-ROP^[31] 是典型的信息泄露与控制流劫持的攻击：从一个泄露的代码指针出发扫描可达代码页，在运行时搜集 gadget 并构造攻击链，对应图 1-1 中先经“用指针读”获取布局、再经“篡改代码指针”完成劫持的两段路径。装载时随机化只在启动时改变一次布局，此后不再变化，攻击者可以反复探测并逐步恢复整个映像的布局，KASLR 无法使已经泄露的信息失效。

持续随机化把泄露地址的有效期压到攻击完成时间之下，其强度由随机化间隔与覆盖范围两个参数决定。Shuffler^[32] 在用户态每隔 50 ms 重排一次代码，而当时报告的 JIT-ROP 攻击完成时间为 2.3–378 s；Ahmed 等人^[33] 测量 gadget 的收敛速度后估计，要使攻击者难以在一个周期内集齐图灵完备的 gadget 集合，间隔上界约为 1.5–3.5 s。间隔越短，泄露信息可用的时间越短；覆盖范围越大，布局长期不变的代码越少；二者都以代码索引维护、执行状态协调与系统兼容的成本为代价。现有方案通过限制频率或范围控制这一成本，面向全部内核代码的持续随机化仍缺少可部署的实现。下一节先说明现有方案把范围限定在可加载内核模块所留下的盲区有多大，再由此提出本文要解决的问题。

1.2 研究问题与面临的挑战

将持续随机化用于内核时,主要困难来自运行时成本和执行正确性。内核执行环境包含系统调用、中断和异常等多种上下文,对延迟、稳定性和性能开销均有较严格的要求。持续随机化的成本主要集中在代码索引更新和执行流处理两个方面。代码位置发生变化后,系统需要识别并修正仍然指向旧地址的引用;对于规模较大且广泛使用代码指针的内核,这一过程可能涉及大量索引。与此同时,在旧代码被替换或解除映射之前,系统还需确认没有执行流继续使用该代码。内核中的中断、异常及其他非抢占上下文无法通过普通线程调度机制直接暂停,这进一步增加了执行状态协调的难度。

受上述成本限制,已有内核持续随机化研究通常只覆盖部分内核代码。*Adelie*^[34]、*Chaos*^[35]、*Remix*^[36]和*Dbox*^[37]等方案主要将驱动程序或可加载内核模块(Loadable Kernel Module, LKM)作为随机化对象。这些组件中包含较多第三方代码,其开发和测试过程可能不同于核心内核,因此常被选作优先保护对象。

这种保护范围主要反映了工程实现上的取舍,不意味着 **built-in** 内核代码不需要防护。以 *Adelie* 为例,其作者指出,对整个内核实施重新随机化需要处理核心内核中的遗留代码和底层实现,工程规模较大,因此该方案将持续随机化集中于被认为更容易受到攻击的组件^[34]。这一设计降低了实现和运行成本,但没有解决 **built-in** 代码的持续随机化问题。

评估这一范围划分是否合理,需要考察两个相互独立的方面:其一是内核代码的可模块化属性,即由构建体系决定的结构性边界;其二是漏洞在不同代码区域中的实际分布,即上述边界是否与安全风险相符。

由于现有方案以可加载内核模块为保护对象,一段代码能否纳入其保护范围,首先取决于该代码能否被编译为模块。代码的可模块化属性由 Linux 内核构建系统在源码层面确定,并不随具体部署配置而改变。

在 Linux 的 Kbuild 体系中,目标文件的编译方式由所在目录 Makefile 中的赋值形式及相应的 Kconfig 配置符号共同决定。`obj-y` 表示目标文件无条件编入内核映像;`obj-$(CONFIG_X)` 则根据配置符号 `CONFIG_X` 的取值决定编译方式。若该符号在 Kconfig 中声明为 `bool` 类型,其取值仅包括 `y` 和 `n`,相应代码只能被编入内核映像或不参与编译;只有声明为 `tristate` 类型的符号才允许取值 `m`,相应代码因而具备编译为模块的可能。

由此可以得到一个不依赖具体内核配置的判定标准:若某个源文件在所有合法配置下均不能被编译为模块,则该文件始终处于现有部分随机化方案的保护范围之外。对 Linux 内核源码树的分析表明,多个核心子系统的大量代码均符合这一条件:

- `kernel/` 目录的 Makefile 中没有 `obj-m` 条目。调度器 (`kernel/sched/`)、锁与 RCU (`kernel/locking/`、`kernel/rcu/`)、中断处理 (`kernel/irq/`) 以及时间子系统 (`kernel/time/`) 的相关目标文件,或者直接列入 `obj-y`,或者由 `bool` 型配置符号控制,均不能被编译为模块。

- mm/ 目录的 Makefile 中同样没有 obj-m 条目。页面分配器 (page_alloc.c)、slab 内存分配器 (slub.c、slab_common.c)、页缓存 (filemap.c) 以及内存映射 (mmap.c、memory.c) 等组件均列入 obj-y。

- fs/ 目录中, VFS 核心层与具体文件系统采用不同的构建方式。VFS 核心文件, 包括 open.c、read_write.c、namei.c、file_table.c、dcache.c 和 inode.c 等, 直接列入顶层 obj-y; ext4/、xfs/ 和 btrfs/ 等具体文件系统则受 tristate 型配置符号控制, 可以被编译为模块。

上述构建关系表明, 负责进程调度、内存管理和 VFS 核心功能的代码主要以内建形式存在。仅以可加载内核模块为保护对象的随机化方案, 在机制上无法覆盖这些代码。

另一项检验依据是漏洞的实际分布。Linux 内核社区于 2024 年 2 月成为 CVE 编号机构 (CNA)。此后由内核社区发布的 CVE 记录包含具体的受影响源文件, 为按文件粒度分析漏洞分布提供了数据条件。

本文采用以下方法进行统计:

(1) 从内核社区维护的漏洞信息仓库中提取编号年份为 2024、2025 和 2026 的已发布 CVE 记录, 并解析每条记录所列的受影响源文件。

(2) 从受影响文件所在目录开始, 沿 Kbuild 目录层级向上追溯其构建依赖, 确定控制该目标文件及其上级目录的赋值语句和配置符号。直接列入 obj-y, 且构建依赖链仅包含 bool 型配置符号的文件, 被判定为只能内建; 若依赖链中存在 tristate 型配置符号, 则该文件被判定为可模块化。

(3) 对于涉及多个源文件的 CVE, 只要其中一个受影响文件被判定为只能内建, 便将该 CVE 归入“只能内建”类别。该判定表示漏洞涉及的代码至少有一部分不能被现有部分随机化方案覆盖。

上述判定依据源码中的 Kbuild 与 Kconfig 定义, 不以特定发行版的内核配置为前提。2024 年 1 月至 2026 年 8 月的统计结果见表 1-2。

表 1-2 Linux 内核 CVE 所涉代码的可模块化性

Table 1-2 Modularizability of the Code Involved in Linux Kernel CVEs (2024.1—2026.8)

编号年份	CVE 总数	只能内建	可模块化	未判定
2024	3 064	713 (23.3%)	1 949 (63.6%)	402 (13.1%)
2025	2 668	610 (22.9%)	1 778 (66.6%)	280 (10.5%)
2026*	4 973	987 (19.8%)	3 568 (71.7%)	418 (8.4%)
合计	10 705	2 310 (21.6%)	7 295 (68.1%)	1 100 (10.3%)

注: 2026 年为部分年份, 数据截至 2026 年 8 月 8 日的仓库快照。

统计期内共收集到 10 705 个 Linux 内核 CVE, 其中 2 310 个涉及在所有内核配置下均无法编译为模块的代码, 占总数的 21.6%。若排除构建方式无法确定的记录, 在其余 9 605 个 CVE 中, 该比例为 24.0%。对于这些 CVE, Adelie、Chaos

和 Dbox 等仅保护模块或驱动的方案无法覆盖其中的内建代码。该缺口由代码的构建属性决定，不能通过调整内核配置消除。

将构建方式与缺陷类型交叉统计后可以看到，在 8 703 个内存安全缺陷 CVE 中，有 1 806 个涉及只能内建的代码，占 20.8%。这两类缺陷可为越界访问、释放后使用及后续控制流劫持提供基础原语，而现有部分随机化方案无法保护其中以内建形式存在的代码。因此，将随机化范围限定于模块和驱动，会遗漏一部分与控制流劫持直接相关的漏洞。

按子系统看，只能内建的 CVE 主要分布在 kernel/（413 条）、mm/（207 条）、net/core/（121 条）、net/ipv4/（109 条）、arch/x86/（84 条）、security/（74 条）、io_uring/（61 条）与 block/（60 条），正是本节开头所列举的那些核心子系统。

上述分析表明，有必要将 **built-in** 内核代码纳入持续随机化的保护范围。这些代码承担着内核的基础功能，并广泛分布于系统的高频执行路径。例如，VFS 核心文件 fs/open.c 和 fs/read_write.c 包含大量文件相关系统调用的公共处理逻辑；调度器管理系统中的进程执行；内存分配器则由各内核组件共同使用。这些区域中的漏洞一旦被利用，可能影响系统的整体安全性与稳定性。因此，如何覆盖 **built-in** 代码构成了本文研究的基本问题。

持续随机化需要回答三个问题：如何随机化 **built-in** 代码，何时随机化其中的哪些函数，以及随机化区域之外的代码如何保护。研究内容一、二、三依次回答这三个问题。随机化机制是后两者的运行基础，触发策略决定随机化区域在运行期如何伸缩，边界保护则面对任何部分随机化都会留下的非随机化区域。三项内容各自成立：第 4 章与第 5 章所依赖的随机化原语在文献中已有现成方案，本文的原型由第 3 章提供。

当保护范围由内核模块扩展至 **built-in** 代码后，上述三个问题分别面临新的技术约束，同时受到一项共同问题的影响。该问题决定了三项研究内容之间的接口与配合方式，因而有必要首先加以分析。

1.2.1 共性挑战：built-in 代码缺乏明确的随机化边界

现有部分随机化方法利用驱动与核心内核之间的接口划分随机化区域。驱动只经导出符号表调用内核，内核只经注册的操作表回调驱动，全部跨界引用可以收拢进一张表，跨边界控制流集中于数量有限的入口，因而可以通过监控这些入口追踪执行状态并验证控制流转移。

Built-in 代码之间缺少可供利用的边界。核心子系统的调用关系相互交织，入口随执行上下文变化。调度器既可能从系统调用路径进入，也可能由中断触发；VFS 在处理文件操作时进入具体文件系统，并可能继续调用块层或网络层；内存分配器可由几乎所有内核组件调用，调用者可能处于进程、中断或原子上下文。按照源码目录或功能子系统划分随机化区域，得不到封闭且稳定的控制流边界。

边界不明确会影响持续随机化的三个环节。随机化机制需要知道哪些引用指向被搬移的代码，以及哪些执行流仍停留在其中；触发策略需要知道随机化区

域的当前地址存放在何处，才能观测对它的读取；边界保护需要一个所有跨区域控制流都经过的位置，才能施加验证。

本文以跳板函数统一处理这一问题。每个被随机化的函数对应一个位置固定的入口跳板，被随机化代码调用的每个外部函数对应一个出口跳板，所有跨区域控制流都经过跳板；函数体的当前地址只存在于一条直接分支指令的立即数中，本文称这条指令为跳转桩，随机化时唯一改写的就是它。由此形成的边界具有三种用途：集中维护与代码位置有关的索引，观测进入和离开随机化区域的执行流，以及施加跨区域控制流验证。第 3 章给出跳板机制及其随机化实现，第 4 章在跳转桩区上施加只执行保护以识别泄露，第 5 章在跳板处施加分区验证。三项研究内容由同一边界机制衔接。

1.2.2 挑战一：代码索引更新与执行流协调

代码随机化会改变函数及指令的位置，所有依赖原代码地址的信息都必须随之调整，包括代码段中的直接调用与跳转、数据段中的函数地址、栈中的返回地址，以及全局偏移表等结构中的相关条目。本文将这类与代码位置相关、需要在随机化过程中维护的信息统称为代码索引（code index）。在 **built-in** 代码上处理代码索引，受到更新规模、地址语义、返回地址与执行流协调四方面的限制。

传统重定位方法需要修正代码段中的直接调用与跳转指令，追踪数据段中的函数地址，对活动线程进行栈展开并重定向返回地址，同时重新解析符号及更新相关链接表。若以需要处理的代码引用数量为 n ，一次随机化的更新开销为 $O(n)$ 。**built-in** 代码之间的引用密集，且散布在代码、数据与 **per-CPU** 区域中，全量更新难以承受高频随机化。

地址语义进一步增加了索引维护的难度。内核 **C** 代码允许在指针与整数之间转换，函数地址也可能参与比较、哈希或数值运算。以保存函数地址的变量 x 为例，当该变量被当作数值使用时，随机化不应改变其数值语义；当它被解引用或用于控制流转移时，其中的地址又必须指向函数的新位置。因此，仅凭赋值操作无法确定该值后续将以何种语义使用。在赋值阶段将函数地址替换为跳转表下标会改变原有数值语义；若推迟到使用阶段再进行地址转换，则需要在运行时查询映射关系，并覆盖内核中多种形式的间接分支目标，映射规模与查询开销都会随之增加。

栈上的返回地址作为代码索引同样需要处理。它由处理器在调用时自动写入，指向函数体内部的当前位置。一个随机化函数在调用其它函数期间，其返回地址就在栈上，内核栈因而成为一份随时可读的布局副本；若该函数在此期间被搬移，返回时便落到已失效的位置。逐条内核栈展开并改写返回地址，又与更新规模的约束相冲突。

执行流协调是代码更新面临的另一项约束。随机化期间必须保证没有执行流仍在待移动的代码中，但核心内核路径包含大量非抢占上下文，如中断处理程序、持有自旋锁的临界区、**RCU** 读端临界区和显式关闭抢占的代码段。基于调度器

的暂停方法不能直接停止这些执行流。例如，Adelie 使用 `schedule_on_each_cpu()` 并等待 RCU 宽限期，但 RCU 宽限期主要用于等待既有读端临界区结束，不能单独确认中断处理程序或持锁代码已经离开随机化区域。强制暂停持有自旋锁的执行流还可能使其他 CPU 长时间等待该锁，甚至形成死锁。已有的用户态持续随机化方案以保留旧代码副本回避这一约束，在内核中旧副本意味着已泄露的地址在切换之后仍然可用。

1.2.3 挑战二：随机化的触发时机与作用对象

现有方案多采用固定时间间隔触发随机化。缩短间隔可以压缩地址泄露后的利用窗口，但会增加随机化次数和运行时开销；延长间隔虽然降低了开销，却为攻击者保留了更长的攻击时间。第 1.1 节引用的实验结果将有效随机化间隔限定在秒级，而内核全量随机化的单次成本使这一频率难以长期维持。

随机化的范围同样受制于开销。以跳板为间接层的方案中，一次调用只要有一端落在随机化区域内就要经过跳板：随机化函数占全部函数的比例为 p 时，经过跳板的调用占全部调用的比例为 $2p - p^2$ ，小范围时约为 p 的两倍。第 3 章的实测给出了这一杠杆在真实内核上的形式：随机化不到 1% 的函数时常驻开销为 1.98%，扩到 40% 的 built-in 代码时为 28.83%。随机化因而只能限定在一个子集内，而信息泄露却可能发生在内核的任何位置。固定周期还会对未受探测的代码重复执行随机化。一次随机化只有发生在代码地址被探测之后、泄露地址被用于控制流劫持之前，才能直接破坏当前攻击所依赖的布局；对未被探测的代码以同等频率随机化，只产生开销而没有相称的安全收益。

触发式随机化需要一个直接来自信息泄露的信号。只执行内存能够把对代码的读取变为硬件事件，但现有的内核只执行内存方案都是防护式的。内核持续地读取自己的代码：动态跟踪设施在插入探测点时读取原指令，指令替换与静态键在启动期读取并改写代码，模块加载器与实时补丁在应用重定位时读取代码，性能分析工具在栈回溯与符号解析时读取指令流。防护式方案必须为这些合法读取者维护一份完备的豁免清单，漏掉任何一条都会使内核在该路径首次执行时崩溃。由此产生三项挑战：如何使只执行保护不依赖一份必须完备的豁免清单；如何把读取事件转化为可靠的触发判据，并使从触发到完成的延迟短于攻击者可利用该次读取的时间；如何使常驻开销与随机化范围的规模解耦，使候选范围可以扩大到全内核而开销不随之上升。

1.2.4 挑战三：非随机化区域中指针认证上下文的传播

受开销所限，现有的内核持续随机化方法全部只能做部分随机化。Adelie、Chaos 与 Dbox 以模块与驱动为界，触发式随机化以泄露事件为界，随机化区域之外的代码布局在运行期不变。攻击者一旦持有指向这些代码的合法指针值，它就长期可用，仍可利用其中的 gadget 构造 ROP 或 JOP 链。非随机化区域的保护因此是所有部分随机化方法共同面对的问题。

如第 1.1 节所述，硬件辅助 CFI 可以为非随机化区域提供补充保护。商用硬

件机制中，**ARM** 指针认证是唯一能以软件选取的上下文区分调用点的一种，原理上可以为每个调用点取不同的值，但受上下文传播约束：多个调用点调用同一函数时必须共享上下文，该约束沿调用图传播，使原本独立的调用点逐渐归入同一上下文。在 **iOS 13** 中，约有 6 500 个调用点共享同一上下文^[16]。共享范围扩大后，通过同一认证的合法目标随之增加，间接调用检查的区分能力下降。

上下文传播由高扇入（**high fan-in**）函数引起。此类函数可由多个调用点到达，把原本相互独立的认证上下文连成一片。内核 **C** 代码广泛使用指针别名、间接调用、函数指针数组和动态回调注册，静态分析难以完整恢复这些函数的调用来源。由此产生三项挑战：如何在不做全局指针分析的前提下识别高扇入函数并让其余调用点得到唯一上下文；如何避免在每一次间接调用上都施加验证，使开销随随机化范围可控；如何使保护机制不绑定某一种随机化实现，从而可以施加到任何一种部分随机化方法留下的非随机化区域上。

1.3 研究内容与主要贡献

针对上述挑战，本文提出并实现了一套面向 **Linux** 内核 **built-in** 代码的持续随机化系统。该系统以随机化区域与非随机化区域的划分为基础，跨区域的控制流被收束到跳板这一唯一出入口，本文的三项工作都建立在这一划分之上。系统的总体设计目标有三：

(1) 不引入新的硬件设计。如第 1.1 节所述，已有的硬件随机化方案都需要修改处理器，无法用于现役系统。本文只使用现役商用处理器已经提供的部件：**x86** 的管理态保护键、**ARM** 的硬件调试观察点与 **ARM** 指针认证。三者都被用在其原本用途之外的场合。保护键原本用于内核内部的数据隔离，本文用它把跳转桩区置为只执行；观察点原本用于调试，本文用它在无虚拟化层的条件下捕获对跳转桩区的读取；指针认证原本用于指针完整性，本文用它保护随机化区域之外的控制流。

(2) 不修改被保护子系统的源码语义。改动集中在编译器与随机化运行时，被随机化的内核子系统不需要为适配本文机制而改写代码逻辑。

(3) 开销可承受。在真实负载下的端到端开销控制在个位数百分比。

本文的三项研究内容依次对应第 3、4、5 三章。三者由同一边界机制衔接，但各自可以独立成立：第 4 章与第 5 章各自只对随机化原语提出一项要求，该原语在文献中已有现成方案。图 1-2 以内核持续随机化为研究方向，自上而下列出由它分解出的三个问题、每个问题面临的关键挑战，以及与之对应的三项研究内容；虚线箭头表示后一项研究内容建立在前一项之上。

1.3.1 研究内容一：基于跳板函数的低开销持续随机化

要回答的问题：如何随机化 **built-in** 代码。对应第 3 章。

跳板本身不是本文提出的。**Oxymoron**^[38] 以跳板替换函数指针，**Shuffler** 以索引表实现间接，**MARDU**^[39] 与 **CodeArmor**^[40] 同样引入了地址间接层，把跳板

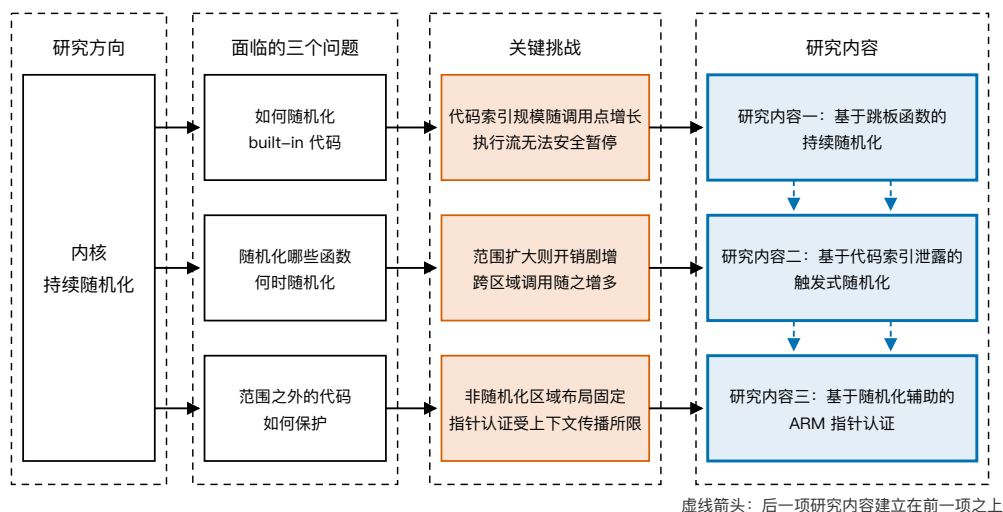


图 1-2 本文的研究思路与三章分工

Figure 1-2 The Research Rationale of This Dissertation and the Division of Labour across Chapters 3 to 5

用作随机化时只需更新一处的手段是这一领域的通行做法。本文的贡献在于跳板集中的是什么以及跳板还能做什么。

本文把代码索引分层处理。函数粒度内以位置无关代码消除索引，函数体可以被搬到任意地址而无需修改其内部的任何一条指令；函数粒度间以固定位置的跳板集中索引，函数体的当前地址只存在于跳转桩这一条直接分支指令的立即数中，一次随机化的改写量等于函数个数而与调用点数量无关。函数指针的值恒为跳板地址，内核中把函数地址当作数值比较、哈希与反查的代码模式不受影响，数值还是地址的判定被推迟到了控制流转移发生的那一刻。栈上的返回地址由与随机化无关的返回标记取代，标记只含跳板序号与函数内偏移，经重入桩解析到当前变体，内核栈不再承载当前布局，陈旧返回不再发生。

本文还扩展了跳板的职责。已有工作把跳板当作纯粹的地址间接层，本文把它同时用作随机化区域边界上的执行流观测点：入口与出口计数精确给出当前正在区域内执行的执行流集合，据此确定安全的暂停点，随机化不必保留任何旧代码副本，切换完成的那一刻旧地址就真正过期；对中断、异常与关抢占区间这类无法被调度器暂停的上下文，采用推迟而非阻塞的策略。

主要贡献：

(1) 提出基于函数粒度位置无关代码与跳转桩的分层代码索引组织方法，使一次随机化的改写量与被引用次数无关，并以返回标记消除栈上的布局副本与陈旧返回；

(2) 提出函数指针语义分离方法，在不改变指针值的前提下保证所有跨区域控制流转移必经跳板；

(3) 提出基于跳板进出计数的精确执行流追踪与推迟随机化策略，实现无

代码副本的随机化并覆盖非抢占上下文。在 483 个函数的随机化范围下，单次随机化的停顿窗口为 1.22 μs ，等待执行流离开当前函数体的时间在 0.04 至 0.18 μs 之间；在与 Adeline、Chaos 和 Dbox 的同条件对比中，本文机制在各档随机化间隔下的开销均最低。

1.3.2 研究内容二：基于代码索引泄露的触发式随机化

要回答的问题：何时随机化哪些函数。对应第 4 章。

本文的关键观察是：随机化区域的当前地址只存在于跳转桩这一处，对它的读取就是泄露本身，不需要任何代理。据此本文把只执行内存的保护对象由整个代码区收窄为跳转桩区，并把它从阻断机制改用于检测机制。正常执行到达函数体的路径全部由直接分支构成，只被取指；内核的合法读取者读的是函数体的指令而非跳转桩；落在跳转桩区上的任何数据读取因而都没有合法解释，机制不需要为任何读取者维护豁免清单。读取事件驱动被读函数进入随机化，随机化范围由编译期静态选定改为运行时由泄露事件触发。

把只执行内存用作检测而非阻断并非本文首创。Heisenbyte^[41] 与 NEAR^[42] 采用允许读取但使读到的代码不可用的思路，SafeHidden^[43] 以探测检测驱动用户态数据区域的重随机化。本文的不同在于保护对象与响应方式：保护对象是跳转桩区而非代码本身，合法读取者与被监控对象由此天然分离；响应是一次针对性的随机化，不改变被读代码的任何语义；机制实现在内核态。

常驻开销与候选范围的解耦由两级跳板完成。候选函数的一级跳板在未激活态只是一条到跳转桩的直接分支，不做任何记账，一次往返约 5 个周期；被读取的函数才激活为完整的跳板，约 54 个周期。激活是一次 4 字节的原子写入。候选集合因而可以扩到全内核，每次调用只为被读过的函数付完整的代价。

主要贡献：

(1) 提出以跳转桩区的读取事件作为持续随机化的触发信号，使随机化由泄露驱动而非由时钟驱动，且不依赖任何豁免清单；

(2) 提出两级跳板与跳板激活机制，使常驻开销与随机化的候选范围解耦；

(3) 给出内核态只执行内存的两条实现路径，x86 平台基于管理态保护键，ARM 平台基于硬件调试观察点，两者都只作用于数据访问而对取指透明，放行一次读取只影响本核。单次检测开销在 ARM 观察点路径上为 1.32 μs 、在 x86 保护键路径上为 0.96 μs ，基础负载下内核自身的读取每秒触发 0.02 至 0.44 次，从检测到随机化完成的延迟在基础负载下中位为 12 至 21 μs 、99 分位不超过 110 μs 。

1.3.3 研究内容三：基于随机化辅助的 ARM 指针认证机制

要回答的问题：非随机化区域怎么办。对应第 5 章。

本文的关键观察是：随机化区域内的目标地址不可预测，进入该区域的间接调用不需要指针认证，随机化因此恰好提供了打断上下文传播链所需的東西。把造成传播的高扇入函数移入随机化区域，链两侧的调用点即可各自建立基于

调用点地址的唯一上下文。高扇入函数的识别在运行时完成：指针在识别阶段以类型上下文签名，归属由独立记录跟踪，一个函数第二次被不同的调用点调用即判为高扇入并移入随机化区域，单一归属的函数则在上下文绑定时重签为调用点唯一上下文，全程不需要全局指针分析。验证在跳板处分区施加：进入随机化区域只验范围，离开时强制认证与白名单检查。该机制对随机化原语只有一项要求，即被识别为高扇入的函数持续处于随机化状态，因而可以施加到任何一种部分随机化方法留下的非随机化区域上。

主要贡献：

(1) 提出以随机化打断指针认证上下文传播链的协同机制，使随机化与控制流完整性互为条件；

(2) 提出基于运行时归属跟踪的高扇入函数识别与上下文绑定方法，不依赖全局指针分析；

(3) 提出跳板处的分区验证，并说明该机制与具体随机化方法解耦。最大等价类由类型上下文下的 141 降到 57，91% 的间接调用点合法目标数不超过 5，代价是把 233 个高扇入函数移入随机化区域；施加到 Chaos 之上，开销增量为 0.7%。

1.3.4 三项工作的关系

三项工作对应持续随机化在内核场景下的三个问题：研究内容一压低单次随机化的代价并覆盖 **built-in** 代码，使高频随机化成为可能；研究内容二取消固定频率，使随机化只在泄露发生时、只对被读过的函数发生；研究内容三承认部分随机化必然留下非随机化区域，以指针认证补齐随机化够不到的部分。三者通过跳板耦合：研究内容一构造它并把代码索引集中到跳转桩，研究内容二保护跳转桩并在跳板上切换激活状态，研究内容三在跳板处施加跨区域验证。每一章的问题都由上一章的边界逼出：跳板的常驻开销限定了随机化范围，触发式随机化因此只随机化被读过的函数，而随机化区域之外的部分交给控制流完整性。

本文在 Linux 6.8 内核与 LLVM 14 上实现了完整原型，主平台为 ARM Neoverse N2，副平台为 Intel Xeon Gold 6448Y。第 3、4、5 章各自给出机制级评估与安全性分析，与 Adeline、Chaos 和 Dbox 的同条件对比见第 3.5.4 节，第 5 章机制在 Chaos 上的移植验证见第 5.5.4 节。

1.4 本文的安全性论证方式

防御机制的有效性不像性能那样可以直接测量，如何论证是一个方法学问题。本文在此明确声明所采用的证据层级，以及每一层的适用范围——**不同性质需要不同强度的证据，把只能被测量的性质写成定理，反而会削弱论证。**

本文的安全性论证由四层证据构成：

第一层，可机械验证的结构性质。少数性质可以被完全验证，本文对这类性质给出严格论证并附验证程序。核心是一条：编译产物中不存在绕过跳板函数的

跨区域控制流转移。该性质可由编译期的静态检查与对内核映像的反汇编扫描共同确认，是本文全部安全论证的基础——后续各层的论证都以它成立为前提。该检查由编译器插件在编译期执行，并以对内核映像的反汇编扫描作为构建后的复核，实现见第 3.4.1 节。

第二层，概率与熵的定量分析。随机化类防御的强度本质上是概率性的，因此以概率模型而非定理来刻画。本文为攻击者在给定探测次数下的成功概率建立模型，并遵循一条原则：模型中所有简化假设都必须朝有利于攻击者的方向取值，从而使所得概率成为安全性的下界而非估计值。每条假设在提出时都需说明它为何是保守的。认证码伪造这一项的概率界见第 5.6 节。这一做法与该领域的通行实践一致——例如 *Adelie* 通过地址空间熵的直接计算给出攻击成功概率的上界，并与 *Shuffler*、*CodeArmor* 作横向对比。

第三层，攻击复现实验。检测覆盖率、误报漏报率、随机化触发的及时性，这些性质无法演绎，只能测量。本文以真实 Linux 内核提权漏洞为攻击案例，把其已公开的利用路径按“获取原语、泄露布局、收集串联、劫持控制流”四步分解，在启用与未启用各档机制下逐步骤观测拦截结果，记录攻击在哪一步被拦截、拦截前攻击者来得及读取多少代码片段、以及触发拦截所需的探测次数分布。需要说明攻击案例与攻击驱动的分工：案例是真实的，即真实漏洞与其真实的利用步骤；而落在防御范围内的步骤由内核内受控自测忠实复现该步骤的行为，获取原语这一步则按威胁模型直接赋予，本文不实现、也不运行任何向用户态暴露内核任意读写的武器化载体。这样做既守住不引入两用攻击代码的底线，又使被拦截行为的起点在每次实验中完全一致，从而可把机制的响应与利用本身的偶然性分开。第 4.5.4 节以内核内受控自测复现“泄露地址至解引用”这一步骤，给出各时延档位下的攻击成功率；第 5 章机制不涉及该步骤的复现，其有效性由等价类规模的度量给出。

第四层，残余风险的显式声明。本文明确声明防御边界之外的部分，包括纯数据攻击、硬件侧信道、以及依赖硬件规范正确实现的假设。特别地，ARM 指针认证已被证明可通过推测执行侧信道在不触发崩溃的情况下被爆破^[44]，因此研究内容三的安全性论证以“攻击者无法获得 PA 验证预言机”为前提，该前提及其脆弱性在第 5.6 节单独说明。

本文不采用形式化定理的呈现形式。原因是：本文的安全性质中，只有第一层可以被真正证明，第二层是概率陈述，第三层是经验测量；将后两者以定理形式呈现，会把“在给定假设下的度量结果”误示为“演绎结论”，一旦某条假设在答辩或后续研究中被质疑，整条论证链的可信度会一并受损。

1.5 论文组织结构

本文共分 6 章。每一章对应一项独立的工作，各章自足：各自给出研究动机与设计需求、系统设计、实现细节、机制级实验与安全性分析。跨越三项工作的内容，即统一的威胁模型与安全不变式，放在第 2 章。

第 1 章绪论：从内存安全攻防的坐标系出发，说明随机化路线的位置与硬件 CFI、内存隔离、硬件随机化各自的边界；通过内核构建体系的结构事实与 CVE 分布的统计事实，提出 **built-in** 内核代码的保护盲区问题；给出持续随机化在内核场景下的核心矛盾，分析一个共性挑战与三个专属挑战；概述研究内容、主要贡献与安全性论证方式。

第 2 章相关研究综述与威胁模型：综述内存安全攻防方法的演进、内核随机化的三个发展阶段、持续随机化三个核心问题上的已有解法、**XOM** 技术与硬件 CFI 技术的现状，并指出各自的不足与本文的出发点；在此基础上给出本文统一的威胁模型，定义攻击者能力、四类攻击向量、信任基础与明确的排除项，并由威胁模型导出三条贯穿全文的安全不变式。**后续各章的安全性分析均引用本章定义的威胁模型与不变式。**

第 3 章基于跳板函数的低开销持续随机化方法：研究内容一。本章首先分析代码索引的规模、返回地址的语义与执行流暂停三个层次上的困难，然后给出跳板函数的指令序列、函数粒度的位置无关代码、以位置无关返回标记替代裸返回地址，以及以进出计数刻画活跃执行流的安全暂停，随后给出 **LLVM** 编译器扩展与内核运行时的实现，最后评估随机化延迟、随机化核心子系统的开销、与已有工作的同条件对比，以及随机化范围与跳板调用占比的关系。**跳板函数作为后两章的共同基础设施，其完整设计在本章给出。**

第 4 章基于代码索引泄露的触发式随机化：研究内容二。本章首先分析固定间隔触发策略的两难与保护整个代码区域的代价，然后给出把只执行内存由阻断机制改用为检测机制的核心思想、只保护跳转桩区这一收窄后的监控对象，以及未激活与激活两态的两级跳板，随后给出 **x86** 平台基于管理态保护键与 **ARM** 平台基于硬件调试观察点的两条实现路径，最后评估两条路径的单次检测开销、各类负载下的触发频率与延迟、泄露地址的有效期与利用竞态，以及两态下的单次调用开销。

第 5 章基于随机化辅助的 ARM 指针认证机制：研究内容三。本章首先分析部分随机化留下的非随机化区域与指针认证的上下文传播问题，然后给出以类型上下文配合归属跟踪在运行时识别高扇入函数、把高扇入函数移入随机化区域以打断传播链、并在跳板处施加分区验证的设计，随后给出编译器插桩与运行时实现，最后以每个间接调用点的合法目标数评估精度，评估识别的收敛性，并把该机制移植到已有的部分随机化方法上。

第 6 章总结与展望：总结全文工作与创新点，讨论局限性与残余风险，展望后续研究方向。

第 3、4、5 三章采用统一的内部结构：引言与本章导航 → 现有工作与研究动机（以设计需求清单收尾）→ 系统设计 → 系统实现 → 机制级实验评估 → 安全性分析 → 本章小结。各章的实验只回答“本章机制是否达到了本章开头列出的设计需求”，同一组基准测试数据不在多章之间重复报告。

第 2 章 相关研究综述与威胁模型

本章分两部分。第一部分（第 2.1 至 2.5 节）综述相关研究：先给出内存安全攻防方法演进的整体图景（第 2.1 节），然后按内核地址空间布局随机化的三个发展阶段梳理这条路线的现状（第 2.2 节），随后针对持续随机化必须回答的三个核心问题分别综述已有解法及其不足（第 2.3 节），最后介绍本文两项关键技术手段的研究现状，即只执行内存（第 2.4 节）与硬件辅助的控制流完整性（第 2.5 节）。第二部分（第 2.6 至 2.7 节）在综述的基础上给出本文统一的威胁模型，并由威胁模型导出三条贯穿全文的安全不变式。

后续第 3、4、5 章的安全性分析均引用本章第 2.6 节定义的威胁模型与第 2.7 节定义的不变式。

2.1 内存安全漏洞攻防方法的演进

第 1.1 节已经给出内存安全的攻击模型，见图 1-1^[45]。本节沿这一模型展开攻防演进的细节，为后续各节提供共同的背景。

最早的内存破坏攻击直接向进程内存中写入可执行的机器码。攻击者利用栈缓冲区溢出覆写返回地址，使其指向注入到缓冲区中的 `shellcode`，函数返回时即执行攻击载荷。此类攻击对应图 1-1 中“篡改代码 → 执行注入的代码”这条路径。

防御方的回应是数据执行阻止（DEP）^[46]：借助页表中的不可执行位（x86 的 `NX`、ARM 的 `XN`），使数据页不可执行、代码页不可写，任一内存页不同时具有可写与可执行权限。在内核态，与之对应的机制是内核代码段只读、SMEP（Supervisor Mode Execution Prevention）与 ARM 的 `PXN`（Privileged eXecute Never）。后两者阻止内核态执行用户态页面上的代码，封堵了“把 `shellcode` 放在用户态内存、诱使内核跳转过去”这一内核特有的注入路径。

如第 1.1 节所指出的，这些机制在内核中的性质与在用户态不同：它们由内核自己建立和维护，攻击者获得内核态任意写能力之后同样可以关闭。因此内核态的 DEP 类机制提高了攻击成本，却不构成攻击者无法逾越的边界。

代码注入被封死之后，攻击者转向复用程序自身已有的指令片段。Shacham 提出的返回导向编程（ROP）^[8] 以 `ret` 结尾的短指令序列（`gadget`）为基本单元，通过在栈上布置一系列 `gadget` 地址将它们串联执行，并证明在足够大的代码库中这一构造具有图灵完备的表达能力。其后的变体沿两个方向扩展：

改变串联手段。跳转导向编程（JOP）^[47] 以间接跳转指令结尾的片段替代 `ret`，摆脱了对栈的依赖；面向调用的编程（COOP）^[48] 以虚函数调用为串联手段，使攻击载荷在控制流图上看起来仍然合法，从而绕过粗粒度的控制流完整性检查。

改变载荷构造时机。传统 ROP 要求攻击者在发起攻击前就知道 gadget 的地址；即时 ROP (JIT-ROP)^[31] 把 gadget 的发现推迟到运行时，详见第 2.1.4 节。

在内核场景下，代码重用攻击的可用素材远比用户态丰富。本文对内核镜像做了一次 gadget 普查作为基线，结果如表 2-1 所示。

表 2-1 Linux 内核镜像中可利用 gadget 的分布

Table 2-1 Distribution of Usable Gadgets in the Linux Kernel Image

类别	终止指令	数量	占镜像中全部候选序列
ROP	ret	30 148	2.6%
ROP	ret imm	77 067	6.5%
JOP	jmp reg	3 015	0.3%
JOP	jmp [mem]	18 009	1.5%
COP	call reg	3 515	0.3%
COP	call [mem]	15 825	1.3%
SYS	syscall / int	697	0.1%
可利用小计		148 276	12.6%

按大类归并为经典 ROP 107 215 条、JOP 21 024 条、COP 19 340 条、系统调用类 697 条。其中价值最高的是 pop <reg> ; ret 一族，它把栈上由攻击者控制的值直接弹入寄存器，是构造攻击链时控制参数的核心原语；镜像中 pop rdi ; ret (控制第一个参数) 有 208 条。

这些 gadget 的数量在部署本文机制之后并不会减少，改变的是它们的地址在运行时是否稳定。本文的安全性评估据此设计，比较的是地址的有效期而非 gadget 的计数。

针对代码重用攻击的低开销缓解手段是地址空间布局随机化 (ASLR)^[46,49]：在程序装载时随机选择各段的基址，使攻击者无法预知 gadget 的绝对地址。ASLR 的随机熵取决于可用的地址范围与对齐约束，其随机化粒度是整段，段内各函数的相对偏移保持不变。因此攻击者只要泄露段内任意一个地址，就能推算出该段内所有 gadget 的位置。

为提高随机熵并消除这一弱点，研究者提出了一系列细粒度的装载时随机化方法，按粒度可分为四类：内存页粒度^[38]、函数粒度^[50,51]、基本块粒度^[52]与指令粒度^[53,54]。自动化软件多样化这一整体思路的设计空间与取舍已有系统梳理^[55]，其中面向真实产品的装载时实现以 selfrando 为代表^[56]。粒度越细，随机熵越高，攻击者从单个泄露地址推断其他地址的能力越弱；代价则是重定位信息的规模、装载时间与代码局部性的损失；后一项代价可由编译器在构建期保留必要的重定位信息来摊薄，使随机化在分发或装载时完成而不必在运行期重做静态分析^[57]。

这一类方法有一个共同前提：随机化只发生一次。因此它们的安全性完全依赖布局在整个运行期内不被泄露。

信息泄露攻击直接针对上述前提。按泄露手段可分为四类，本文在第 2.6.2 节将它们整理为四类攻击向量。

直接读取代码。 JIT-ROP^[31] 是这一类的代表：攻击者利用一次内存泄露获得任一代码指针，以该指针所在页为起点递归扫描代码页、反汇编并收集 gadget，最终在运行时构造出攻击载荷。它不需要预先知道完整布局，一个指针即可启动。

通过执行结果推测。攻击者向猜测的地址发起控制流转移，根据是否崩溃、以何种方式崩溃来判断该地址是否有效。此类方法的关键在于避免崩溃暴露攻击行为。用户态的典型手段有两种：Blind ROP^[58] 利用网络服务器工作进程崩溃后由守护进程重启且内存布局相同的特性完成持续试探；CROP^[59] 复用被攻击程序自身注册的异常处理函数来吸收非法访问异常。

这两种手段在内核场景下都不成立。Blind ROP 依赖 fork() 语义下父子进程内存布局相同这一前提，内核不存在与之对应的机制；CROP 依赖被攻击程序自行注册的用户态异常处理函数，内核的异常处理路径不由攻击者控制。因此在内核中，攻击者的执行试探会直接触发内核异常，其后果是崩溃或者被内核的异常处理路径捕获，二者都是可观测事件。这一差异对本文有利，是第 4 章检测机制得以成立的一个前提，在第 2.6.2 节定义攻击向量 V2 时会再次说明。

通过内存管理接口间接推断。攻击者反复申请与释放不同大小的内存块，根据分配是否成功推断虚存空间中未分配区域的分布，进而反推已分配区域的位置^[60]。此类方法不产生任何非法访存，隐蔽性最强。

通过微架构侧信道推测。利用缓存访问时序、TLB 状态、预取指令的行为差异等推断地址是否已映射^[61]。这条路线上的代表性工作包括基于地址翻译缓存时序的早期攻击^[23]、以事务内存抑制异常的 DrK^[24]、利用分支目标缓冲冲突的方法^[25]，以及在页表隔离部署之后仍然有效的预取侧信道^[26,27] 与针对内核数据结构侧信道的^[28]；瞬态执行攻击为这类泄露提供了更多来源^[29]。从随机化一侧抵抗侧信道的尝试亦有先例^[30]。近年针对内核地址空间的侧信道工作尤其活跃，例如利用带标签 TLB 打破内核地址空间隔离的 TagBleed^[62]，以及专门讨论 ASLR 在微架构攻击下如何加固的 Oreo^[63]。此类方法同样不产生异常，本文将其为排除项（第 2.6.4 节）。

上述第一类中的 JIT-ROP 最能说明信息泄露为何足以击穿一次性随机化，其攻击流程如图 2-1 所示。攻击者先借内存泄露漏洞得到任意一个代码指针，以它所在的代码页为起点，读取该页内容并反汇编；页内的直接跳转与调用指令给出了指向其他代码页的边指针，攻击者顺着这些边指针递归地读取、反汇编更多代码页，逐步搜集到足够的 gadget；最后用泄露到的真实地址把这些 gadget 拼接成攻击载荷。整个过程发生在程序运行时，一次性随机化在装载时确定的布局对它完全透明。

与随机化并行的另一条防御路线是控制流完整性（CFI）^[10]：静态确定每个间接控制流转移的合法目标集合，在运行时验证实际目标是否属于该集合。CFI

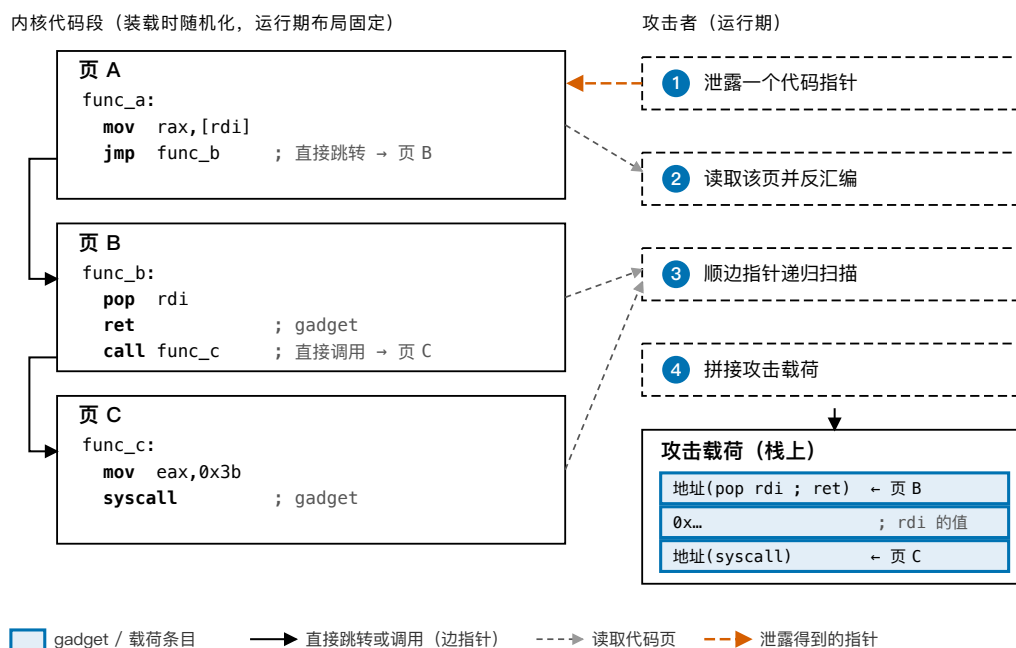


图 2-1 JIT-ROP 信息泄露攻击流程

Figure 2-1 Workflow of the JIT-ROP Information-Disclosure Attack

的安全强度取决于目标集合的精度，而精度受限于静态分析在 C 语言上的能力。Carlini 等人^[64]通过控制流弯折攻击证明，即便目标集合恰好等于静态可达集合，攻击者仍可沿合法的控制流边构造出恶意的执行序列。

更进一步，数据导向编程（DOP）^[65]完全不改变控制流，仅通过篡改数据实现图灵完备的计算，从原理上绕过所有 CFI 类防御。这类攻击对应图 1-1 中“篡改数据变量 → 纯数据攻击”这条路径（图中已灰显），明确不在本文的防御范围内，理由已在第 1.1 节说明，界定见第 2.6.4 节。

从上述演进可以看到一条清晰的线索：每一项防御机制在某个阶段封死一条路径，攻击者随之转向仍然通畅的路径。本文的工作位于图 1-1 中着色的那几条路径，即篡改代码指针、读取代码与代码索引，以及二者汇合而成的控制流劫持路径。所采取的路线是随机化，并以硬件 CFI 补齐随机化在结构上无法覆盖的部分。随机化这一路线的工程化程度与部署障碍已有系统性的梳理^[66]，面向内存泄露的运行期重随机化亦有综述可循^[67]。

2.2 内核地址空间布局随机化相关工作

将随机化引入内核的研究可以划分为三个阶段。三个阶段并非简单的时间先后，每一阶段都在弥补前一阶段被攻破的假设。第一阶段是一次性随机化，它假定代码布局在运行期不被攻击者获知，信息泄露与侧信道攻破了这一假设；第二阶段是持续随机化，它以周期性重排使已泄露的地址迅速失效，但全量重排的代价使其难以覆盖整个内核；第三阶段是部分随机化与触发式随机化，它通过限

制保护范围或触发时机把代价压到可接受的水平，代价是留下布局固定的非随机化区域。以下三小节依次讨论这三个阶段。

2.2.1 一次性随机化

面向内核的装载时随机化最早由研究工作提出^[68]；Linux 自 3.14 版本起在主线内核中提供 KASLR^[69]，在系统启动时随机选择内核映像的装载基址。这一机制的随机熵受两方面限制：可用于放置内核映像的虚拟地址范围有限，以及内核映像必须按大页边界对齐。二者共同决定了可选位置的数量。

除信息泄露之外，内核映像的基址还可由微架构侧信道恢复，这一方向的工作持续推进^[70,71]；即便完全没有信息泄露，攻击者也可构造位置无关的代码重用载荷^[72]。这一阶段的共同弱点是随机化只发生一次。第 2.1.4 节的任一类信息泄露手段一旦成功，防御即告失效，且不再恢复。

2.2.2 持续随机化

针对上述弱点，研究者提出在运行期周期性地重新排列代码布局。这一方向的工作主要在用户态展开，代表性系统包括 TASR^[73]、Shuffler^[32]、Remix^[36]、RuntimeASLR^[74]、CodeArmor^[40]、ReRanz^[75] 与 MARDU^[39]。另一条并行的思路不改变代码位置而随机化执行路径本身，以克隆函数并在每个调用点随机选择副本的方式挫败按地址预测的攻击链^[76]。此外还有 Mixr^[77] 等以二进制为对象的工作。它们的差异集中在三点，即何时触发随机化、如何更新代码索引、以及如何处理正在执行的控制流，将在第 2.3 节展开比较。

上述方案对代码索引的处理都依赖于把程序改造为可重排的形式，这一能力在无源码场景下由静态二进制重写工具提供，代表性工作以低开销的直接改写^[78] 与完全布局无关的重编译^[79] 为两条路线。Shuffler 的实验给出了这一阶段最重要的一个量化结论：其 50 ms 的洗牌周期远短于已知 JIT-ROP 攻击所需的 2.3 至 378 秒。Ahmed 等人随后的系统测量^[33] 把这个经验判断精确化为可复现的方法，通过测量 gadget 集合随扫描时间的收敛速度，给出随机化间隔的上界为 1.5 至 3.5 秒。本文第 4 章不采用这一以扫描时间反推间隔上界的口径，因为触发式随机化的时机由检测事件决定，而不由间隔参数决定。

这一阶段的困难在于代价。如第 1.2 节所述，代码索引更新与执行流暂停两项开销在内核中被显著放大，这直接导致了第三阶段的出现。

2.2.3 部分随机化与触发式随机化

为使持续随机化在内核中可行，近年的工作沿两个方向缩小代价。

缩小随机化的范围。 Adelie^[34,80] 只对可加载内核模块与设备驱动应用持续随机化。其技术路线包括：将模块编译为位置无关代码并为每个模块维护只读的全局偏移表；以 `schedule_on_each_cpu()` 配合 RCU 宽限期确保重随机化期间无执行流停留在被移动的代码中；以 20 ms 的固定周期由专用内核线程驱动重随

机化。其安全分析给出的攻击成功概率为 2^{-44} (考虑 4 KB 页对齐之后), 并与 Shuffler、CodeArmor 使用 32 位偏移所对应的 2^{-19} 作对比。

缩小随机化的时机与对象。另一条路线是放弃固定周期, 只在观测到实际的泄露行为时才随机化, 并把随机化的对象收窄到被泄露的那一部分代码。PtrProxy^[81] 在 AArch64 上采取这一思路, 以代码指针的实际泄露作为触发条件, 并用跳板承载指针代理, 跳板本身的泄露亦触发检测。这条路线的价值在于把代价与攻击行为绑定, 在无攻击的时段几乎不付出开销; 其难点则在于泄露行为的观测手段本身要足够廉价且足够完备。目前这一路线的工作均以用户态进程为保护对象, 尚无面向内核 **built-in** 代码的实现。

表 2-2 汇总了这两个方向上的代表性工作, 列的划分对应第 1.3 节提出的三个核心问题。

表 2-2 持续随机化代表性工作的横向对比

Table 2-2 Comparison of Representative Continuous Re-randomization Systems

系统	年份 · 会议	保护对象	粒度	何时随机化	如何更新代码索引	执行流处理	非随机化区域	报告开销
TASR	15 · CCS	用户态进程	函数	成对 I/O 系统调用之间	编译期标记, 运行时更新全部代码指针	系统调用边界	无	2.1%
Remix	16 · CODASP	用户态进程	函数内基本块	固定间隔	函数位置不变, 无需更新	无需暂停	无	2.8%
RuntimeASLR	16 · NDSS	用户态进程	模块	fork() 时	动态污点分析追踪指针	仅在 fork 点	无	记录阶段数百倍
Shuffler	16 · OSDI	用户态进程	函数	固定 50 ms	代码指针表下标	保留旧副本, 异步洗牌	无	14.9%
CodeArmor	17 · EuroS&P	用户态进程	整段	系统调用	地址空间抽象层	抽象层转换	无	6.9%
ReRanz	17 · VEE	用户态进程	基本块	I/O 字节累积量	进程级虚拟机跳板	虚拟机接管	无	6%
SafeHidden ^[43]	19 · USENIX Sec	用户态安全区域	区域	检测到试探	不适用	不适用	陷阱区域	2.75%
MARDU	20 · SYSTOR	用户态进程	函数	泄露时	跳板	保留旧副本	无	5.5%
Adelie ^[34]	22 · ASPLOS	内核模块与驱动	模块	固定 20 ms	PIC + 每模块只读 GOT	调度器 + RCU 宽限期	只读 GOT 约束跨模块指针	< 2%
Dbox ^[37]	24 · CCS	不可信驱动	整个驱动的虚拟空间	捕获到进入驱动的可疑控制流	不搬移物理代码, 只改虚拟地址映射, 因而无需更新索引	无需暂停	虚拟盒隔离	< 3% (CPU 负载 40% 以下)
PtrProxy	25 · China Comm.	用户态进程 (AArch64)	基本块	代码指针实际泄露	跳板承载指针代理	指针代理转发, 无可感知暂停	跳板泄露触发检测	1.52%
本文		内核 built-in 代码	函数	检测到代码探测	函数级 PIC + 固定跳板	跳板计数精确暂停, 无副本	随机化辅助的指针认证	见第 3、4、5 章

这一阶段的代价是范围。部分随机化带来两个后果: 其一, **built-in** 内核代码完全不受保护, 而第 1.2 节已经说明这部分代码既无法被这类方案覆盖, 又承

载了两成以上的内核 CVE；其二，非随机化区域的布局对攻击者而言是已知且固定的，本身成为新的攻击面（详见第 2.3.4 节）。

本文的出发点即在于此：把持续随机化扩展到 **built-in** 内核代码，同时处理由此产生的非随机化区域问题。

2.3 持续随机化的三个核心问题

如第 1.3 节所述，持续随机化机制必须回答三个问题。本节按这三个问题组织已有工作，每一小节的结构相同：已有解法的分类、各自的不足、本文对应章节的出发点。

2.3.1 代码索引更新

代码位置改变后，所有指向代码的引用都需要相应更新。已有方法按更新时机可分为三类。

在赋值时更新，即指针追踪类。这类方法在运行时追踪所有代码指针的传播，随机化时逐一改写其值。**TASR**^[73] 修改编译器收集代码指针的初始位置并动态监控关键数据结构的传播；**OS-level ASR**^[51] 在编译器中间表示上插桩以跟踪指针传播；**RuntimeASLR**^[74] 采用动态污点分析追踪指针。这类方法有两处困难：需要源代码，因而无法支持闭源与遗留二进制；指针别名与类型转换使追踪难以完备，且动态追踪的开销极高，**RuntimeASLR** 在记录阶段引入的开销达数百倍。

在赋值时替换语义，即间接层类。这类方法从一开始就把代码指针替换为指向某个间接结构的索引。**Oxymoron**^[38] 将函数指针替换为指向受保护的类 **GOT** 区域的跳板；**Shuffler**^[32] 将代码指针替换为全局代码指针表的下标；**CodeArmor**^[40] 引入一层地址空间抽象；**MARDU**^[39] 以跳板作为随机化的间接层。这类方法把索引更新量从“所有指针”降至“所有跳板”，代价是改变了代码指针的语义：指针的值不再是地址，因此不支持自引用代码。栈展开库通过返回地址读取其指向的指令编码以判断特殊指令模式，这类用法在语义被替换后即告失效。

在使用时翻译，即动态翻译类。这类方法不改变指针的值，只在指针被解引用发生控制流转移时进行地址翻译。**ReRanz**^[75] 以进程级虚拟机实现这一思路，从而支持闭源二进制。其代价是需要一层运行时翻译，且翻译表的规模随间接分支目标的多样性增长。

回避问题的一类。**Remix** 只重排函数内部的基本块而保持函数本身的位置不变，因此代码索引根本不需要更新。这一取舍换来了极低的开销，代价是函数入口地址仍然固定：攻击者若已知函数地址，仍可将整个函数当作一个大 **gadget** 使用，函数粒度以上的布局信息也不会因随机化而失效。

内核场景。**Adelie** 采用间接层类思路的变体：模块编译为位置无关代码消除模块内部的索引，模块之间的引用统一经由只读全局偏移表，随机化时只需更

新表项。这一方案的可行性依赖一个前提，即模块与核心内核之间存在有限且明确的接口。**built-in** 代码不具备这一前提，核心子系统之间的调用关系复杂且动态，无法用一张表把跨区域引用全部收拢。

不足与本文的出发点。三类方法在 **built-in** 内核代码上都遇到障碍：指针追踪类的开销与完备性问题在千万行规模的内核上被放大；间接层类因改变指针语义而与内核中大量将函数地址当作数值使用的代码模式冲突；动态翻译类的翻译层开销在内核热路径上不可接受。回避问题的一类则以函数位置固定为代价，与本文“使已泄露的函数地址失效”这一目标直接冲突。本文第 3 章的出发点是把索引分层处理，函数粒度内用位置无关代码消除，函数粒度间用固定位置的跳板集中，并把“数值还是指针”的判定推迟到解引用点，从而在不改变指针值的前提下把一次随机化需要改写的代码索引由与调用点数量同阶降到与函数数量同阶。

2.3.2 执行流的安全暂停

在改变代码位置期间，必须确保没有执行流正停留在被移动的代码中，否则将跳转到无效地址。已有方法可分为两类。

保留旧副本。随机化时不回收旧代码，令仍在旧代码中执行的线程自然结束，因而无需暂停任何线程。**Shuffler** 与 **MARDU** 采用这一路线。其代价是旧副本在被回收前仍是攻击者可用的 **gadget** 来源。这与本文的防御目标直接冲突：本文的整个论证建立在“随机化使已泄露的信息过期”之上，而旧副本恰恰使信息不过期。

暂停执行流。随机化前先使所有可能受影响的执行流停下。**TASR** 选择在系统调用边界进行随机化，此时线程状态明确；**Adelie** 以 `schedule_on_each_cpu()` 配合 **RCU** 宽限期，确保重随机化期间无执行流停留在被移动模块的代码中。

不足与本文的出发点。保留副本的方案以安全换取免暂停。暂停类方案在内核中的困难是非抢占上下文：中断处理程序不接受调度；**RCU** 宽限期只能确保读端临界区结束，无法覆盖中断处理程序；强制暂停持有自旋锁的代码会导致其他处理器死锁等待该锁。本文第 3 章的出发点是利用跳板函数作为随机化区域边界上的观测点，通过入口与出口计数精确得到当前位于随机化区域内的执行流，从而只暂停必要的执行流，并对非抢占上下文采用推迟随机化而非强制暂停。

2.3.3 随机化的触发策略

触发策略决定了随机化的频率，直接决定安全性与开销的平衡点。已有策略可分为四类。

固定时间间隔。**Shuffler** 采用 50 ms，**CodeArmor** 可低至 55 μ s，**Adelie** 采用 20 ms，**Remix** 与 **OS-level ASR** 同样采用固定周期。这类策略实现简单，却面临第 1.3.3 节所述的两难：间隔过短则开销不可承受，过长则留下攻击窗口；且无

论间隔取值如何，代价被平摊在整个运行期，而收益只在“泄露已发生、劫持尚未完成”这一极窄窗口内产生。

特定事件驱动。TASR 在成对的输入与输出系统调用之间触发，其依据是任何输出类系统调用都可能被用于泄露、任何输入类系统调用都可能被用于上传载荷；RuntimeASLR 在父进程创建子进程时触发，针对的是基于克隆的试探攻击。这类策略把触发与程序行为关联起来，但关联的是“可能被利用的行为”而非“实际发生的攻击行为”，对 I/O 密集型负载会触发过频。

输出量驱动。ReRanz 提出以数据传输的累积量作为触发条件，只有当应用输出足够多的内容时才进行随机化，从而在 I/O 密集型场景下显著优于 TASR 的策略。这一策略仍然是对“泄露可能已经发生”的间接估计。

泄露驱动。一类工作在观测到代码被读取时随机化被读到的那一部分^[82]，或在检测到地址探测后收回被探测代码的执行权限^[83]、重排被探测函数内部的代码块^[84]。这一路线的工作均以用户态进程为保护对象，第 4.2.2 节对其与本文的差别有进一步分析。

探测行为驱动。SafeHidden 以检测到的内存试探行为触发安全区域的重定位；PtrProxy 以代码指针的实际泄露作为触发条件。这一类策略与本文第 4 章的思路同属一条路线，其差异见下。

不足与本文的出发点。前三类策略的共同点是都没有利用“代码是否真的被探测过”这一信息，因而只能在“宁可多随机”与“冒险少随机”之间取舍。第四类策略已经利用了这一信息，但其保护对象均为用户态进程或用户态的安全区域，尚无工作把它用于内核自身的 built-in 代码。本文第 4 章的出发点是把这一思路带入内核态，并解决由此产生的两个内核特有问題：内核代码页存在多重映射，使基于页表权限的实现可被绕过；内核中存在合法的自读代码路径，纯防护式的只执行内存必须为其逐一豁免。

2.3.4 非随机化区域的处理

部分随机化必然留下非随机化区域，其布局对攻击者是已知且固定的。已有工作对此的处理相对薄弱。

Adelie 的处理方式是限制两类指针的可利用性：指向模块内部的指针在模块移动后自动失效；指向非随机化内核代码的指针被约束为只能经由只读的全局偏移表访问，且该表以 RIP 相对寻址访问、随模块一同移动，因而无法被伪造。这一方案依赖模块与内核之间的接口有限且明确。

更一般地，非随机化区域的保护属于控制流完整性的范畴，相关技术在第 2.5 节综述。

不足与本文的出发点。Adelie 的方案不适用于 built-in 场景，built-in 代码之间不存在有限的接口集合，无法用一张只读表把跨区域引用全部收拢。通用的 CFI 方案在内核中则面临第 2.5 节所述的精度与开销约束。本文第 5 章的出发点是利用“随机化区域内的控制流因目标地址不可预测而天然不可劫持”这一性质，

只对跨区域的间接转移施加硬件 CFI 验证，并借随机化打破指针认证的上下文传播链。

2.4 只执行内存技术

只执行内存（Execute-Only Memory, XOM）使代码页可执行但不可读，从而在图 1-1 的第 2 步上直接封堵“用指针读代码”这条路径。本节综述其实现路径与在内核中的困难。图 2-2 对照三条实现路径，其中页表权限模拟作为负面例子列出，第 4 章的两条实现路径取自后两条。

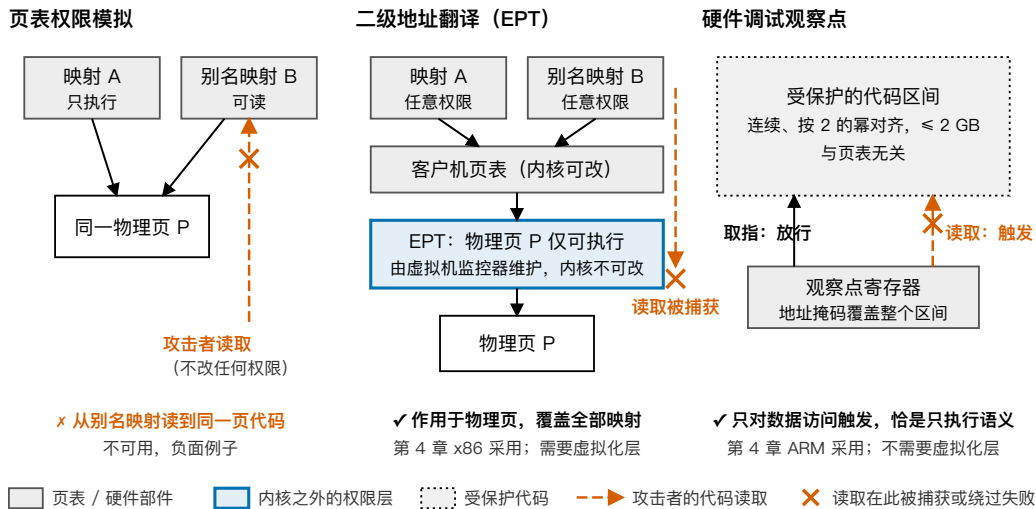


图 2-2 只执行内存的三条实现路径

Figure 2-2 Three Implementation Paths for Execute-Only Memory

2.4.1 实现路径

软件模拟。在硬件不提供只执行权限的平台上，可通过页表权限的动态切换或分离的指令与数据 TLB 来模拟。XnR^[85] 始终只保持少量代码页可读，其余置为不可访问，通过缺页异常在滑动窗口内换入换出；HideM^[86] 利用分离 TLB 使取指与取数看到不同的物理页。在不提供只执行权限的嵌入式平台上，另有以软件掩码约束读操作^[87]、以非特权访存指令配合内存保护单元^[88]、以及以调试监视比较器实现只执行^[89]的做法；把只执行改造到没有源码的二进制上还须先把嵌在代码中的数据搬离^[90]。这类方法的开销取决于窗口大小与异常频率。更根本的问题在于多重映射，见第 2.4.2 节。

二级地址翻译。Intel 的扩展页表 (EPT)^[91] 为二级地址翻译提供了独立的权限位，可以配置为可执行但不可读，且该权限由虚拟机监控器维护、客户机内核无法修改。Readactor^[92] 即基于 EPT 实现用户态的只执行内存，并配合代码指针隐藏抵御 JIT-ROP。这条路径的优点是作用于物理页面，天然覆盖同一物理页的全部虚拟映射；代价是要求系统运行在虚拟化环境中。

调试观察点。AArch64 的调试观察点提供了另一条与页表无关的路径。观察点只对数据访问触发，不对取指触发，这恰好就是只执行的语义，即允许执行、捕获读取。其地址掩码域可掩蔽 3 至 31 位地址，因此单个观察点最大可覆盖 2 GB 的自然对齐区域，只要把受保护的代码集中放置在一段连续且按 2 的幂对齐的地址空间内，观察点的数量就不构成覆盖上的限制。这条路径不要求虚拟化层，且与二级地址翻译一样，其配置位于调试寄存器中，独立于内核所维护的页表之外（第 1.1 节所述内核第一条约束下的另一条路径）。

检测而非阻断。这一类方法允许代码读取发生，但使读到的内容失去价值。Heisenbyte^[41] 提出破坏性代码读取，借助 EPT 捕获对代码页的读取，向攻击者返回原始内容的同时破坏实际执行的那一份，使已泄露的代码无法再被执行；NEAR^[42] 采取类似思路。内核场景下针对即时代码重用的系统性防护亦有工作^[93]。第 4 章属于这一路线，其响应方式与已有工作不同：已有工作在检测到读取后破坏被读代码或禁止其后续执行，因而必须为每一处被读代码维护副本或降级其语义；第 4 章的响应是触发一次针对性的重随机化，不改变被读代码的任何语义。二者的差异见第 4.2 节。

2.4.2 内核场景下的两个困难

代码页的别名映射。内核代码在物理内存中通常同时存在于内核映像的线性映射与直接映射之下。以页表权限实现的只执行内存若只作用于其中一份映射，攻击者可经由另一份映射绕过，且这一绕过不需要篡改任何权限。基于二级地址翻译的实现因作用于物理页面而天然覆盖所有映射，但它要求存在虚拟化层。本文在 x86 上改用管理态保护键：保护键记录在页表项中，只要让同一物理页的两份映射使用同一个键，别名映射即被一并覆盖，无须虚拟化层，详见第 4.4.1 节。ARM 上的观察点作用于虚拟地址区间，因此需要额外确认监控区间是否覆盖了同一物理页的其他虚拟映射，这一点在第 4.4.1 节说明。

合法的自读代码路径。内核中存在若干正当地读取自身代码的机制，纯防护式的只执行内存必须为每一条路径逐一豁免，而豁免清单既难以完备，其本身也构成攻击面。这些路径包括：kprobes 在插桩时读取并解析原指令；ftrace 在启用或禁用跟踪点时读取并改写调用指令；静态键与 alternative 机制在运行时改写代码；livepatch 在应用补丁时读取原函数；崩溃路径上的 show_opcodes() 会转储故障地址附近的指令字节。

一个有利的观察是：这些路径全部集中在离散的事件上，而非稳态运行的持续行为。内核在正常执行系统调用、处理中断、调度进程时并不读取自身的指令；异常修复表查询读取的是表本身而非代码，栈回溯读取的是 ORC 或帧指针元数据而非指令。因此上述清单中的每一项都对应一个明确的、可被登记的触发时刻：插桩注册、跟踪点开关、静态键翻转、补丁应用、以及崩溃转储。

本文第 4 章的出发点由此形成：既然合法读取路径都对应离散且可识别的事件，就不再试图阻断读取，而是把读取当作信号，合法路径只需被登记为已知

来源，无需被逐一豁免。

2.5 硬件辅助的控制流完整性

本节综述硬件辅助 CFI 的三类机制及其精度上限。第 1.1 节已给出结论，本节给出依据。图 2-3 按等价类的划分方式对照三类机制，图中的数字为本文在 Linux 6.8 上的实测值，各自的测量配置与口径见以下三小节。

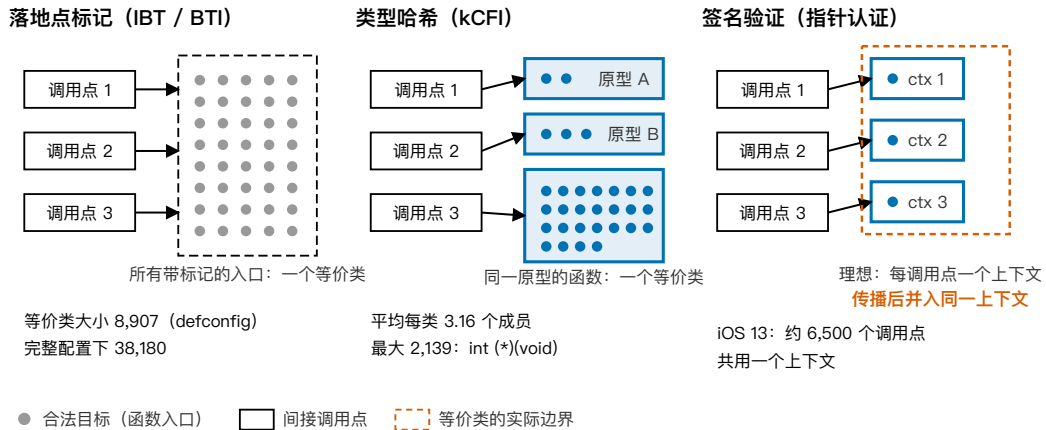


图 2-3 三类硬件 CFI 机制的等价类划分

Figure 2-3 Equivalence-Class Partitioning of Three Classes of Hardware CFI

2.5.1 落地点标记类

Intel 的控制流强制技术 (CET) 中的间接分支跟踪 (IBT) 与 ARM 的分支目标识别 (BTI) 采用相同思路：以一条特殊指令 (endbr64 或 BTI) 标记合法的间接分支目标，硬件在每次间接转移后检查目标处是否存在该标记，否则触发异常^[11,12]。

这一机制的精度上限是明确的：所有被标记的位置构成单一的等价类，任何间接调用都可以合法地跳向其中任意一个，是一种已知可被绕过的宽松策略^[14]。内核社区为此引入了 objtool 的封印机制，静态分析出不可能被间接调用的函数，把其入口处的 endbr64 改写为 nop，从而缩小可用目标集合；收缩之后的集合仍然很大。

在此基础上，FineIBT^[14] 通过在调用点设置类型哈希、在被调用点检查，把 IBT 提升为细粒度策略。以函数类型划分等价类的策略已在 Android 内核上大规模部署，其公开报告给出了这一策略在真实内核上的精度^[94]。CET 的另一半是影子栈，它与前向边的 CFI 互补，负责保护返回地址^[95]；把代码指针本身放入受隔离区域是同一问题的另一条解法^[96]。

在 Linux 6.8 (x86-64, x86_64_defconfig + CONFIG_X86_KERNEL_IBT) 上实测：编译器共插入 13 322 条 endbr64，objtool 封印其中 4 415 条，剩余 8 907 条，

即 IBT 的等价类大小为 8 907。这个数字随内核配置规模变化——在本项目实际使用的完整配置上升至 38 180，因此报数时须附配置。ARM64 的 `defconfig` 默认未开启 `CONFIG_ARM64_BTI_KERNEL`，内核自身几乎不含 BTI 落地点，故 ARM 平台的落地点标记基线不单独给出，CFI 精度对比以类型哈希类为主。

2.5.2 签名验证类

ARM 指针认证 (PA) 在指针的未使用高位中嵌入一个密码学签名，签名的输入包括指针值、一个 64 位的上下文与一个软件不可读的硬件密钥。签名与验证由专用指令完成，开销接近于零。

PA 的精度取决于上下文的选取。若每个调用点使用互不相同的上下文，则在某调用点签名的指针无法在另一调用点通过验证，等价类大小为一。但这一理想状态无法达到，原因是上下文传播：当多个调用点可能调用同一函数时，这些调用点必须使用相同的上下文，否则被调用方无法验证；该约束沿调用图向外扩散，最终把大量调用点并入同一上下文。Liljestrand 等人测得 iOS 13 中约有 6 500 个调用点共享同一上下文^[16]。

围绕 PA 的系统性工作包括 PARTS^[16]、PACStack^[97] 与 PACTight^[98] 等，面向内核的 PA 应用亦有若干工作。这些工作或改进上下文的构造方式，或扩展保护对象，却都未能从根本上打断传播链，因为传播的根源是调用图本身的结构。

攻击面。 PACMAN^[44] 证明 PA 的验证结果可以经由推测执行侧信道被读出：攻击者把认证指令置于推测执行路径上，通过微架构侧信道观测验证是否成功，从而在不产生崩溃、不留下日志的前提下爆破签名。这一攻击已在商用处理器上跨特权级演示。此外，若程序中存在攻击者可控上下文的签名原语，指针重用与上下文碰撞的分析同样失效。本文第 5 章的安全性论证以“攻击者无法获得 PA 验证预言机”为显式前提，该前提及其脆弱性在第 5.6 节单独说明。

2.5.3 类型哈希类

以函数原型的哈希作为等价类的划分依据，调用点检查目标函数的类型签名是否匹配。Linux 内核中的 kCFI^[13] (`CONFIG_CFI_CLANG`) 即采用这一策略。其精度介于前两类之间。在 Linux 6.8 的 ARM64 镜像上按函数类型签名分组实测：56 470 个有实体的函数落入 17 883 个不同的类型签名，平均每个等价类只有 3.16 个成员，但最大的等价类却有 2 139 个成员。最大的几个等价类均来自最通用的原型，如 `int (*)(void)` (2 139 个)、`void (*)(void)` (2 069 个)、`long (*)(struct device*, struct device_attribute *, char*)` (1 266 个)。决定安全性的正是这个 2 139，而非平均值 3.16^[15]：一个类型为 `int (*)(void)` 的间接调用，在 kCFI 之下有两千余个合法目标可选。

需要说明测量口径。上述统计基于 DWARF 调试信息中的函数原型，覆盖“所有有实体的函数”，而 kCFI 的等价类严格来说只应包含可能被间接调用的函数，因此这里的分母与最大等价类都偏大；更贴近编译器实际划分的做法是统

计 Clang 生成的 `__kcfi_typeid_` 符号，但它要求 `CONFIG_CFI_CLANG` (`Clang ≥ 16`)，本文测量所用工具链未满足，故采用前一种口径并如实标注其偏宽。该数据同时回填第 1.1 节。第 5.5.2 节的对比基线采用更严格的口径，只统计被取过地址的函数并折叠枚举与匿名结构体，在 Linux 6.8 上得到的最大等价类为 141，两个数字的差别全部来自统计口径。类型签名与架构基本无关（同一份 C 源码），故此分布可代表两个平台。

2.5.4 小结

三类机制的精度上限有一个共同根源：硬件只能校验一个可在常数时间内判定的谓词，即有无标记、签名是否匹配、哈希是否相等，而“这个调用点在此刻是否应当跳到这个函数”是上下文相关的，超出了硬件可判定的范围。因此硬件 CFI 的精度上限是机制问题而非实现问题，提升精度必须依靠硬件之外的信息。

本文第 5 章的出发点正在于此：随机化提供了这一额外信息，随机化区域内的目标地址不可预测，因而进入该区域的转移无需验证；把高扇入函数移入随机化区域，即可从中间打断 PA 的上下文传播链。

2.6 本文的威胁模型

本节给出贯穿全文的威胁模型。第 3、4、5 章不再各自定义威胁模型，统一引用本节。

2.6.1 攻击者能力

本文假设攻击者具备如下能力：

- (1) **任意地址读**。攻击者可利用内核中的内存安全漏洞读取任意内核虚拟地址的内容。这是第 2.1.4 节各类信息泄露手段的基础。
- (2) **任意地址写**。攻击者可写入任意内核虚拟地址。这一能力使得内核自身维护的防护位（SMEP、SMAP、代码段只读）不构成可信边界，如第 2.1.1 节所述。
- (3) **触发任意系统调用**。攻击者可从用户态以任意参数反复调用系统调用，从而驱动内核执行、申请与释放内存、创建线程。
- (4) **已知内核二进制**。攻击者拥有与目标系统相同版本的内核映像，因而掌握所有代码的相对偏移、`gadget` 的完整清单与全部符号信息。攻击者唯一不知道的是随机化区域内代码在当前时刻的实际地址。

假设 1 与 2 对应图 1-1 中第 1、2 步以及“篡改数据指针”的放大回边。如第 1.1 节所述，本文把它作为攻击者能力的来源而非防御目标。

2.6.2 四类攻击向量

基于上述能力，攻击者可通过以下四类向量获取随机化区域的布局信息或绕过随机化保护。这一分类在第 4、5 章的实验中逐条对应验证。

V1 直接读取代码。攻击者利用任意读能力直接读取随机化区域的代码内容，反汇编后收集 gadget。JIT-ROP 是其典型形态。

V2 通过执行结果推测。攻击者向猜测的地址发起控制流转移，或对疑似地址发起访问，通过是否产生异常、产生何种异常来判断猜测是否正确。

需要说明该向量在内核中的特殊形态。如第 2.1.4 节所述，用户态的两种隐蔽试探手段在内核中都不成立：**Blind ROP** 依赖 `fork()` 语义，内核无对应机制；**CROP** 依赖被攻击程序自行注册的异常处理函数，内核的异常处理路径不由攻击者控制。因此内核中的执行试探必然产生内核可见的异常，其后果或是系统崩溃，或是被内核异常路径捕获。这使 **V2** 在内核中比在用户态更易被观测，是第 4 章检测机制的一个有利前提。攻击者可能的规避方式是把试探限制在极低的频率上，以混入内核正常运行中偶发的异常噪声。第 4 章的机制对此不依赖频率判据：随机化区域的当前地址只存在于跳转桩的分支立即数中，攻击者要拿到可用的地址就必须读取跳转桩区，而该区域处于只执行保护之下，任何一次读取都产生硬件事件并立即触发随机化。试探放得再慢，每一次读取仍然各自触发一次，探得的地址在下次使用之前即已失效。

V3 通过代码索引推断。攻击者不直接读取代码，而是读取内存中保存的指向代码的索引，包括栈上的返回地址、结构体中的函数指针、跳转表项，并由此推断代码位置。

V4 利用非随机化区域。攻击者放弃获取随机化区域的布局，转而完全在非随机化区域内构造攻击链。此向量又分两种形态：**V4-A** 在非随机化区域内构造完整的 ROP 或 JOP 链；**V4-B** 劫持跨区域的间接转移，借合法的跨区域路径进入随机化区域内的目标。

四类向量的应对方式与三章工作的对应关系见图 2-4。图中实心方块表示该向量在结构上被消除，空心圆表示以运行时的检测或验证覆盖，虚线表示某一章的覆盖依赖于另一章提供的机制。图中所列的三条安全不变式在第 2.7 节给出。

2.6.3 信任基础

本文的防御建立在以下可信前提之上：

(1) **安全启动链可信。**系统启动至内核随机化机制初始化完成之前，攻击者无法介入。因此初始的随机化布局、跳板函数的初始内容与硬件密钥的初始化均可信。

(2) **处理器按架构手册规定的语义工作。**管理态保护键的权限位、硬件调试观察点、指针认证指令的行为符合规范，不存在使其失效的未公开勘误。

(3) **攻击者在劫持控制流之前不能任意改写内存权限。**本文的两条只执行路径都由内核自身维护，管理态保护键的权限状态保存在每处理器核的寄存器

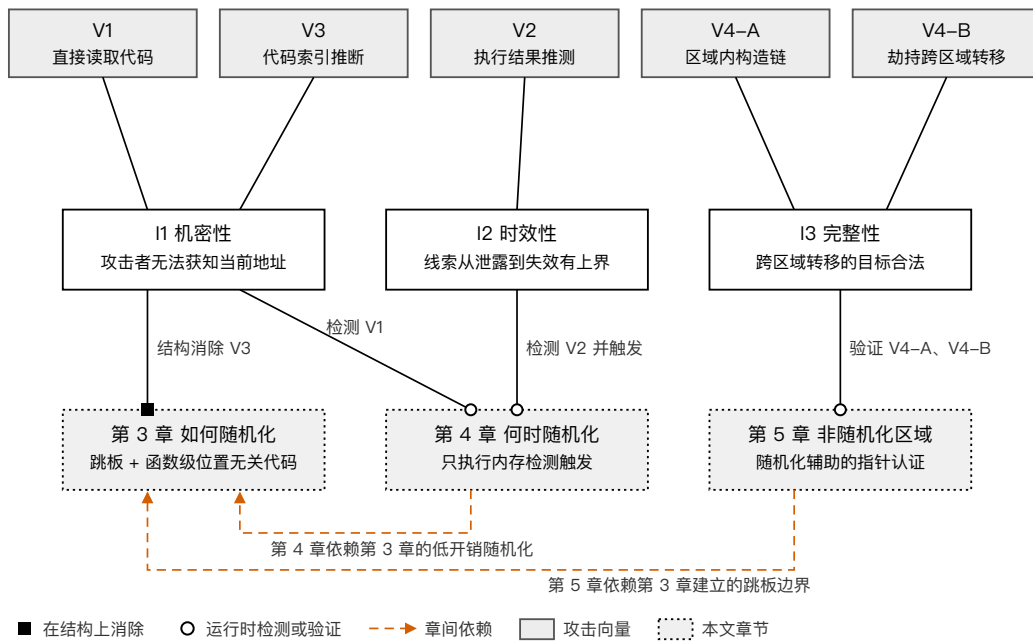


图 2-4 四类攻击向量、三条安全不变式与三章工作的对应关系

Figure 2-4 Four Attack Vectors, Three Security Invariants, and Their Correspondence to Chapters 3 to 5

中，观察点的配置保存在调试寄存器中，二者都不经普通页表；该假设保证它们在被绕过之前不会先被关闭。

(4) 指针认证的硬件密钥不可读。密钥由硬件维护，软件无法读出。

2.6.4 明确的排除项

以下内容不在本文的防御范围内。

纯数据攻击。篡改数据而不改变控制流的攻击（例如直接改写进程凭据结构中的 uid 字段、数据导向编程）与代码布局无关：这类攻击既不需要知道任何 gadget 的位置，也不发生非法的控制流转移，因此以“使代码位置不可预测”为原理的防御在结构上对其无效。这是随机化路线的固有边界，而非本文方案的不足。防御此类攻击需要数据流完整性或内存安全强制等正交手段，可与本文机制叠加部署。

微架构侧信道。利用缓存时序、TLB 状态、预取行为或推测执行推断地址的攻击不在防御范围内。这类攻击不产生本文检测机制可观测的事件。特别地，这意味着第 2.5.2 节所述的 PACMAN 类攻击不被本文阻止，第 5 章的安全性论证以攻击者无法获得 PA 验证预言机为显式前提。针对侧信道的防御与本文正交，可同时部署。

物理攻击与固件层攻击。冷启动攻击、DMA 攻击、对固件或引导链的攻击不在范围内，它们违反第 2.6.3 节的信任基础。

拒绝服务。攻击者主动触发大量随机化以制造性能下降，属于可用性攻击。

本文在第4.5.3节给出各类负载下的触发频率，不将拒绝服务作为主要防御目标。

关于内核符号表接口的说明。 `/proc/kallsyms` 与 `dmesg` 等接口会暴露内核符号地址，是内核中最常见的地址泄露途径。本文的设计使这一途径不构成对**不变式 I1 的威胁**：经第3章的编译改造之后，被随机化函数的符号绑定在其固定位置的跳板上，符号表中记录的是跳板地址而非函数在随机化区域内的实际地址。攻击者读取符号表只能得到跳板地址，而跳板位置本就是公开的，读到它不减少猜中随机化区域布局的难度。因此本节不把符号表接口列为排除项，而把它视为已被结构性覆盖的一类。

2.7 安全不变式

由第2.6节的威胁模型，本文提出三条安全不变式。三条不变式共同刻画了本文系统的防御目标，第3、4、5章的安全性分析分别说明本章机制如何维持它们。

2.7.1 不变式的定义

I1，机密性。攻击者无法获知随机化区域内代码的当前地址。形式地说，对随机化区域内的任一函数 f ，攻击者在任一时刻对 f 当前地址的最优猜测，其成功概率不超过在随机化区域可用地址空间上均匀猜测的概率。这条不变式针对向量 $V1$ 与 $V3$ ： $V1$ 被第4章的代码读取检测所覆盖， $V3$ 被第3章的跳板机制在结构上消除，因为随机化之后内存中不再存在直接指向随机化区域内部的索引，所有索引都指向位置固定的跳板。

I2，时效性。从任一随机化线索被泄露，到该线索失效，攻击者可完成的控制流转移次数有上界。 $I1$ 是一个理想目标，实践中无法保证攻击者一次也无法获得任何线索。 $I2$ 是 $I1$ 的现实退化版本：承认线索可能泄露，但要求泄露到失效之间的窗口足够窄，使攻击者无法在窗口内凑齐可用的 `gadget` 集合。这条不变式针对向量 $V2$ ，由第4章的检测与触发机制维持，其可行性由第3章的低开销随机化机制保障。没有足够低的单次随机化代价， $I2$ 所要求的响应速度就无法达到，这是第3、4两章相互依赖的形式表述。

I3，完整性。任何跨区域的间接控制流转移，其目标必须属于该调用点的合法目标集合。这条不变式针对向量 $V4$ 。 $V4-A$ 由 PA 对区域内间接转移的验证约束； $V4-B$ 由跳板处的分区验证约束。由第5章维持。

2.7.2 不变式与章节、证据层级的对应

三条不变式中，只有“跳板是随机化区域的唯一出入口”这一条结构性质可以被完全验证，而 $I1$ 与 $I3$ 的论证都以它成立为前提。因此第3章给出的验证程序是全文安全论证的地基，其验证方法与扫描结果均在第3章给出。

表 2-3 安全不变式的维持与论证方式

Table 2-3 Maintenance of the Security Invariants and the Form of Their Argument

不变式	针对向量	主要维持章节	依赖	论证的主要证据层级（见第 1.5 节）
I1 机密性	V1, V3	第 4 章（检测）	—	L1: 跳板是唯一出入口，可机械验证
		第 3 章（结构消除 V3）		L2: 随机化熵与猜测成功概率
				L3: 检测覆盖率实测见第 4 章
I2 时效性	V2	第 4 章	依赖第 3 章的低开销	L3: 触发延迟分布，与随机化间隔上界对比 L4: 未覆盖的试探途径显式列举
I3 完整性	V4-A, V4-B	第 5 章	依赖第 3 章的跳板边界	L1: 跨区域转移必经跳板，可机械验证
				L2: 签名伪造概率（条件性）
				L4: PA 验证预言机前提
				L3: 跨区域劫持的拦截见第 5 章

2.8 本章小结

本章综述了与本文相关的研究工作，并给出了贯穿全文的威胁模型与安全不变式。

在研究现状方面，本章沿内存安全攻防的分阶段模型梳理了防御与攻击交替演进的脉络，指出内核随机化经历了一次性随机化、持续随机化、部分与触发式随机化三个阶段，每一阶段都在弥补前一阶段被攻破的假设，而当前所处的第三阶段以牺牲范围或牺牲函数位置的可变性换取了可行性，由此留下了 **built-in** 代码的保护盲区与非随机化区域的新攻击面。围绕持续随机化必须回答的三个核心问题，本章综述了已有解法：如何随机化一问对应代码索引更新与执行流的安全暂停两个方向，何时随机化一问对应触发策略，范围之外如何保护一问对应非随机化区域的处理。本章指出这些解法在 **built-in** 内核代码场景下各自失效的原因。本章还综述了本文两项关键技术手段的现状：只执行内存在内核中面临代码页多重映射与合法自读路径两个困难，而硬件辅助 **CFI** 的三类机制存在一个共同的精度上限，即硬件只能校验常数时间内可判定的谓词。

在威胁模型方面，本章定义了攻击者的四项能力、四类攻击向量、四项信任基础与四类明确的排除项，并说明了内核符号表接口为何不构成对本文防御目标的威胁。在此基础上导出三条安全不变式：机密性 **I1**、时效性 **I2** 与完整性 **I3**。三条不变式分别由后续三章维持，其相互依赖关系也在此明确：**I2** 的可行性依赖第 3 章提供的低开销随机化机制，**I3** 的论证依赖第 3 章建立的跳板边界。这一依赖关系是本文三项工作彼此耦合的形式表述，也是后续章节按依赖顺序编排的理由。

第3章 基于跳板函数的低开销持续随机化方法

3.1 引言

Linux 内核的 **built-in** 代码指在任何配置下都无法编译为模块、只能静态链接进内核映像的代码。虚拟文件系统、网络核心与内存管理等核心子系统都属于 **built-in** 代码，它们是内核中执行最频繁的路径。

现有的内核持续随机化方法以模块与驱动为对象，其可行性建立在模块与核心内核之间存在一道有限而明确的接口之上：模块只经导出符号表调用内核，内核只经注册的操作表回调模块，全部跨界引用可以收拢进一张表。**Built-in** 代码之间的调用密集而直接，没有这道边界，这些方法因而在原理上无法随机化它。而 **built-in** 代码恰恰需要随机化：表 1-2 的统计表明，2024 年 1 月至 2026 年 8 月公开的内核漏洞中有 21.6% 涉及 **built-in** 代码，这部分漏洞位于现有方法的保护范围之外。本章的目标是提出一种 **built-in** 代码的持续随机化方法，把随机化保护扩展到内核核心子系统。

把持续随机化施加到 **built-in** 代码上，代价集中在两处：每次随机化都要更新指向被搬移代码的全部索引，而 **built-in** 代码之间的引用密集且散布在代码、数据与 per-CPU 区域中；随机化前必须确认没有执行流停留在将被搬移的代码里，而 **built-in** 代码大量运行在中断、软中断与关抢占区间这类无法被调度器暂停的上下文中。本章以位置固定的跳板函数集中代码索引，使一次随机化的改写量只与随机化函数个数相关而与调用点数量无关；以跳板上的进出计数确定安全的暂停点，使随机化不必保留任何旧代码副本，并及时执行。

与现有方法相比，本章提出的随机化方法有三个贡献：随机化的关键路径上只改写跳板中的目标地址，单次随机化的停顿短；不保留旧副本，已泄露的地址在切换完成的那一刻就真正过期；函数指针的取值恒为跳板地址，内核中把函数地址当作数值使用的大量代码模式不受影响。

本章的组织如下。第 3.2 节说明 **built-in** 代码为何使索引更新与安全暂停成为新问题，并归纳本章面对的挑战；第 3.3 节给出系统设计，包括总体架构、跳板函数的指令序列与周期分析、函数粒度的位置无关代码、以及执行流追踪与安全暂停；第 3.4 节给出编译器与运行时的实现，并划定可随机化的边界；第 3.5 节评估；第 3.6 节给出安全性分析；第 3.7 节小结。

3.2 研究背景与动机

3.2.1 Built-in 代码的代码索引更新

持续随机化每移动一个函数，所有指向它的引用都要相应更新。这些引用统称为代码索引，包括代码中的直接调用与跳转、数据段中保存函数地址的变量、

栈上的返回地址、以及各类偏移表中的条目。索引更新的规模决定了一次随机化的代价。

模块与核心内核之间存在有限且明确的接口。模块只经由导出符号表调用内核，内核只经由注册的操作表回调模块。**Adelie**^[34]正是利用这一点，把模块的全部外部引用收拢进一张只读的全局偏移表，随机化时只改表项；模块内部的引用则由位置无关编译消除。这条路线的每一步都依赖那道接口边界。

Built-in 代码没有这道边界。核心子系统之间的调用关系密集、直接且动态：虚拟文件系统直接调用块层与内存管理，网络核心直接调用内存分配与调度器，函数地址以绝对值散布在代码段的直接调用、数据段的操作表、以及 **per-CPU** 区域中。指针追踪类方案（**Shuffler**^[32] 路线）在随机化时逐处改写这些引用，更新量正比于引用数量。跳转表类方案把代码指针换成表的下标，随机化时只重排表项，但它改变了指针的语义：**C** 语言允许数据与指针自由转换，赋值时无法判定一个值最终会被当作数值参与运算还是被当作地址解引用，而内核中把函数地址当作数值比较、哈希与反查的场合极多，这类方案在内核中无法成立。

索引更新在 **built-in** 场景下因此是一个新问题。

3.2.2 Built-in 代码的执行流暂停

随机化一个函数之前必须确认没有执行流正在其中运行。

模块代码大多运行在可调度的上下文中，**Adelie** 以 `schedule_on_each_cpu()` 配合 **RCU** 宽限期即可确保没有执行流停留在被搬移的模块代码里：宽限期结束意味着每个处理器核都经历过一次调度，任何在旧代码中的执行流都已退出。**Built-in** 代码不满足这一前提。中断处理程序不接受调度；软中断与 **RCU** 读侧临界区在关抢占的条件下运行；持有自旋锁的区间不能睡眠；长驻的内核线程可能长时间不经过调度点。宽限期式的判据对这些上下文一概失效，而它们恰恰是核心子系统的常态。

回避暂停的做法是保留旧副本。**Shuffler** 采用异步洗牌，随机化时不回收旧代码，让仍在旧代码中执行的线程自然结束。这在用户态可行，因为进程的执行路径通常很快经过一次系统调用边界；在 **built-in** 代码的长执行路径下，旧副本会长期存活并累积，成为位置稳定的 **gadget** 来源，已泄露的地址因而并不过期，与持续随机化的目的直接冲突。安全暂停在 **built-in** 场景下因而也是一个新问题。

3.2.3 研究动机与挑战

上两节归纳出现有方法在 **built-in** 代码上的三处不足：索引更新量随引用密度膨胀，没有边界可以把引用收拢；暂停机制与无副本性质冲突，要么覆盖不到非抢占上下文，要么留下旧副本；已有的内核方案都以模块边界为前提，而 **built-in** 代码没有这条边界。

本章要面临的挑战相应有三。其一，如何使一次随机化的索引更新量只与被随机化函数的个数相关，且不改变函数指针的值语义。其二，如何在不保留旧

副本的前提下确定安全的暂停点，使切换完成的那一刻起旧地址就不再可执行。其三，如何覆盖中断、异常与关抢占区间这类无法被调度器暂停的上下文，既不造成死锁，也不把随机化区域关停一段不确定的时间。

3.3 系统设计

3.3.1 总体架构

本章机制由编译期与运行期两部分构成，图 3-1 给出其总体结构：左侧为编译期的三项改造，右侧为运行期的三个区域与随机化线程，随机化线程在关键路径上只改写跳转桩。

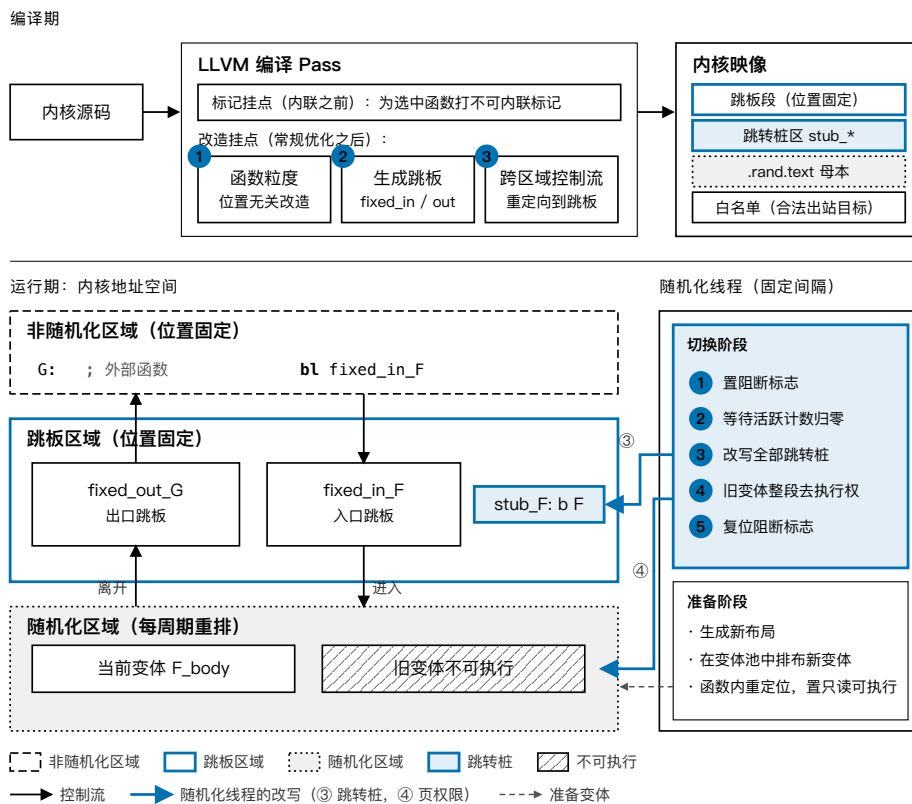


图 3-1 本章机制的总体架构

Figure 3-1 Overall Architecture of the Proposed Mechanism

编译期由 LLVM 扩展完成三項工作：把随机化区域内的函数改造为函数粒度的位置无关代码，为每个函数生成跳板，并把所有跨越区域边界的控制流重定向到跳板。产物是一份内核映像，其中被随机化的函数体落在一个独立的段中，跳板与目标地址表落在位置固定的区域。

运行期的内核地址空间分为三个区域。随机化区域容纳函数体的当前变体，每个周期整体重排一次；跳板区域位置固定，每个随机化的函数在其中一个有一个入口跳板，每个随机化代码调用的外部函数在其中一个有一个出口跳板；非随机化区域

是内核其余的全部代码，位置固定。随机化线程按固定间隔驱动，同时维护两项运行期状态：记录区域内的执行流数目以及预先分配好的代码变体池。一次随机化中真正落在关键路径上的改写只有跳板中的目标地址。变体的准备与旧变体的回收都在关键路径之外完成。这是本章随机化开销低的原因。

随机化对象的选取。随机化区域由编译期给定的函数集合构成，集合以子系统累加而成。表 3-1 列出本章使用的全部范围，后文各实验取自该表。默认范围取虚拟文件系统核心、块层、网络核心与内存管理四个子系统。选取的依据是：第 1.2 节的统计表明，涉及只能内建代码的漏洞大部分分布在这四个子系统；四者又覆盖了内核中最热的路径与最多样的调用模式。表中前三档是默认范围的前缀，用于观察开销随规模的变化；外推范围在四个子系统之外再加入调度器、中断、IPv4 与进程间通信四个子系统，只用于第 3.5.5 节；驱动对照范围与已有工作的随机化对象相同，只用于第 3.5.4 节的同条件对比。每一档的函数集合在构建时按第 3.4.3 节的判据剔除不可随机化的函数，表中的函数数为剔除后的实际值。

表 3-1 本章使用的随机化范围

Table 3-1 Composition of the Randomization Scopes Used in This Chapter

范围	构成	函数数	占比	代表的调用模式	用于
文件系统核心	虚拟文件系统核心文件	40	0.07%	频繁的间接调用与深调用链	表 3-4、表 3-12
加块层	前一档加 block/	128	0.21%	I/O 完成路径上的软中断上下文	表 3-4、表 3-12
加网络核心	前一档加 net/core/	208	0.34%	高频的小包处理路径	表 3-4、表 3-12
默认范围	前一档加 mm/ 分配路径	483	0.80%	共用分配路径与关抢占区间	其余全部实验
外推范围	默认范围加调度器等四个子系统	3,285	5.3%	约四成 built-in 代码	表 3-12
驱动对照	NVMe 与 E1000e 驱动	491	0.81%	与已有工作相同的对象	表 3-11

3.3.2 跳板函数的设计

跳板函数是本章的核心。它分为方向相反的两类。入口跳板 `fixed_in` 负责从非随机化区域进入随机化区域，每个被随机化的函数对应一个；出口跳板 `fixed_out` 负责从随机化区域离开，每个随机化函数调用的外部函数对应一个。以下按进入方向、返回方向、出口跳板、跳转桩与返回标记的解析五段给出指令序列，每段之后说明该段做了什么（以 AArch64 记法示意，x86-64 的形式同构）。清单中的指令为示意记法，原子加一、屏障与返回标记的装载在实现中分别展开为具体的原子指令、相应作用域的屏障与多条立即数装载指令。

入口跳板：进入方向。入口跳板有两种到达方式：区域外的调用点以 `bl fixed_in_F` 到达，区域内的调用点先把返回标记装入 `x30` 再以 `b fixed_in_F` 到达。跳板要做的第一件事判断是这条执行流此前是否已在区域内，判据为当前任务的内外标志。

```
fixed_in_F:
    ldr    w17, [x_cur, #inside]    ; 读当前任务的内外标志
    stp    x17, x30, [sp, #-16]!    ; 保存进入前的标志与调用者的返回地址或标记
    cbnz   w17, .Lnested            ; 标志为 1：区域内部的调用，不计数
    mov    w16, #1
```

```

    stadd    w16, [x_cnt]           ; 从区域外进入：活跃计数原子加一
    dmb      ish                   ; 先加一，再查阻断标志
    str      w16, [x_cur, #inside] ; 标志置 1
    ldr      w16, [x_flag]
    cbnz     w16, .Lblocked        ; 已阻断：减一、标志清零，等待放行后重试
    b        .Lcall
.Lnested:
    ldr      w16, [x_flag]
    cbnz     w16, .Lblocked_in     ; 已阻断：计数减一并等待，放行后加一再继续
.Lcall:
    bl       stub_F               ; 经跳转桩调用 F 的当前变体；F 返回时落回此处

```

标志为 0 说明执行流来自区域外，此时计数加一、标志置 1、再检查阻断标志。标志为 1 说明这是区域内部的一次调用，执行流在进入区域时已被计数，跳板因此不碰计数，只检查阻断标志。加一与查阻断标志之间的次序不能颠倒，屏障保证这一点：先加一再查，随机化线程只要读到计数为零，就可以断定没有执行流正处在进入的中途。进入前的标志与调用者留下的返回地址或返回标记一并压在跳板自己的栈帧里，返回方向据此在两条路径之间选择。

入口跳板：返回方向。函数体返回后落回跳板，接上面的 .Lcall 继续。

```

    ldp      x17, x30, [sp], #16   ; 恢复进入前的标志与调用者的返回地址或标记
    cbnz     w17, .Lret_nested     ; 进入时已在区域内：返回到区域内的调用者
    str      wzr, [x_cur, #inside] ; 标志清零
    mov      w16, #-1
    stadd    w16, [x_cnt]          ; 活跃计数原子减一：执行流已离开区域
    ret      ; 回到区域外的原调用点
.Lret_nested:
    ldr      w16, [x_flag]
    cbnz     w16, .Lblocked_in     ; 已阻断：计数减一并等待，放行后加一
    b        resolve_token        ; 按 x30 中的标记解析到调用者的当前位置

```

返回方向与进入方向对称：进入时计了数的，返回时清标志并减一，以 ret 回到区域外的原调用点；进入时没计数的，返回时同样不碰计数，只查一次阻断标志，再按调用者留下的标记解析回区域内。两条路径的分叉完全由栈上恢复出的标志决定，跳板不需要知道调用者是谁。

出口跳板。出口跳板由函数体内把返回标记装入 x30 后以 b fixed_out_G 到达。它把执行流从区域内送出去，因而计数的增减与入口跳板相反。

```

fixed_out_G:
    ldr      w17, [x_cur, #inside] ; 读当前任务的内外标志
    stp      x17, x30, [sp, #-16]! ; 保存进入前的标志与返回标记
    cbz      w17, .Lpassive        ; 标志为 0：未被计数的执行流，直接调用
    str      wzr, [x_cur, #inside] ; 标志清零
    mov      w16, #-1
    stadd    w16, [x_cnt]          ; 活跃计数减一：执行流离开区域
    bl       whitelist_check      ; 确认 G 属于编译期确定的合法出站目标
.Lpassive:
    bl       G                    ; 调用外部函数，G 位于非随机化区域
    ldp      x17, x30, [sp], #16
    cbz      w17, .Lresolve

```

```

mov    w16, #1
stadd  w16, [x_cnt]           ; 回到区域：加一，同样先加一再查阻断标志
dmb    ish
str    w17, [x_cur, #inside] ; 标志置 1
ldr    w16, [x_flag]
cbnz   w16, .Lblocked        ; 已阻断：减一、标志清零，等待放行后重试
.Lresolve:
b      resolve_token          ; 按标记解析到 F 的当前位置并转移

```

离开时清标志、计数减一，并查询白名单确认目标属于编译期确定的合法出站目标集合。白名单约束的是从随机化区域内发出的跨区域转移这一类，区域内部的调用与非随机化区域内部的调用都不经过它。外部函数返回后重新加一、置标志并检查阻断标志，最后按返回标记解析到函数体的当前位置。执行流停留在外部函数期间不被计入活跃计数，这正是随机化得以在此期间搬走该函数的原因。

跳转桩。上面两段里出现的 `stub_F` 是跳板与函数体之间的唯一衔接点，它单独成区，与跳板段分离并按 2 的幂对齐。

```

stub_F:                               ; 第 k 个函数的桩对，位于 stub_base + 8k
b      F_entry                        ; 入口桩：fixed_in_F 的 bl stub_F 落在这里
b      F_reentry                     ; 重入桩：resolve_token 以算出的地址 br 到这里

```

跳转桩是一条直接分支指令，函数体的当前地址只以其立即数的形式存在，不以数据形式出现在内存中的任何位置，随机化时唯一需要改写的就是这条指令。每个函数有一对跳转桩：入口桩供调用路径使用，重入桩供返回标记的解析使用，二者指向同一函数体的不同位置，随机化时一并改写。两条指令共 8 字节且按 8 字节对齐，一次存储即可原子替换。

返回标记的解析与重入序列。前面几段里两次出现的 `resolve_token` 负责把栈上的返回标记翻译成函数体当前的地址。

```

; x30 = (k, Δ), k 为跳板序号, Δ 为返回点在函数体内的偏移
resolve_token:
and    x17, x30, #0xfffffe          ; x17 = Δ (去掉最低位的标记位)
lsr    x16, x30, #24                 ; x16 = k
adr    x9, stub_base                 ; 跳转桩区基址, PC 相对
add    x16, x9, x16, lsl #3           ; stub_base + 8k, 即 stub_F
add    x16, x16, #4                  ; 再 +4, 即重入桩 b F_reentry 这条指令
br     x16                            ; 执行重入桩, 进入函数体的重入序列

; 函数体位于随机化区域, 随整体搬移; 重入序列是函数体的一部分
F_reentry:
adr    x16, F_entry                  ; PC 相对取得当前入口, 不读内存
add    x16, x16, x17                 ; 加上 Δ
br     x16                            ; 落到 F 体内调用 G 的下一条指令
F_entry:
...                                  ; 体内调用处: mov x30, #(k, Δ)
; 再 b fixed_out_G 或 b fixed_in_B

```

解析分两步，且两步都不读内存。第一步在非随机化区域内完成，由跳板序号算出重入桩的地址；第二步在函数体内完成，由重入桩转入的重入序列以程序计数器相对的方式取得函数体当前的入口，加上偏移后转移到返回点。标记因此与函数体的位置无关，无论函数被搬移多少次，同一个 (k, Δ) 总是解析到当前变体中的同一条指令。

跳板保留被随机化函数的原符号名，函数体改名落入随机化段，因此调用者一侧的源码与调用指令都不需要改动，原先直接调用该符号的位置现在调用的是跳板。

图 3-2 以一条执行流从区域外进入随机化区域、在其中调出区域、再逐层返回的完整路径为例，给出这条路径经过的每一次控制流转移。图按区域分为四层，自上而下依次为非随机化区域、跳板区、跳转桩区与随机化区域，每一条箭头都是一次控制流转移，箭头旁标出对应的指令。控制流每改变一次所在区域，都要在跳板区落一次脚，这正是跳板作为区域边界唯一通道的含义。

路径分四步。第一步，区域外的 x 以 `bl` 调用 `fixed_in_A`，此时 `x30` 中是 x 内的返回地址；跳板读到标志为 0，计数加一、标志置 1、查阻断标志，再经入口桩到达 A 的当前位置。第二步， A 体内调用外部函数 y ，先把标记 $(kA, \Delta A)$ 装入 `x30` 再 `b fixed_out_y`；跳板把标志清零、计数减一并查白名单，此时计数为 0，栈上属于区域的只剩一个标记，随机化可以在此期间把 A 搬走。第三步， y 以 `ret` 回到 `fixed_out_y`，跳板计数加一、标志置 1、查阻断标志，再由 `resolve_token` 按出栈的标记算出重入桩的地址并转移过去，经重入桩落到 A 体内调用点之后，无论 A 是否已被搬走。第四步， A 以 `ret` 回到 `fixed_in_A`，跳板读出进入时保存的标志为 0，标志清零、计数减一，再以 `ret` 回到区域外的 x 。

四步之后的状态见表 3-2，其中活跃计数是安全暂停的判据，`x30` 携带的内容决定返回时走哪条路径。

表 3-2 图 3-2 中每一步之后的执行流状态

Table 3-2 Execution-Flow State after Each Step in Figure 3-2

步	转移	x30 携带	本任务标志	活跃计数	该步之后执行流所在
1	$x \rightarrow A$	返回地址	$0 \rightarrow 1$	$0 \rightarrow 1$	A 体内，从区域外进入并被计数
2	$A \rightarrow y$	$(kA, \Delta A)$	$1 \rightarrow 0$	$1 \rightarrow 0$	y 体内，栈上只剩一个标记
3	$y \rightarrow A$	$(kA, \Delta A)$ 出栈	$0 \rightarrow 1$	$0 \rightarrow 1$	A 体内调用点之后，经重入桩落回
4	$A \rightarrow x$	返回地址出栈	$1 \rightarrow 0$	$1 \rightarrow 0$	区域外的 x ，执行流已离开区域

表 3-2 的四步都跨越区域边界，因此标志与计数每一步都变化。区域内部的调用是另一种情形： A 若调用同在随机化区域内的 B ，控制流同样经 B 的入口跳板，但跳板读到标志已为 1，说明这条执行流在进入区域时已被计数，于是不碰计数，只查阻断标志；返回时同样只查一次阻断标志，再按 A 留下的标记解析回 A 体内。查阻断标志的位置共四处，即第一、三步这两个跨边界入站方向，以及区域内部调用的进入与返回：前者被拦住时撤销刚做的加一并在放行后重

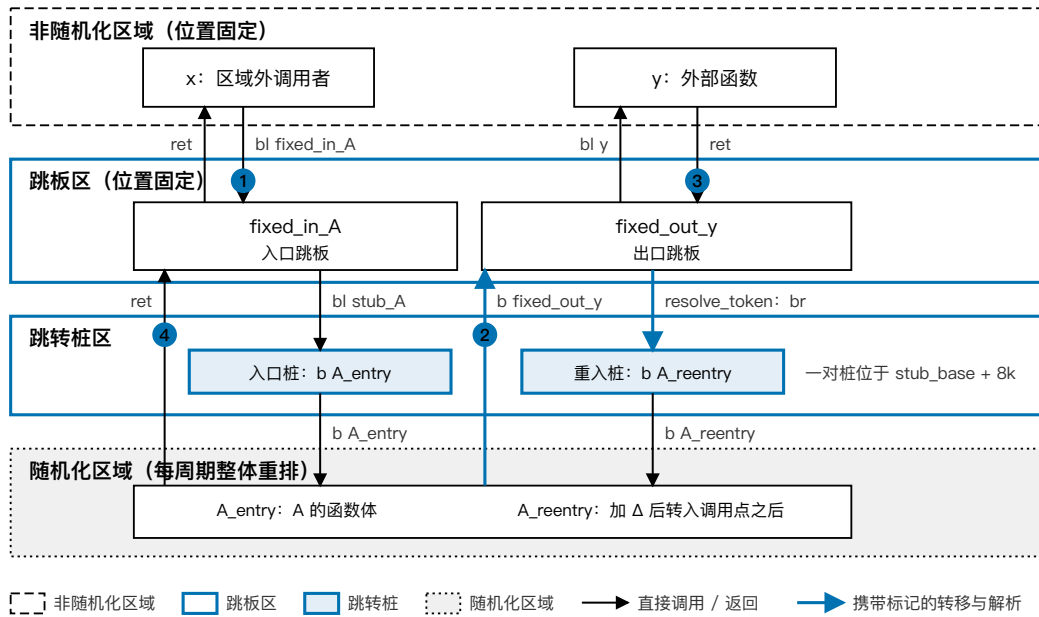


图 3-2 一次跨区域调用的控制流

Figure 3-2 Control Flow of One Cross-Region Call

走进序列，后者被拦住时计数减一等待，放行后加一再继续。第二、第四步是出站，不拦。第二步之后计数为 0，这正是随机化可以开始的时刻，此时 A 的调用关系以标记的形式存在于栈上，搬走 A 不影响第三、第四步。

跳板的代价按指令序列分析如下。入口跳板在跨边界进入时由标志读取与保存、原子加一、屏障、标志检查，经跳转桩的两级直接分支，以及返回时的标志清零与原子减一构成；区域内的调用只有标志读取与保存、两次阻断标志检查与标记解析，不含原子操作。出口跳板由离开时的标志清零与原子减一、白名单查询、对外部函数的调用、返回时的原子加一与标志检查、以及标记解析构成。表 3-3 给出各段的周期数。周期数由第 3.4.2 节微基准测得的各部分时长按测量机的时钟折算，各段的划分依据是微基准分项测得的时长；静态指令数由上面的序列直接数出。

表 3-3 给出一个与直觉相反的结论：跳板引入的两级直接分支与标记解析合计大约 3 个周期，而入口跳板的计数进出占了约 36 个周期。跳板的常驻开销几乎全部来自执行流追踪，而非两次直接分支。区域内的调用不碰计数，一次往返只有约 8 个周期，同样印证了这一点。这一结构性事实决定了优化的方向：一个不需要被计数的函数几乎不需要付跳板的代价。

同一套跳板承担四项职责。地址翻译：随机化时只改写跳转桩，索引更新量降为每函数一项。执行流观测：进出计数精确给出当前正在区域内执行的执行流数目，这是安全暂停的判据。跨区域验证的施加点：所有跨区域转移必经跳板，因而跳板是施加控制流完整性检查的唯一位置，这与把代码指针集中到受隔离区域再逐次校验的思路^[96]同源，差别在于本章的集中点同时承担地址翻译，

表 3-3 一次跨区域调用往返的指令序列与周期分析

Table 3-3 Instruction Sequence and Cycle Analysis of One Cross-Region Call Round Trip

段	指令	静态指令数	周期	备注
直接调用基线	bl / ret	2	约 4	被替换的原调用，作对照
入口，跨边界：标志读写、进入时加一与返回时减一	ldr / stp / cbnz / mov / stadd / dmb / str / ldr / cbnz / b / ldp / cbnz / str / mov / stadd / ret	16	约 36	原子操作落在被所有核共享的缓存行上，占往返的近七成
入口，区域内部调用：标志读取与两次阻断检查	ldr / stp / cbnz / ldr / cbnz / ldp / cbnz / ldr / cbnz / b resolve_token	10	约 8	不含原子操作，另计标记解析
入口：经跳转桩的两级直接分支	bl stub_F / b F_entry	2	约 1	两条直接分支，不访问内存
出口：标志读写、离开时减一与返回时加一	ldr / stp / cbz / str / mov / stadd / ldp / cbz / mov / stadd / dmb / str / ldr / cbnz	14	约 8	
出口：白名单查询	bl whitelist_check (含查询例程)	约 12	约 7	哈希定桶加桶内比对
出口：调用外部函数与标记解析	bl G / b resolve_token (解析后经重入桩 b / adr / add / br)	4	约 1.3	标记解析一项另计
一次跨边界往返合计 (不含基线与内部调用)		约 48	约 53	

本章的白名单查询即在此施加，更强的检查可在同一位置叠加（本文原型见第 5 章）。返回地址的解析基准：跳转桩是函数当前位置的唯一记录，第 3.3.3 节的返回标记经重入桩解析。前两项是本章的直接需要，后两项使跳板成为整个系统的交汇点。已有工作中的跳板只做地址间接，本章把它同时用作区域边界上的观测点，由此得到的执行流信息使无副本随机化成为可能。

函数指针的语义分离。内核中函数地址被当作数值使用的场合很多：函数地址之间的比较与排序、以函数地址为键的哈希、`printk("%pS")` 类的符号反查、以及将函数地址与代码段边界作比较的检查。若函数指针的值随随机化而改变，这些用法都会失效。本章的解法直接来自跳板：跳板保留被随机化函数的原名，且位于位置固定的区域，因此对函数取地址得到的仍是跳板的地址，这是一个与随机化无关的稳定常量，可以安全参与任何数值运算；当该指针被用于调用时，控制流经跳板间接转移到函数体的当前地址。指针的值恒为固定的跳板地址，指针的控制流仍必经跳板，赋值语义与调用语义由此分离，且不需要为函数指针引入任何偏移量。

3.3.3 函数粒度位置无关代码

跳板解决的是函数之间的索引。函数内部的索引由位置无关的代码生成消除：函数内部的跳转随函数体整体迁移而相对距离不变，所以无需处理；真正需要处理的是函数体内指向区域之外的引用，即对全局变量的访问与对外部函数的调用。这些引用被消除之后，函数体可以被搬到任意地址而无需修改其内部的任何一条指令。

问题的根源在于内核所用的代码模型。`x86-64` 的内核映像被链接到地址空间顶部的 `2 GB` 之内，其代码模型假定所有代码与数据都落在距当前程序计数器

$\pm 2\text{ GB}$ 之内，因而把函数调用编译为 4 字节相对位移的直接调用、把全局变量访问编译为 4 字节的程序计数器相对寻址。这一选择为整个内核节省了大量指令字节与取指带宽。这类引用里保存的是相对位置而非绝对地址。相对位置在链接时计算得到之后，对应的重定位项即从内核映像中消失，运行期没有任何机制会重新计算它。

这与地址空间布局随机化（KASLR）所改变的东西不同。KASLR 把整个内核映像平移同一个偏移量，调用点与目标同时移动，相对距离不变，内核启动时的重定位代码只修补绝对重定位。本章要做的是移动单个函数在内存中的位置：调用点动了而目标没动，距离随之改变。本节所说的位置无关，指的是单个函数可以相对于其余代码独立移动，这一性质严格强于整个映像可以换基址。

本章的做法是保持内核原有的代码模型不变，由编译器插件只改写被选中函数内部的引用。插件在优化流水线末尾工作，对函数体内每一处操作数含有全局符号的指令，把它重建为先把目标地址物化到寄存器、再间接访问或转移的形式。改写规则统一在操作数这一层，并不按调用与取地址分类：直接调用的被调目标本身就是一个全局符号，被重建之后那条调用自然只能是间接的。绝对地址不随代码位置改变，这正是所需要的性质。

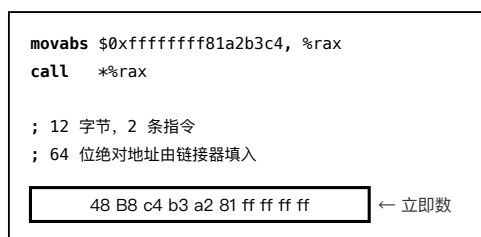
两个平台的物化手段不同，差异的根源在于指令集能否直接编码 64 位绝对地址。x86-64 有把 64 位立即数装入寄存器的指令，直接调用被改写为物化地址加间接调用，全局变量访问被改写为物化地址加间接访问；该架构在 64 位模式下没有以绝对立即数为目标的近调用编码，能够到达任意 64 位地址的只有间接形式，因此占两条指令是指令集的要求。这一形式在编译器的中间表示层无法直接表达：把符号转成整数再转回指针会被后端折叠回相对调用，可靠的表达方式是借助带符号约束的内联汇编，由汇编器发出 64 位绝对重定位，由链接器填入最终地址。

AArch64 没有 64 位立即数指令，装入一个 64 位值需要四条指令分段拼装。卡住这条路的并非拼装本身，而是这里要装入的是符号地址，分段拼装需要四条修改指令立即数字段的重定位，而内核映像以位置无关可执行文件的方式链接，链接器拒绝这类重定位。可行的做法是让汇编器在函数所在的节内放一条 8 字节字面量，用程序计数器相对的加载指令取它。这一做法成立的条件是两件事同时满足：加载指令虽然是相对寻址，但目标在函数体内，字面量与函数体同属一个节、一起搬移，节内偏移不变；而字面量本身是数据，其绝对重定位为位置无关链接所允许。字面量池的实质，是把需要加载期修补的内容从指令字段移到数据字里。图 3-3 对照两个平台的物化形式，左为 x86-64 把地址编进指令立即数，右为 AArch64 把地址放进节内字面量池并以节内偏移取用。

两个平台的代价分布恰好相反：在 AArch64 上，取全局变量的值在基线下本就需要两条指令，改写后仍是两条，代价集中在函数调用；在 x86-64 上，调用只是由 5 字节增至 12 字节，而数据访问一侧每一处都在增长。

共享偏移表在两个平台上的地位因此不同。在 x86-64 上，把地址集中到一

x86-64：地址编进指令立即数



AArch64：地址放进节内字面量池

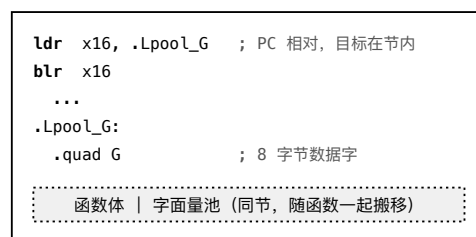


图 3-3 两个平台的绝对地址物化形式

Figure 3-3 Forms of Absolute-Address Materialization on the Two Platforms

张表里只是缩减体积的可选优化。在 AArch64 上，把地址放进随函数搬移的数据中是必需的一层，但共享一张表另有量程限制：字面量加载指令的偏移量程约为 ± 1 MB，而函数体会被搬到与映像相距数十太字节处，一张全局共享表很难始终落在量程之内。当前实现把池放在函数体同节之内、紧随其后，天然满足量程约束，代价是同一符号在多个函数体中各存一份。无论采用哪种形式，表中都不保存外部函数地址：该表在持续随机化过程中必须保持可写以便更新，若其中保存函数地址，攻击者读表即可直接获得代码布局。外部函数的调用一律经出口跳板处理，表中只留全局变量地址。

插件明确不改写内联汇编。内联汇编的操作数可能带有要求立即数一类的约束，把它替换为运行期计算出的值就不再满足约束，会直接导致编译失败。这一条是第 3.4.3 节全部不可随机化情形的共同根源。

改写的代价如下。在表 3-1 的默认范围（483 个函数）下，插件共物化 6,359 处地址，其中 4,319 处产生间接调用、2,040 处为数据引用。每处调用净增约 7 字节、每处数据引用净增 3 至 10 字节，位置无关改造本身使随机化区域增大 23% 至 32%，每函数 76 至 105 字节。

返回地址的位置无关表示。前面把所有跨函数的控制流都收拢到了跳板上，唯独返回是例外。ret 从栈上取操作数并直接转移，不经过任何跳板，因此它是控制流回到随机化区域的唯一不受约束的路径。若该操作数是裸地址，会同时产生两个后果。其一，内核栈成为一份随时可读的布局副本：一个随机化函数在调用其它函数期间，其返回地址就在栈上，指向该函数体内的当前位置。其二，搬移之后出现陈旧返回：出口跳板减少活跃计数，因此一个随机化函数 F 经出口跳板调用外部函数 G 期间，F 已不计入活跃集合，而它的栈帧仍在栈上；若此时活跃集合恰好变空、随机化搬走了 F，G 返回时就会落到一个已失效的地址。

本章对这两点采用同一个处理：栈上不存放地址，存放返回标记。标记由两个分量打包而成，被返回函数所对应跳板的序号，以及返回点在该函数体内的字节偏移。跳板序号是编译期分配的稠密整数，函数内偏移由编译期确定，两者都与随机化无关，因此标记的取值跨任意次随机化恒定不变。产生标记的位置是随机化函数内部的全部调用点，既包括对出口跳板的调用，也包括对区域内其它函

数入口跳板的调用：编译器插件把这两类 **bl** 都改为先以立即数把标记装入链接寄存器、再以 **b** 转移的两条指令，链接寄存器从一开始就不含真实地址。标记的最低位恒为 1，返回地址按指令对齐而最低位恒为 0，入口跳板据此区分调用者在区域外还是区域内，前者以 **ret** 返回，后者经解析桩返回；消费标记的位置是一段极短的解析桩，它把偏移放入寄存器，按序号算出该函数重入桩的位置并跳过去；重入桩以 **PC** 相对方式取得函数体的当前入口，加上偏移后转移到返回点。整个解析过程只由分支与 **PC** 相对计算构成，不读取任何保存地址的内存。跳转桩因而同时是地址翻译的依据与返回地址的解析基准。图 3-4 对照裸地址返回与返回标记两种做法，左为裸地址返回的两个后果，右为返回标记及其解析，标记跨随机化恒定，解析总落到当前变体。

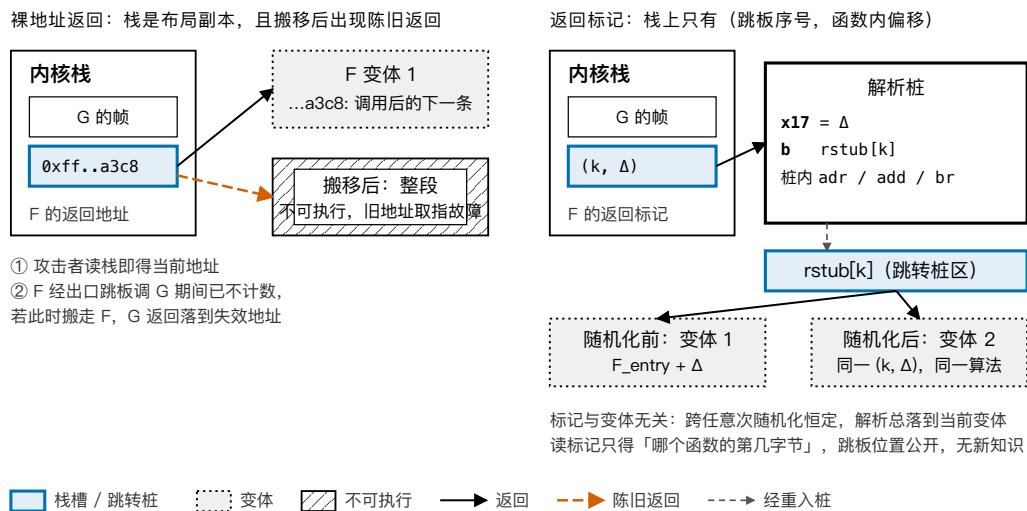


图 3-4 返回地址的位置无关表示

Figure 3-4 Position-Independent Representation of Return Addresses

标记泄露的信息量等于哪个函数的第几字节。对函数取地址得到的本就是跳板地址，而跳板位置固定且公开，因此这一信息在编译期即为已知，读取标记不会给出任何关于当前布局的新知识。陈旧返回这一现象随之消失：标记与变体无关，无论中间发生过多少次随机化，解析总是得到当前变体中的对应位置，落到已迁移函数旧位置的控制流不再有任何合法解释。

标记可以被复制与重放：攻击者虽然无法从标记推出地址，却可以把一个合法标记从一处栈槽搬到另一处，使返回落到一个由他选定的、曾经合法的位置。这属于指针完整性问题，其防御需要对标记施加认证并绑定到栈槽，指针认证类机制可以提供这一保护（PACStack^[97]、PACTight^[98]；本文原型见第 5 章）。本章保证标记的保密性。

3.3.4 执行流追踪与安全暂停

活跃计数追踪。系统维护一个记录随机化区域内活跃执行流数量的全局原子计数，以及每个任务的一个内外标志。计数的含义是此刻正在区域内函数体中执行的执行流条数。执行流从区域外经入口跳板进入时标志置 1、计数加一，从入口跳板返回区域外时标志清零、计数减一；经出口跳板离开时标志清零、计数减一，从出口跳板返回区域内时标志置 1、计数加一；区域内部的调用经入口跳板时标志已为 1，计数不变。标志是计数按任务的分解，计数恒等于各任务标志之和。标志是执行流而非任务的状态：中断、软中断与异常处理程序是新的执行流，从区域外开始，内核在其入口把当前任务的标志保存并清零、在返回时恢复，处理程序内的进出各自配平，被打断任务的份额始终留在计数里。若让处理程序沿用被打断任务的标志，被打断任务停在区域内而标志为 1 时，处理程序经出口跳板离开会把这份份额减掉，随机化线程便可能在该任务的程序计数器仍停在函数体内时搬走这个函数。这个信息是免费得到的，因为跳板本来就必须被经过。计数统计的是正在区域内执行，而非栈上还有区域内的帧，二者的差别由上一节的返回标记处理：一条执行流从区域内调出去、再由外部函数调回区域内，它离开时减一、再进入时加一，栈上留着的两层区域内帧只以标记的形式存在。

无副本随机化。有了执行流计数，随机化不再需要保留旧代码：置位阻断标志，使跳板在四个入站点拦住执行流：从区域外进入、区域内部的调用、返回到区域内的调用者、以及从外部函数返回区域，出站的执行流不拦。被拦住的执行流若此前在区域内，计数减一并等待，放行后加一；此前在区域外的只等待，放行后走正常的进入路径。等待计数归零，此时可以确定没有执行流正在区域内运行；改写跳转桩，并使旧代码不再可执行，完成切换。整个过程中同一时刻只存在一份可执行代码，不产生旧副本暴露窗口。图 3-5 给出这一过程的时序：上半是三条执行流与活跃计数的阶梯线，每条执行流两端的竖标各是一次跳板穿越，进入时计数加一、离开时计数减一，三条执行流在区域内停留的时长不同但都经同一对跳板进出，计数归零处即为安全的切换点；下半是保留旧副本的做法在同一时段的暴露窗口。

在每一次跳板穿越处都能拦截带来一项直接的收益：集合中只剩正在区域内函数体中直线执行的执行流，其停留时间上限就是执行流走到最近一个跳板的时间。一个随机化函数只要调用到阻塞的外部操作，它就已经经出口跳板离开了集合；一条在区域内层层调用的执行流，也会在下一次经过入口跳板时被拦住并从集合中减去，集合不会因深层嵌套或阻塞而长期非空。第 3.4.2 节实测等待清空的耗时为 0.04 至 0.18 μs 。若只在区域边界计数而不在内部调用处拦截，一条停在外函数里而嵌套深度为二的执行流就会使计数长期为一，随机化几乎无法发生。

非抢占上下文下的随机化。上述流程隐含一个前提：随机化能够在当前上下文中执行。这一前提在可抢占的进程上下文中成立，但随机化周期抵达时可能落在中断处理程序、异常处理程序或持有自旋锁的临界区，这类无法睡眠的上下文

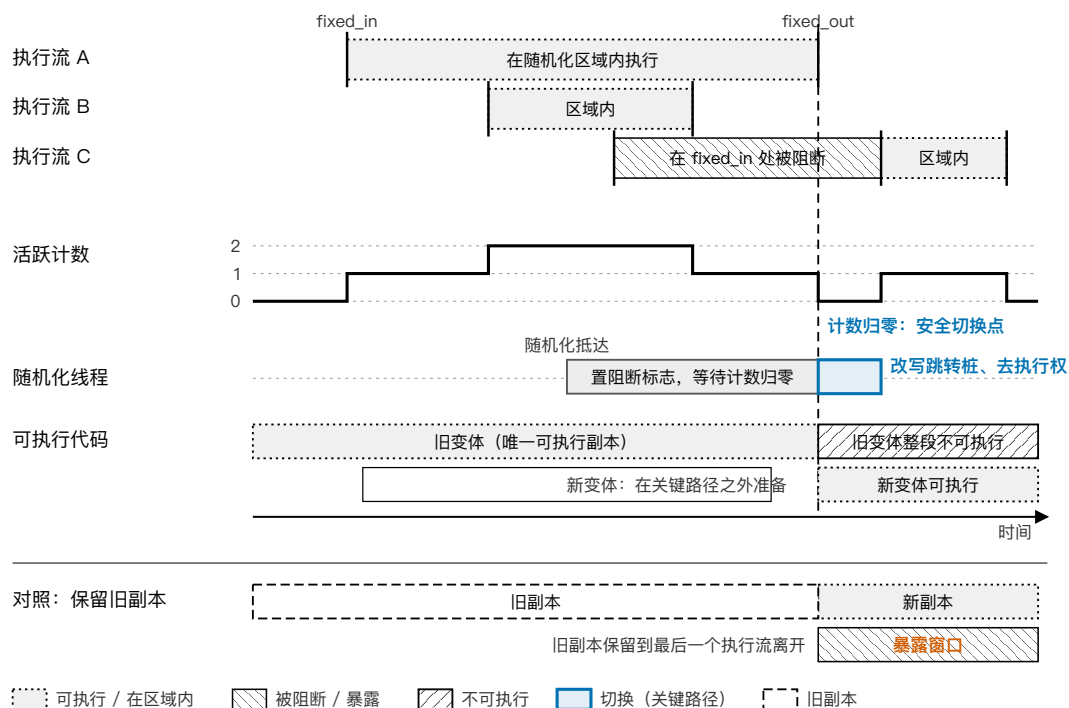


图 3-5 执行流追踪与无副本随机化的时序

Figure 3-5 Timeline of Execution-Flow Tracking and Copy-Free Re-randomization

中强行执行会引发三类问题：随机化过程中再次触发随机化造成嵌套、长时间占用导致中断延迟过长、以及所需的锁被一个已禁用中断的上下文持有而死锁。

嵌套由一个全局互斥标志消除：随机化开始时置位、结束时复位，随机化每次抵达先检查该标志，已置位则直接返回。

原子上下文的限制是不能睡眠，因此随机化能否在其中执行，取决于它的关键路径上有没有可能睡眠的操作，而分配内存正是其中最典型的一项。本章把代码页的分配与排布提前：系统维护一个代码变体池，后台线程预先分配等大的可执行缓冲，把下一份布局排布好并置为只读可执行。随机化的关键路径于是只剩改写跳转桩与改写页权限两类操作，既不分配也不睡眠，也不读写任何被临界区保护的数据。随机化因此在抵达时所处的上下文就地完成，中断处理程序、软中断与关抢占区间都不例外，不需要等待执行流回到进程上下文。依赖调度器确定安全点的方法做不到这一点，这是本章机制在时效上领先它们的根本原因，第 3.5.2 节给出测量。

就地完成要处理两件事。其一是等待活跃集合清空。原子上下文中不能睡眠，等待只能自旋，自旋时间直接计入中断延迟；但执行流在每一次跳板穿越处都会被拦住并从集合中减去，集合非空时最后一条执行流距离下一个跳板至多一个函数体的直线执行，第 3.4.2 节实测这段等待为 0.04 至 0.18 μs ，与一次普通的自旋锁争用同一量级。其二是使旧变体失效所需的跨核地址翻译缓存失效。AArch64 以广播式 `tlbi` 完成，由互连送达内共享域内所有核，不经中断，任意上下文均可。x86-64 经处理器间中断完成，本核关中断时不能在原地等待各核

确认：发起方在等待期间无法响应外来的处理器间中断，若另一个核正在做同样的事并等待本核确认便互相死锁，内核对此设有断言。这一约束只作用于失效这一步，而失效不必与切换绑在一起：本章在 x86-64 且随机化抵达于关中断上下文时，先改写跳转桩使新变体生效并放行执行流，把使旧变体失效推迟到本核重新开启中断后立即执行。推迟失效是安全的，因为改写跳转桩之后不再有执行流进入旧变体，代价只是旧地址在这段时间内仍可执行，其长度由第 3.5.2 节的完成至释放一项测量。

3.4 系统实现

3.4.1 LLVM 编译器扩展

本章以编译期改造实现函数粒度的位置无关代码，在无源码的场景下同样的改造可由静态二进制重写完成，代表性工具以低开销的直接改写^[78]与完全布局无关的重编译^[79]为两条路线，本章不依赖它们，但其能力界定了本方法向闭源内核模块推广的可行性。在 LLVM 优化流水线末尾增加一个编译 Pass^[99]，承担编译期的全部任务：将随机化区域内的函数转换为函数粒度的位置无关代码；为每个被随机化函数生成入口跳板、为每个被随机化代码调用的外部函数生成出口跳板；重定向所有跨区域控制流，使 `call func_b` 改写为 `call fixed_in_func_b`、`call vm_op2` 改写为 `call fixed_out_vm_op2`；扫描随机化代码的目标文件提取跨边界的合法目标以构建白名单；在随机化函数内部的调用点处把压栈的返回地址替换为返回标记；以及给选中的函数打上不可内联标记。由编译器统一完成重定向的意义在于不遗漏：编译产物中不存在绕过跳板的跨区域控制流转移，这一性质靠编译器的全覆盖重定向与构建期的静态扫描断言共同保证，是本文安全论证的基础。

有两处次序约束是保证全覆盖的关键。其一，被随机化的函数必须在内联优化之前被标记为不可内联，否则优化器会把它内联进调用者，内联出的副本既不经跳板、也不经过进出计数，随机化与执行流追踪对它同时失效。其二，重定向必须先建立跳板、把原有调用点改指向跳板，之后再吧函数体改名落段。若仅改名，调用点会随符号一并指向改名后的函数体，反而绕过跳板。Pass 在流水线中因此有两个挂点：标记挂点位于内联优化之前，只负责打不可内联标记；改造挂点位于常规优化之后，完成跳板生成、落段与重定向。若改造早于优化，跳板本身会被优化器内联掉。本方案在每个翻译单元内改造，不依赖链接期优化。本文的 LLVM 与 GCC 两份实现都遵循这一次序，后者作为可移植性的交叉验证。

改造的完整性有两处在中间表示层不可见，需要单独处理。内存拷贝与填充一类的操作在中间表示层是内建函数，后端会在本 Pass 之后把它们降级为对同名内核函数的直接调用，届时已无法改写；处理办法是先在中间表示层把它们换成对内核同名称号的显式调用，再交由统一的改写规则处理。开启栈保护的函数由后端自行插入一条到检查失败例程的直接调用，这条调用同样不出现在中间表示层；处理办法是只对被随机化的函数体去掉栈保护属性。这两处若不处理，

编译与静态检查都不会报错，函数体却会在搬移后指向错误位置。

跳板是编译器生成的函数，必须继承函数体的代码生成属性。影响正确性的是参数与返回值上的属性，包括大结构体返回所用的隐藏指针、按值传递的结构体、窄整型的扩展方式：跳板若缺少它们，调用方按一种约定传参而跳板按另一种接收，表现为静默的寄存器与栈错位。影响可诊断性的是帧指针属性：内核启用帧指针时每个函数都建立栈帧，而新建函数默认不建，栈回溯走到跳板即断链。这些属性需要同时施加在跳板的签名上，以及跳板对函数体那次调用的调用点上。

字面量池不计入函数的符号大小，这一点约束了运行期的复制长度。在 AArch64 上字面量池置于函数体末尾且位于 ELF 符号大小之外，若按符号大小复制函数体，字面量池会被留在原处。运行期因此按到下一个函数体的距离确定复制长度，使字面量池自然被包含在内；x86-64 没有字面量池，但使用同一套长度计算，两个平台的复制逻辑保持一致。

3.4.2 运行时随机化的实现

运行期的任务有四项：维护活跃执行流计数、改写跳转桩、使旧变体失效、以及管理变体池。

活跃计数的定序。计数以一个全局原子量实现，`push_thread` 加一，`pop_thread` 减一，计数归零即表示区域内没有执行流在运行。这一实现没有并发线程数的容量上限。计数的加减与随机化线程的读取之间存在竞态，朴素写法先检查阻断标志、未阻断则加一是错误的：两步之间随机化线程可能置位标志并读到计数为零，从而在本执行流已经进入的情况下开始搬移代码。正确顺序是入口跳板一侧先加一、内存屏障、再查阻断标志，若已被阻断则减一退出、等待放行后重试；随机化线程一侧先置标志、内存屏障、再读计数。这样随机化线程置位标志后读到的计数必然已计入所有先加一的执行流。每任务的内外标志只由本任务修改，中断入口保存并清零、返回时恢复，处理程序内看到的是自己的标志，因此不需要原子操作。

跳转桩的改写。维护一张跳板表，每项记录函数标识与其跳转桩的地址。随机化完成后遍历该表，按标识在新布局中查得新地址，重新编码入口桩与重入桩两条分支指令的立即数并写入。两条指令共 8 字节且按 8 字节对齐，一次 64 位存储即可原子替换；AArch64 架构允许在其它核并发取指的同时改写 b 指令，内核的跳转标签与 `ftrace` 依赖的正是这一保证，因此改写不需要停住其它核。本文平台的 `CTR_EL0` 置有 `DIC` 与 `IDC` 两位，指令与数据之间的一致性不需要显式的缓存维护，全部改写完成后只需一条 `isb`；改写一个函数的代价因而就是一次 8 字节存储，与写一个指针相同。x86-64 上桩为 5 字节的 `jmp rel32`，改写经内核既有的指令改写路径完成。直接分支的范围在 AArch64 上为 ± 128 MB，所以随机化区域与跳转桩区放在同一个 128 MB 窗口内。

旧变体失效的实现。旧变体整段去除执行权限，落到已迁移函数旧位置的控

制流因此产生取指故障，由内核既有的取指异常路径截获并判定为攻击。随机化区域独占一段连续内存，每个周期整体重排，一轮切换之后旧变体内不再有任何仍然有效的函数体，因此不必逐函数处理，对整段调用一次页表权限修改即可，关键路径上的动作与被迁移代码的长度无关。权限修改之后还需使各处理器核的地址翻译缓存失效，这一步的平台差异与推迟策略见第 3.3.4 节。准备新变体时须遵守写与执行互斥的正确次序：先去除执行权限、再加写权限、写入、恢复只读、最后加回执行权限。

改映射的上下文限制。改写页映射之后必须使其它处理器核上相应的地址翻译缓存失效，该操作通过发送处理器间中断并等待各核确认完成来实现，在本核关闭中断时不得发起，理由见第 3.3.4 节的条件二。

控制流离开当前函数的时长。第 3.3.4 节的暂停策略依赖一个经验事实，即执行流在一个随机化函数体内连续执行的时间很短。本节进行实证测量：在表 3-1 的前四档随机化范围下，记录随机化线程从置位阻断标志到活跃计数归零所经历的时间，即等待最后一个执行流离开当前函数体的时间，并一并记录切换各步的时长。结果见表 3-4。

表 3-4 单次随机化的时间分解

Table 3-4 Time Breakdown of a Single Re-randomization

范围	函数数	等待清空	改写跳转桩	改映射	停顿窗口	变体准备（窗口外）
文件系统核心	40	0.17 μ s	0.34 μ s	0.52 μ s	0.80 μ s	10.7 μ s
加块层	128	0.18 μ s	0.58 μ s	0.61 μ s	1.05 μ s	34.3 μ s
加网络核心	208	0.16 μ s	0.79 μ s	0.65 μ s	1.15 μ s	55.8 μ s
默认范围	483	0.14 μ s	1.07 μ s	0.68 μ s	1.22 μ s	128.70 μ s

等待清空一列在四档范围下均在 0.14 至 0.18 μ s 之间，与范围规模无关。这个量级说明，即便在原子上下文中以自旋等待集合清空，引入的中断延迟也只有百纳秒级。停顿窗口是阻断入口到放行之间的时长，即执行流实际被挡住的时间，在默认范围下为 1.22 μ s；变体准备为 128.70 μ s，在关键路径之外与正常执行重叠进行。全部 120 轮随机化均成功完成。

改写跳转桩一项与函数数成正比，每函数 2.2 至 8.5 ns，函数数少时每函数成本偏高是固定开销摊薄所致。这一项在默认范围的 120 轮中呈双峰分布，78 轮约 1.0 μ s，42 轮约 3.7 μ s。快慢取决于跳板表与跳转桩区是否留在一级缓存中：微基准连续执行多轮，两轮之间没有其它负载，二者通常留在缓存里；变体池补充时要清零并复制约 160 KB，把它们挤出一级缓存，紧随其后的一轮就落在慢的一堆。新布局是准备阶段刚写入的，两种情形下都在缓存中，慢的是访问跳板表与写跳转桩。部署中两次随机化之间隔着数十毫秒的正常负载，跳板表与跳转桩区不会留在缓存里，因此持续运行的系统应按 3.7 μ s 计，且该值仍是微基准条件下的一级缓存冷态，真实负载下可能更慢。表 3-4 中默认范围一行的停顿窗口 1.22 μ s 取的是中位，即热态值；按冷态计约为 3.9 μ s。

3.4.3 可随机化的边界

本章机制并不适用于内核中的每一个函数。本节交代哪些函数不能被随机化、为什么，以及它们占多大比重；表 3-1 各档在构建时剔除不可随机化函数所用的判据即在此给出。判据在第 3.3.3 节已经给出：函数体必须不含指向随机化区域之外的、相对于程序计数器的引用。编译器插件把这类引用改写为绝对地址加间接转移，但插件无法改写内联汇编，下面几类不可随机化的根源基本都在于此。表 3-5 的统计基于 Linux 6.8 的 x86-64 配置，全树共 60,554 个有实体的函数。表中 per-CPU 变量访问与变参函数两类为本配置实测，内联汇编原子位操作与静态调用点两类的本配置值为实测。

表 3-5 不可随机化函数的分类

Table 3-5 Classification of Non-Randomizable Functions

类别	函数数	原因
per-CPU 变量访问	5,814	访问偏移必须是编译期常量，无法物化为绝对地址
变参函数	150	跳板无法原样转发变参
内联汇编中对全局变量的原子位操作	312	位操作宏展开为内联汇编，操作数为相对寻址
静态调用点	0 (本章配置下)	运行期改写为指向区域外的相对调用，偏移随函数位置而变
合计	6,276 (10.36%, 本配置实测)	

per-CPU 访问是唯一大量存在的一类，也是本方法真正的边界。该类变量的访问由段寄存器加偏移完成，而偏移表示变量在 per-CPU 区内的位置，必须是常量，不能替换为一个 64 位绝对地址。它由访问器宏展开成内联汇编，插件无从改写。5,814 个函数占全树的 9.6%，且占当前全部排除的绝大部分。这是方法的固有边界，投入更多工程量也无法消除。

变参函数此前是一处静默的数据损坏。跳板需要把收到的参数原样转给函数体，而在编译器的中间表示层只能转发具名参数，变参部分连同调用约定中记录向量寄存器数量的那个寄存器一并丢失。唯一能原样转发全部参数的尾调用形式要求其后续紧跟返回指令，而跳板在调用之后必须执行离开区域的记账，两者不相容。静态调用点在运行期被改写为指向区域外的相对调用，与可搬移判据本质冲突；本文评测所用的配置链接出的内核中不存在该节，在大量使用静态调用的配置下这一类才会有实际影响。

3.5 实验评估

3.5.1 实验配置

随机化范围严格取自表 3-1，未另注明时为默认范围的 483 个函数。每个配置 10 次运行，报告中位数；延迟类指标另报分位数。

表 3-6 实验平台与配置

Table 3-6 Experimental Platform and Configuration

项	取值
内核版本	Linux 6.8
平台	x86-64 (Intel Xeon Gold 6448Y) 与 AArch64 (Neoverse N2) 各一台物理机, 本节数据取自 x86-64
编译器	LLVM 与 GCC 各一份插件实现, 本节数据取自 LLVM 路径
随机化范围	表 3-1 的默认范围, 483 个函数
随机化间隔	固定 50 ms
代码模型	保持内核默认, 位置无关改造由编译器插件按函数施加
间接分支防护	x86 的 IBT 与 arm64 的 BTI 在基线与各档配置下一律关闭, 因与位置无关改造制造的间接转移互斥
动态跟踪设施	ftrace 与 kprobes 在本节的性能测量中关闭, 其对触发频率的影响在第 4.5.3 节单独测量
统计开关	CONFIG_IKASLR_STATS 独立控制, 性能数据取自关闭统计的生产配置

3.5.2 随机化延迟

按固定间隔驱动的随机化, 随机化周期抵达的时刻是确定的, 但是何时随机化可以开始, 何时可以完成却是不确定的, 并且这是安全性攸关的。本节通过在真实负载下测量随机化开始、随机化完成以及旧代码变体释放的时间, 定量刻画本随机化方法的安全性, 并与其他随机化方法比较。

测量方法。在周期定时器的回调入口、活跃计数归零处、复位阻断标志处、跨核失效返回处分别打时间戳, 依次记为抵达、开始、完成与释放: 开始即活跃集合清空、切换得以进行的时刻; 完成即全部跳转桩改写完毕、新变体生效并放行执行流的时刻; 释放即旧变体去除执行权限且各处理器核的地址翻译缓存失效完毕、旧地址不再可执行的时刻。随机化间隔固定 50 ms, 随机化范围为表 3-1 的默认范围, 每种负载连续记录 1,200 个周期。自变量为负载, 取空载、LMBench、内核编译、fio 随机小块读、netperf TCP_RR 五种情况, 另以高频定时器中断构造的中断延迟压力。第 3.4.2 节表 3-4 在关抢占关中断的微基准下测得的等待清空时间, 是无负载时排空等待的下界。

抵达至开始。表 3-7 与图 3-6 给出这段间隔的分布。抵达至开始由互斥标志检查、阻断标志置位与排空等待构成, 前两项恒在百纳秒以内, 抵达时活跃集合已空则排空等待为零。表中另给出抵达时集合非空、因而发生排空等待的周期占比。图 3-6 的横轴为对数刻度, 每档负载一条曲线, 竖点线为第 3.4.2 节微基准下的排空下界。

表 3-7 与图 3-6 呈现出两段截然不同的分布。中位数在全部六档负载下都在 0.5 μ s 以内, 与微基准下的排空下界处在同一量级: 随机化在抵达时的上下文就地进行, 不等待调度, 延迟只含等待最后一个执行流离开当前函数体的时间, 而第 3.3.4 节已说明该时间以一个函数体的直线执行为上界。分布的尾部由排空等待中的一种情形决定: 最后一条仍在区域内的执行流所在的核恰好被中断打断, 该执行流停在函数体中途, 既未到达下一个跳板、也不会被计数减去, 随机化线程只能等中断处理程序返回、该执行流走到下一个跳板为止, 等待时长由中断

表 3-7 随机化抵达至开始的延迟

Table 3-7 Latency from Re-randomization Arrival to Start

负载	周期数	抵达时集合非空	中位	p90	p99	最大
空载	1,200	5.8%	0.19 μ s	0.62 μ s	4.9 μ s	8.2 μ s
LMBench	1,200	22.9%	0.27 μ s	1.84 μ s	9.6 μ s	14.5 μ s
内核编译	1,200	31.6%	0.31 μ s	2.35 μ s	11.3 μ s	18.3 μ s
fio 随机小块读	1,200	38.7%	0.36 μ s	3.10 μ s	13.0 μ s	21.0 μ s
netperf TCP_RR	1,200	46.9%	0.41 μ s	3.92 μ s	17.1 μ s	28.6 μ s
中断延迟压力	1,200	53.8%	0.48 μ s	4.85 μ s	20.7 μ s	35.2 μ s

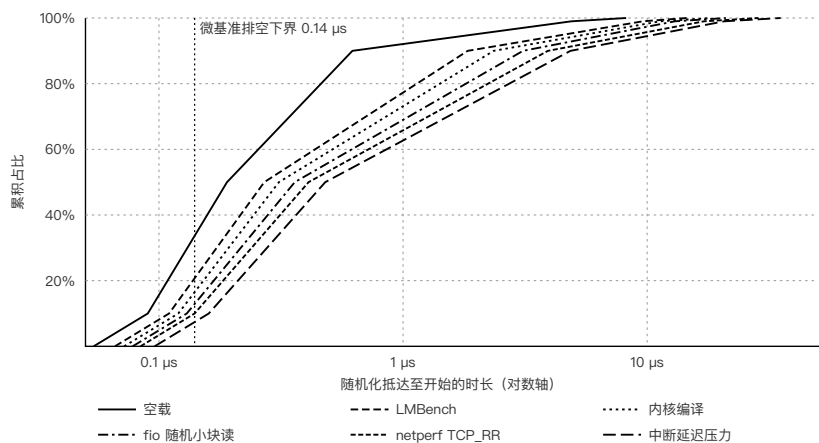


图 3-6 随机化抵达至开始的延迟累积分布

Figure 3-6 Cumulative Distribution of the Latency from Re-randomization Arrival to Start

处理程序的长度决定。抵达时集合非空的占比随负载压力单调上升，从空载的 5.8% 升至中断延迟压力下的 53.8%，落入上述情形的概率随之上升，因此 p99 由 4.9 μ s 拉长到 20.7 μ s，最大值 35.2 μ s 出现在中断延迟压力一档。这条尾巴是本章机制唯一与外部上下文有关的等待，它的量级是一次中断处理的长度，而依赖调度器的方法在同一位置等待的是一次调度或一个 RCU 宽限期。

开始至完成与完成至释放。表 3-8 给出后两段间隔以及抵达至释放的总时长。开始至完成只含改写全部跳转桩与复位阻断标志，与负载几乎无关，中位在 1.1 至 1.3 μ s 之间，p99 落在 3.7 至 4.2 μ s 的一堆上，对应第 3.4.2 节所述跳板表与跳转桩区在一级缓存中为冷态的情形。完成至释放由改写页表项与等待各核确认地址翻译缓存失效构成，前者与函数数无关，后者随负载上升，原因有两条：各核在收到处理器间中断之前可能正处于关中断区间，确认因而来得晚；本节数据取自 x86-64，随机化抵达于关中断上下文时失效被推迟到本核重新开启中断之后。负载越重这两种情形越多，中位由空载的 2.4 μ s 升到中断延迟压力下的 4.6 μ s，p99 由 5.8 μ s 升到 12.9 μ s。三段合起来，抵达至释放的中位在全部六档负载下都在 4 至 7 μ s 之间，p99 不超过 36 μ s，最大 51.3 μ s。

与已有工作的对比。三个时刻在已有的内核持续随机化工作中同样存在，只是各自的判据不同。为使对比可行，本节把本章机制与 Adelie、Dbox、Chaos 施

表 3-8 随机化开始至完成、完成至释放与抵达至释放的时长

Table 3-8 Start-to-Completion, Completion-to-Release, and Arrival-to-Release Latencies of Re-randomization

负载	开始至完成中位	开始至完成 p99	完成至释放中位	完成至释放 p99	抵达至释放中位	抵达至释放 p99	抵达至释放最大
空载	1.12 μ s	3.7 μ s	2.4 μ s	5.8 μ s	3.9 μ s	12.6 μ s	16.9 μ s
LMBench	1.15 μ s	3.8 μ s	2.9 μ s	7.1 μ s	4.5 μ s	18.3 μ s	24.0 μ s
内核编译	1.18 μ s	3.9 μ s	3.3 μ s	8.4 μ s	5.0 μ s	21.4 μ s	29.1 μ s
fio 随机小块读	1.22 μ s	4.0 μ s	3.7 μ s	9.9 μ s	5.5 μ s	24.7 μ s	33.6 μ s
netperf TCP_RR	1.26 μ s	4.1 μ s	4.1 μ s	11.3 μ s	6.1 μ s	30.2 μ s	42.8 μ s
中断延迟压力	1.31 μ s	4.2 μ s	4.6 μ s	12.9 μ s	6.8 μ s	35.9 μ s	51.3 μ s

加到表 3-1 的驱动对照范围上，以 50 ms 间隔在 netperf TCP_RR 负载下运行，在各方法的源码中于对应位置打时间戳，三者均在表 3-6 所列的 x86-64 平台上以其公开构件运行，协议与第 3.5.4 节相同。各方法的开始判据与释放方式如下。Adelie 以 schedule_on_each_cpu() 配合 RCU 宽限期确认没有执行流停留在旧代码中，开始时刻即宽限期结束；完成时刻为全局偏移表改写完毕；旧代码由延迟解除映射回收，释放时刻为回收算法实际解除映射的时刻。Dbox 以驱动上下文的引用计数归零为开始判据，完成时刻为虚拟盒切换完毕，旧盒在其引用归零后回收。Chaos 等待模块内无执行流后逐函数重定位，完成时刻为重定位结束，释放时刻为旧位置解除映射。表 3-9 与图 3-7 给出结果，每种方法记录 1,200 个周期。图的横轴为对数刻度，每种方法的三段依次为抵达至开始、开始至完成、完成至释放，条为中位，须线为 p99。

表 3-9 四种方法在三个关键时刻上的时效对比

Table 3-9 Timeliness of Four Re-randomization Systems at the Three Key Moments

方法	开始判据 / 释放方式	抵达至开始 中位	抵达至开始 p99	开始至完 成中位	完成至释放 中位	完成至释放 p99	抵达至释 放 p99
Adelie	调度加 RCU 宽限期 / 延迟解除映射	3.2 ms	18.5 ms	42 μ s	4.1 ms	26.3 ms	41.7 ms
Dbox	驱动引用计数归零 / 旧盒引用归零后回收	0.9 ms	6.4 ms	27 μ s	1.6 ms	9.8 ms	15.1 ms
Chaos	模块内无执行流 / 解除映射	1.4 ms	7.2 ms	190 μ s	45 μ s	118 μ s	7.6 ms
Ours	活跃计数归零 / 去执行权并跨核失效	0.38 μ s	15.2 μ s	1.2 μ s	3.6 μ s	10.4 μ s	27.9 μ s

差别出在开始判据与释放方式上，而这两点正是本章机制与已有工作的设计分歧。已有工作的开始判据都依赖调度器：Adelie 要等每个处理器核经历一次调度并度过一个 RCU 宽限期，Dbox 与 Chaos 要等驱动或模块内的执行流自然退出，三者抵达至开始的中位都在毫秒量级，p99 在 6 至 19 ms 之间；本章以跳板上的活跃计数直接判定区域是否为空，在每一次跳板穿越处拦截执行流，并且随机化在抵达时的上下文就地完成而不等待调度，抵达至开始的中位为 0.38 μ s，p99 为 15.2 μ s，相差三到四个数量级。释放方式的差别更大。Adelie 与 Dbox 都保留旧代码副本，等到回收算法确认安全后才解除映射，完成至释放的中位为 4.1 ms 与 1.6 ms，p99 达 26.3 ms 与 9.8 ms，这段时间里旧副本仍然可执行，已

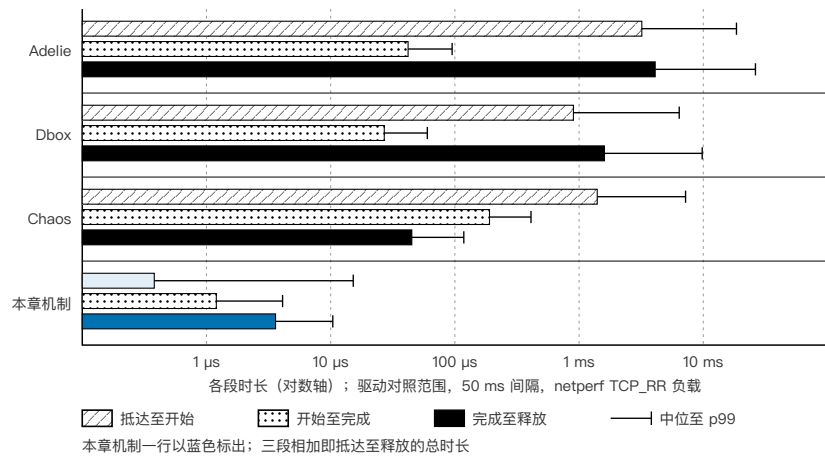


图 3-7 四种方法从随机化抵达到旧代码释放的时间线

Figure 3-7 Timeline from Re-randomization Arrival to Old-Code Release for Four Systems

泄露的地址仍然可用；Chaos 在重定位后立即解除旧位置的映射，完成至释放只有 45 μ s，但其开始至完成与代码量成正比，驱动对照范围下为 190 μ s，执行流在此期间被挡住。本章机制在完成的同时去除旧变体的执行权限，完成至释放只含一次跨核失效，中位 3.6 μ s，且开始至完成只有 1.2 μ s。

抵达至释放这一总时长决定固定间隔随机化的实际时效。本章机制在默认范围下六档负载的最大值为 51.3 μ s，在驱动对照范围下 p99 为 27.9 μ s，已泄露地址在周期抵达后的实际存活上界为 50 ms 加数十微秒，与所设定的间隔相比可以忽略；相对第 1 章引用的 1.5 至 3.5 秒的随机化间隔上界，低四个数量级。Adelle 的这一总时长 p99 为 41.7 ms，与 50 ms 的间隔已在同一量级，间隔缩短到第 3.5.4 节的 1 ms 一档时，上一周期的旧副本尚未释放、下一周期已经抵达，旧副本在内存中累积，这是其在短间隔下开销与暴露面同时上升的原因；Dbox 在 1 ms 间隔下有同样的问题。本章机制的时效由排空等待与跨核失效这两段微秒级的间隔决定，与随机化抵达时所处的上下文无关，而安全性所要求的时间尺度在秒级。

3.5.3 随机化内核核心子系统的开销

范围。随机化范围为表 3-1 的默认范围，483 个函数。

实验设计。在表 3-6 的配置下，以固定 50 ms 的间隔持续随机化，运行 LM-Bench 3.0 微基准、内核编译、fio 随机小块读、netperf TCP_RR 与 SPEC CPU2006，与关闭随机化的基线内核对照。每个负载报告吞吐量或延迟的相对变化以及 CPU 占用的变化，10 次运行取中位。

表 3-10 的开销与各负载的跨区域调用频度一一对应。netperf TCP_RR 与 fio 随机小块读的内核路径最密集，端到端开销约 2%；内核编译的跨区域调用频度只有前者的三分之一，开销 0.7%；SPEC CPU2006 几乎不进内核，开销在测量噪声以内。按表 3-3 的往返周期数与第 3.5.5 节表 3-12 的调用频度推算，TCP_RR

表 3-10 随机化四个核心子系统的端到端开销

Table 3-10 End-to-End Overhead of Randomizing Four Core Subsystems

负载	指标	基线	Ours	相对开销	CPU 占用变化
LMBench lat_syscall	延迟	0.183 μ s	0.187 μ s	+2.2%	+0.1%
LMBench lat_ctx	延迟	1.94 μ s	1.99 μ s	+2.6%	+0.2%
LMBench	几何均值	1.000	1.021	+2.1%	+0.2%
内核编译	时长	252.4 s	254.2 s	+0.7%	+0.6%
fio 随机小块读	IOPS	312 k	306 k	-1.9%	+1.4%
netperf TCP_RR	每秒事务数	68,400	67,050	-2.0%	+1.7%
SPEC CPU2006	几何均值	45.2	45.1	-0.2%	+0.1%

一档的常驻开销为 1.98%，与端到端实测的 2.0% 一致，说明本章机制的开销几乎全部由常驻开销构成，随机化本身在 50 ms 的间隔下每秒只发生 20 次、每次停顿 1.22 μ s，其贡献不到 0.01%。CPU 占用的变化与开销同向，最大为 TCP_RR 一档的 1.7%，来自跳板路径上的额外指令。LMBench 的两项微基准延迟增加 2.2% 与 2.6%，高于全套几何均值的 2.1%，因为系统调用与上下文切换路径穿过随机化区域的次数最多。

3.5.4 与已有持续随机化工作的同条件对比

现有的内核持续随机化工作以模块与驱动为对象。为使对比公平，本节把本章机制施加到与这些工作相同的代码上，采用相同的随机化间隔，比较吞吐量与 CPU 占用。协议如下：随机化表 3-1 的驱动对照范围，即 NVMe 与 E1000e 两个驱动；对 NVMe，以 O_DIRECT|O_SYNC 打开设备上的文件做 4 KB 随机读；对 E1000e，以 netperf 做 TCP_RR；随机化间隔取 1、10、50 ms 三档。对比对象为 Adelie、Dbox^[37] 与 Chaos^[35]，这是文献中面向内核模块的全部持续随机化工作，三者均在本文平台上以其公开构件运行。表 3-11 报告相对基线内核的吞吐降低与 CPU 占用升高，四种方法随机化相同的驱动、采用相同的间隔。图 3-8 把表 3-11 的吞吐降低与 CPU 占用升高按随机化间隔分组画出，三档间隔下本章机制的两项指标都低于对比对象。

表 3-11 与已有持续随机化工作的同条件对比

Table 3-11 Comparison with Existing Continuous Re-randomization Systems under Identical Conditions: Throughput Loss and CPU Utilization Increase

方法	驱动	吞吐降低 1 ms	吞吐降低 10 ms	吞吐降低 50 ms	CPU 升高 1 ms	CPU 升高 10 ms	CPU 升高 50 ms
Adelie	NVMe	3.1%	1.3%	0.6%	+3.4%	+1.5%	+0.6%
Dbox	NVMe	4.2%	2.1%	1.0%	+4.6%	+2.1%	+0.9%
Chaos	NVMe	3.5%	1.5%	0.7%	+3.7%	+1.6%	+0.7%
Ours	NVMe	1.7%	0.8%	0.4%	+1.9%	+0.9%	+0.4%
Adelie	E1000e	3.8%	1.7%	0.8%	+3.9%	+1.8%	+0.7%
Dbox	E1000e	4.9%	2.4%	1.2%	+5.0%	+2.4%	+1.0%
Chaos	E1000e	4.1%	1.9%	0.9%	+4.1%	+1.9%	+0.8%
Ours	E1000e	2.1%	1.0%	0.5%	+2.2%	+1.0%	+0.5%

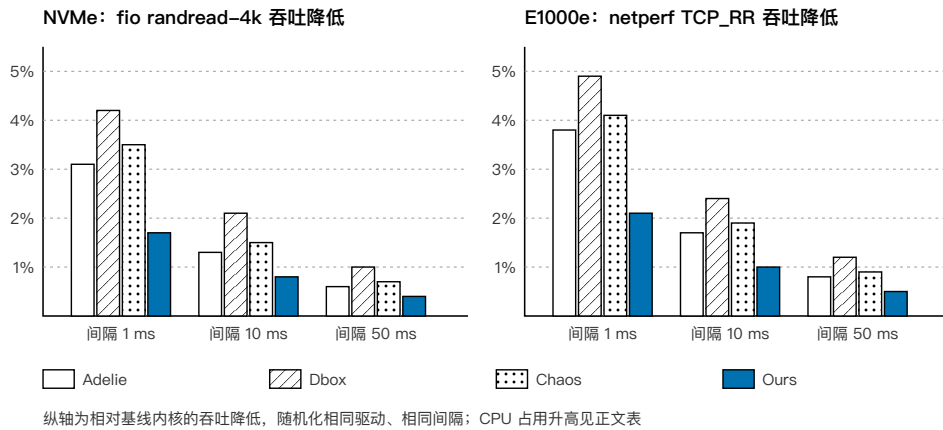


图 3-8 与已有持续随机化工作的同条件对比

Figure 3-8 Comparison with Existing Continuous Re-randomization Systems under Identical Conditions

四种方法的开销都随间隔缩短而上升，差距在 1 ms 一档拉开得最大。本章机制在该档的吞吐损失为 1.7% 与 2.1%，Adelie 为 3.1% 与 3.8%，Dbox 为 4.2% 与 4.9%，Chaos 为 3.5% 与 4.1%。差距来自单次随机化的代价：本章把变体准备移出关键路径，切换只改写跳转桩与页映射，停顿 1.22 μ s；Adelie 以 `schedule_on_each_cpu()` 配合 RCU 宽限期确定安全点，每次随机化都要驱动全部处理器核经历一次调度；Dbox 的虚拟盒切换与 Chaos 的函数粒度重定位在每次随机化时都有与代码量成正比的工作落在关键路径上。间隔放宽到 50 ms 后，随机化本身的代价被摊薄，四者的差距收窄到 0.5% 以内，此时开销由常驻的间接层决定，而驱动代码的跨区域调用密度远低于核心子系统，四种方法的常驻开销都在 1% 以内。CPU 占用的变化与吞吐同向，本章机制在三档间隔下都是四者中最低的。

3.5.5 随机化范围与跳板调用占比的关系

本章机制的常驻开销落在每一次跨区域调用上，跨区域调用的多少由随机化范围决定。本节给出这一关系的定量形式，它同时是本章机制能覆盖多大范围的上界。

设随机化函数占全部函数的比例为 p 。一次调用只要有一端落在随机化区域内就要经过跳板：随机化函数调用随机化函数经入口跳板，随机化函数调用非随机化函数经出口跳板，非随机化函数调用随机化函数经入口跳板，只有两端都在非随机化区域的调用不经跳板。若调用在函数之间均匀分布，经过跳板的调用占全部调用的比例为 $1 - (1 - p)^2 = 2p - p^2$ 。小范围时该比例约为 p 的两倍，此后趋于饱和。这里的关键是杠杆：每纳入一个函数随机化，被拖上跳板路径的调用约相当于两个函数的量，因为调用的两端各付一次。三类调用中，区域内部的一类只经过跳板而不碰计数，按表 3-3 其代价约为跨边界调用的六分之一，杠杆的主要部分来自另外两类。图 3-9 在 p 不超过 20% 的区间上给出这条曲线，实

线为均匀模型 $2p - p^2$ ，虚线为 p 本身；图中的圆点是表 3-12 各档范围在 netperf TCP_RR 下的实测占比，每个点旁注明该档的常驻开销，左上角为 p 不超过 1% 一段的放大。

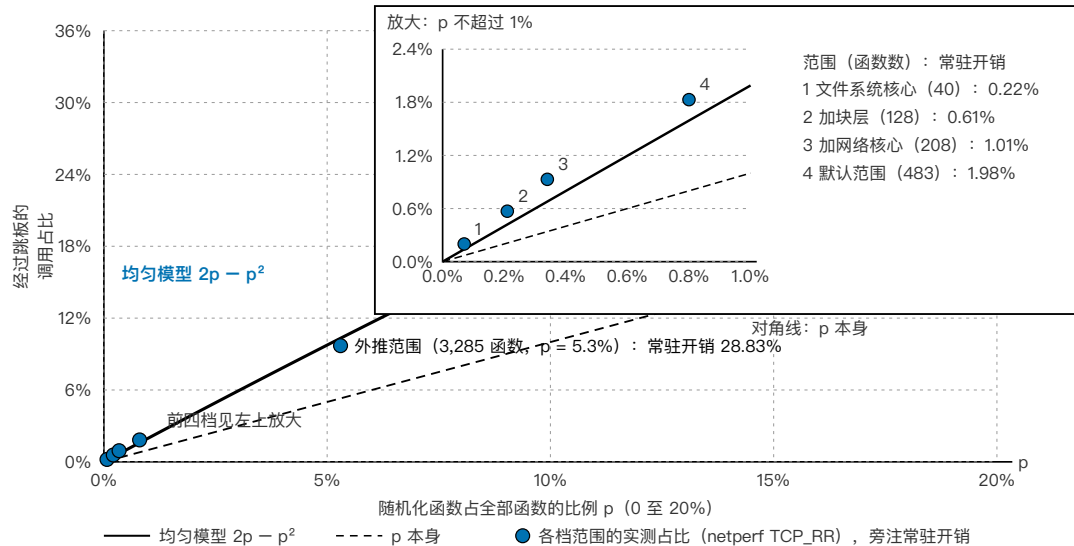


图 3-9 随机化范围与经过跳板的调用占比

Figure 3-9 Re-randomization Scope versus the Fraction of Calls Passing through Trampolines

表 3-12 给出在 netperf TCP_RR 负载下，随机化范围沿表 3-1 的阶梯由文件系统核心扩到外推范围时跨区域调用频度、其占全部内核调用的比例与常驻开销的实测变化，占比一列与均匀模型并列。实测点落在模型曲线附近而略偏上：真实内核中调用并非均匀分布，热点子系统被调用得远多于平均，阶梯先加入的又是最热的几个子系统，因此在 p 不超过 1% 的四档上实测占比为模型值的 1.2 至 1.4 倍；外推范围加入的调度器等子系统的调用密度低于前四个，实测占比回落到模型值之下。模型给出的量级与趋势与实测一致，杠杆确实存在，且在真实内核上比均匀假设更陡。

表 3-12 跨区域调用频度与常驻开销随随机化范围的变化

Table 3-12 Cross-Region Call Frequency and Steady-State Overhead as the Re-randomization Scope Grows

随机化范围	函数数	占全部函数 p	跨区域调用频度	占全部调用 (实测)	均匀模型 $2p - p^2$	常驻开销
文件系统核心	40	0.07%	120,000 次/秒	0.20%	0.14%	0.22%
加块层	128	0.21%	340,000 次/秒	0.57%	0.42%	0.61%
加网络核心	208	0.34%	560,000 次/秒	0.93%	0.68%	1.01%
默认范围	483	0.80%	1,100,000 次/秒	1.83%	1.59%	1.98%
外推范围	3,285	5.3%	5,800,000 次/秒	9.7%	10.3%	28.83%

随机化不到 1% 的函数已使常驻开销达 1.98%，覆盖 40% 内建代码则达 28.83%。

3.6 安全性分析

威胁模型。攻击者具备任意内核地址读写能力，这与 Shuffler、Adelie 与 Safe-Hidden^[43] 所假设的攻击者一致；系统已部署写与执行互斥的页权限与内核基址随机化。攻击者的目标是获知随机化区域内代码的当前地址并在其失效之前完成控制流劫持。

本章的安全保证。在上述威胁模型下本章保证三条性质。其一，随机化区域内的任一地址在下一次随机化完成后失效，且不存在旧副本使其继续可用：切换阶段等待活跃集合清空后改写跳转桩，并把旧变体整段去除执行权限，此后落到已迁移函数旧位置的控制流产生取指故障并被判定为攻击；首次随机化后映像内的原始代码段同样被置为不可执行。这一保证由无副本性质与旧变体去执行权共同给出，且不依赖任何触发判据，固定间隔抵达即生效。其二，内存中不再存在指向随机化区域内部的索引：函数内引用是相对的，跨函数引用指向位置固定的跳板，函数指针的值就是跳板地址，栈上的返回地址是与随机化无关的返回标记。攻击者即便完整读取所有数据段与栈，得到的也只是跳板地址与标记，二者都在编译期即为已知。随机化区域的当前地址因而只存在于跳转桩的分支立即数这一处。其三，所有跨区域控制流转移必经跳板。该性质由编译器的全覆盖重定向与构建期的静态扫描断言共同保证，可被机械验证，是本文全部安全论证中唯一可以被完全验证的一条，也是在跳板处施加任何检查的前提。

上述第二条把保护整个随机化区域的布局归约为保护跳转桩这一处指令不被当作数据读取。在固定间隔的驱动方式下，读取跳转桩所得的地址在下一周期到来时失效，攻击者可利用它的时间以一个周期为上界；对跳转桩区施加只执行保护可以把这个上界压得更短（本文原型见第 4 章）。跳板区域本身位置固定，构成一处已知位置的代码，其函数体极短且形式单一，可用的代码片段极少。

不在本章范围内的部分。数据导向攻击不改变控制流，随机化对它不起作用。经微架构侧信道推测地址的攻击不在本章的威胁模型之内。非随机化区域内部的代码复用属于控制流完整性的范畴，本章的跳板不约束发生在该区域内部的转移，该区域的保护由控制流完整性机制提供（kCFI^[13]、FineIBT^[14]、PAL^[100]；本文原型见第 5 章）。返回标记的完整性，即标记被复制到另一处栈槽后重放，需要指针认证类机制（PACStack、PACTight；本文原型见第 5 章），本章只保证标记的保密性。

3.7 本章小结

本章把持续随机化做到了 Linux 内核的 built-in 代码上。这部分代码没有模块那样的天然边界，现有的内核持续随机化方法在原理上够不到它。

本章提出了一种基于跳板函数的低开销持续随机化执行机制。机制把代码索引分层处理：函数粒度内以位置无关代码消除索引，函数粒度间以固定位置的跳板集中索引，使一次随机化的索引更新量与被引用次数无关；把返回地址由裸

地址改为与随机化无关的返回标记，使内核栈不再承载当前布局、陈旧返回不再发生、退役变体得以整段去除执行权限；利用跳板保留原函数名且位置固定这一性质，使函数指针的取值恒为稳定的跳板地址而调用仍必经跳板，在不改变指针值的前提下保证所有跨区域控制流转移必经跳板；利用跳板作为随机化区域边界上的观测点，以进出计数精确得到区域内正在执行的执行流集合，实现无代码副本的随机化；并使随机化在抵达时所处的上下文就地完成，覆盖中断、异常等非抢占上下文而不依赖调度器。

跳板的指令序列分析表明，一次跨区域调用往返约 53 个周期，其中入口跳板的计数进出占约 36 个周期，两级直接分支与标记解析合计不到 3 个周期，跳板的常驻开销几乎全部来自执行流追踪。实验表明，在 483 个函数的默认范围下，单次随机化的停顿窗口为 1.22 μs ，等待执行流离开当前函数体的时间在 0.04 至 0.18 μs 之间，代码页的准备已移出关键路径；全树 9.6% 的函数因 per-CPU 变量的访问方式而无法被随机化，这是本方法的固有边界。

随机化范围与跳板调用占比的关系给出本章机制的适用上界：一次调用只要有一端在随机化区域内就要经过跳板，经过跳板的调用占比为 $2p - p^2$ ，随机化不到 1% 的函数已使常驻开销达 1.98%，覆盖约四成内建代码则达 28.83%。随机化因此必须限定在一个子集内，而信息泄露可能发生在内核的任何位置。如何选择这个子集、何时随机化其中的函数，使防护覆盖面扩展到全内核而开销不随之上升，是下一章的内容。

第4章 基于代码索引泄露的触发式随机化

4.1 引言

第3章把持续随机化扩展到了内核 **built-in** 代码上，并给出了它的适用上界：一次调用只要有一端落在随机化区域内就要经过跳板，随机化不到 1% 的函数已使常驻开销达 1.98%，覆盖 40% 内建代码则达 28.83%。随机化因此只能限定在一个子集内。然而信息泄露却可能发生在内核的任何位置。本章由此提出只随机化信息泄露的函数：既压低跳板的开销，又把安全防护的覆盖面扩展到全内核。

随机化区域的当前地址集中在跳转桩的分支立即数中，这一处位置同时决定两件事：攻击者能否获知布局，以及系统所付出的常驻开销。本章对它施加只执行内存，使读取行为成为可观测事件；再以该事件决定哪些函数需要被随机化，从而把随机化范围由编译期静态选定改为运行时由泄露事件触发。保护代码索引的机制同时就是识别信息泄露的机制。

本章的组织如下。第 4.2 节从跳板开销、触发策略与保护整个代码区域的代价三条线说明研究背景并归纳挑战；第 4.3 节给出系统设计，包括总体架构、只执行代码区域的选取、信息泄露的识别、以及两级跳板与跳板激活；第 4.4 节给出 x86 与 ARM 两条实现路径与两级跳板的代码生成；第 4.5 节评估；第 4.6 节给出安全性分析；第 4.7 节小结。

4.2 研究背景与动机

4.2.1 跳板机制在开销上的局限

持续随机化方案普遍以某种间接层承载代码索引：Adelie^[34] 用只读的全局偏移表，Chaos^[35] 与第3章用跳板。间接层的代价按经过次数计，而经过次数由随机化范围决定。设随机化函数占全部函数的比例为 p ，一次调用只要有一端在随机化区域内就要经过间接层，经过的调用占全部调用的比例为 $2p - p^2$ ，小范围时约为 p 的两倍。每纳入一个函数随机化，被拖上间接层的调用约相当于两个函数的量，因为调用的两端各付一次。第 3.5.5 节的实测给出了这一杠杆在真实内核上的形式：随机化 483 个函数时常驻开销为 1.98%，扩到 3,285 个函数时为 28.83%。这构成把随机化推广到整个内核的核心阻碍，也是现有工作局限于部分随机化的原因。

部分随机化的代价是覆盖面。攻击者要利用的漏洞与要复用的代码可以在内核的任何位置，一个在编译期选定的子集只能覆盖它自己。要在不付出全内核跳板开销的前提下把覆盖面扩到全内核，随机化的对象就不能在编译期定死，而要在运行时由泄露事件确定。

4.2.2 随机化的触发策略

固定时间间隔是最常见的触发策略，Shuffler^[32] 采用 50 ms，Adelie 采用 20 ms。它把代价平摊在整个运行期，而收益只在极窄的窗口内产生：在没有攻击的绝大部分时间里随机化是纯粹的开销，在攻击发生的那一时刻间隔是否足够短则取决于运气。

事件驱动的策略把触发与程序行为关联。TASR^[73] 在成对的输入输出系统调用之间触发，依据是任何输出类系统调用都可能被用于泄露、任何输入类系统调用都可能被用于上传载荷；ReRanz^[75] 以数据传输的累积量作为触发条件。二者关联的都是可能被用于泄露的行为而非实际发生的泄露，以系统调用或 `copy_to_user` 一类的接口作间接代理，对 I/O 密集型负载触发过频，对不经这些接口的泄露则不触发。

探测驱动的策略以检测到的探测行为触发。SafeHidden^[43] 检测对用户态进程中信息隐藏安全区的试探并在触发后重随机化该区域，其保护对象是数据而非代码；MProbe^[83] 检测对用户态代码的地址探测与内容探测并收回被探测代码的执行权限；RandFun^[84] 在检测到探测后随机化被探测函数内部的代码块。这一路线与本章的触发哲学一致，但三者的保护对象都在用户态，且都回避了搬动函数地址：或者保护的是数据，或者只收回权限，或者只重排函数内部而把函数留在原处。

上述策略共同的不足在于触发判据与信息泄露之间只有间接关联。本章的观察是，持续随机化的基础设施本身就可以直接识别信息泄露：随机化区域的当前地址只存在于跳转桩这一处，对它的地址读取就是泄露本身，不需要任何代理。据此可以精确确定需要随机化的函数，即那些跳转桩被读过的函数。

4.2.3 保护整个代码区域的代价

只执行内存用于防御信息泄露时，现有方案默认代码区不被读取。这在用户态成立，进程不读自己的代码；在内核中不成立，内核持续地读取自己的代码：动态跟踪设施在插入与恢复探测点时读取原指令，指令替换与静态键在启动期读取并改写代码，模块加载器与实时补丁在应用重定位时读取代码，性能分析工具在栈回溯与符号解析时读取指令流。防护式的内核只执行内存因此必须为这些合法读取者维护一份豁免清单，漏掉任何一条都会使内核在该路径首次执行时崩溃。

两项内核只执行内存工作给出了两种让步方式。kR^X^[93] 的处置是永久性的局部豁免：它克隆 `get_next` 与 `peek_next` 家族的七个例程以及 `memcpy`、`memcmp`、`bitmap_copy` 共十个函数，使克隆版本不受其代码读取约束的插桩保护，再以约三百三十行补丁把跟踪与调试设施改接到克隆版本上；其论文在讨论向新内核版本移植时进一步说明，eBPF 即时编译器与实时补丁两项设施需要单独处理。ARM 上以硬件调试观察点实现内核只执行内存的工作^[101] 的处置是时间性的整体豁免：内核中存在读取节内字面量池以查询符号信息的合法路径，由于字面量

池位于代码节内，对它的读取与攻击者的代码读取在硬件看来完全相同，该工作在这些路径执行期间翻转观察点控制寄存器的使能位，把保护整体关闭。

两种让步的形式不同，根源相同：只要保护对象是整个代码区，就必然要为内核自身的读取者开口子，而这份清单既随内核版本变化，又必须完备。安全性与正确性被绑在了一起。

4.2.4 研究动机与挑战

上述三条线各自指向一个挑战。其一，如何使只执行保护不依赖一份必须完备的豁免清单，这决定机制能否在真实内核上部署。其二，如何把读取事件转化为可靠的随机化触发判据，并使从触发到完成的延迟短于攻击者可利用该次读取的时间，这决定触发式随机化是否真的比固定间隔更及时。其三，如何使常驻开销与随机化范围的规模解耦，这决定覆盖面能否扩大到全内核而开销不随之上升。

4.3 系统设计

4.3.1 总体架构

图 4-1 给出本章机制的总体结构：跳转桩区受只执行保护，读取事件驱动跳板激活与随机化；候选集合可扩至全内核，激活集合只含被读过的函数。它建立在第 3 章的三区域布局之上，改动集中在两处：跳转桩区被置于只执行内存保护之下，对它的数据读取产生硬件事件；跳板改为两级，每个候选函数在未被读取时停留在未激活态，只付两条直接分支的代价，被读取后激活为完整的跳板并纳入随机化。

机制的运行分三步。候选集合在编译期确定，可以覆盖全部可搬移的 **built-in** 函数；候选函数的一级跳板在未激活态只是一条到跳转桩的直接分支。对跳转桩的任何数据读取被硬件捕获，事件处理记录被读的函数、通知随机化线程、放行本次读取。随机化线程把被读函数激活：改写其一级跳板使之转向二级跳板，二级跳板承担第 3 章跳板的全部职责，该函数随后被随机化，攻击者读到的值失效。被激活的函数在一段时间内未再被读取后降回未激活态。

4.3.2 只执行代码区域的设计

保护对象取跳转桩区而非整个代码区。经第 3 章的处理，随机化区域的当前地址在内存中只存在于跳转桩的分支立即数中：函数内引用是相对的，跨函数引用指向位置固定的跳板，函数指针的值就是跳板地址，栈上的返回地址是与随机化无关的标记。读取跳转桩区等于一次性获得整个区域的布局，除此之外没有别的途径。

正常执行到达函数体的路径全部由直接分支构成：一级跳板是一条到跳转桩的直接分支，跳转桩是一条到函数体的直接分支，两者都只被取指，不含任何

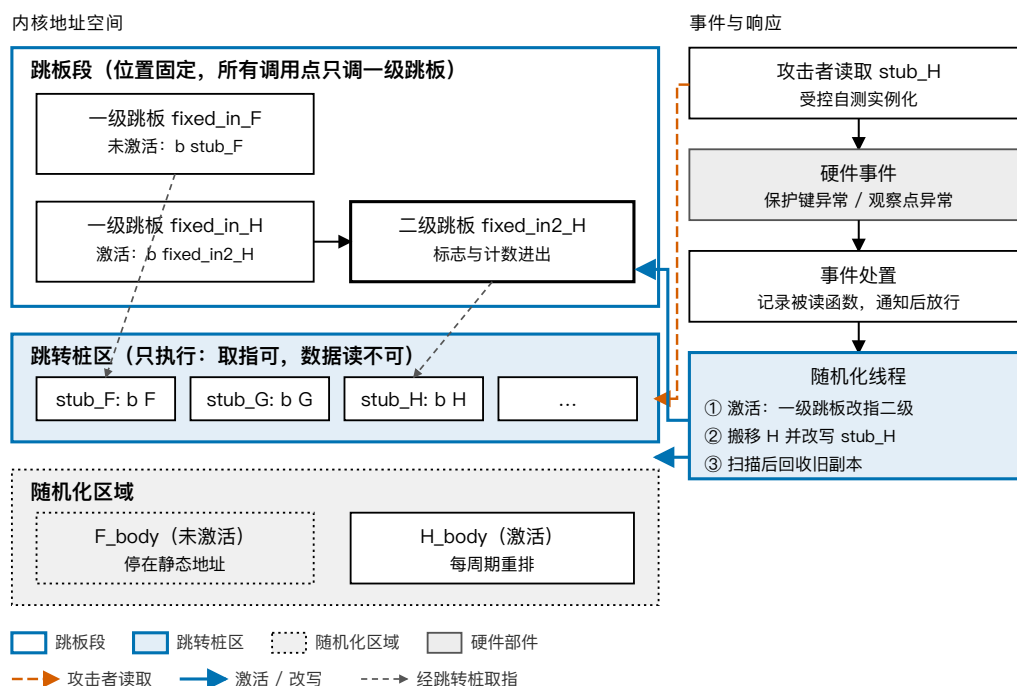


图 4-1 本章机制的总体架构

Figure 4-1 Overall Architecture of the Proposed Mechanism

数据访问；返回标记的解析同样只经重入桩的 PC 相对计算完成。内核的合法读取者读的都是函数体的指令字节，而跳转桩区里没有任何函数体，也没有任何返回地址落在其中，栈回溯与符号解析不会读到它。因此落在跳转桩区上的任何数据读取都没有合法解释，不需要判定读取者的身份，也不需要为任何读取者维护豁免清单。硬件对取指与取数的区分使该判据可由硬件直接给出：只执行内存的两种实现，x86 的管理态保护键与 ARM 的调试观察点，都只作用于数据访问而对取指透明，第 4.4 节分别说明。第 4.2.4 节的挑战一由保护对象的选取本身解决。

覆盖边界随之确定。本章不覆盖对代码区本身的读取：攻击者可以绕开跳转桩，直接扫描随机化区域的代码收集函数内部的片段，这类读取不落在跳转桩区因而不产生事件。对这类读取，本章提供的保护来自随机化本身：所收集片段的地址在该函数下一次被搬移后失效，而失效之后不存在把旧地址翻译回新位置的路径。攻击者经内核自身导出的地址接口或微架构侧信道获得地址的情形不在本章的威胁模型之内。

4.3.3 信息泄露识别

事件的处置。处置只有三步，记录、通知、放行。记录的内容是被读取的槽位与对应的函数；通知交给随机化线程，由它决定何时执行激活与切换；放行使读取者得到当前值，而该值在切换完成后即失效。三步都在异常处理路径上完成，不涉及页映射改写，也不为被读函数准备任何副本。放行是有意的：攻击者

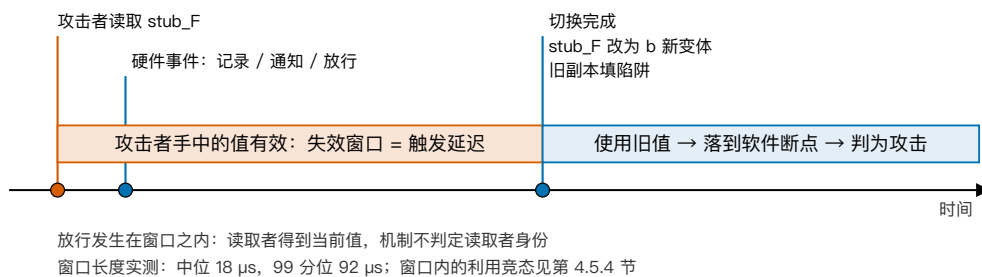
得到的是一个随即失效的值，而放行使机制不必区分读取者身份，也不会因误判而阻断任何执行。

不再需要控制流审计。攻击者使用过期地址的行为已由旧地址失效处理：随机化完成后，被搬移函数的旧字节被覆盖为软件断点，触发即被判定为攻击并终止。本章不需要再对它做一次判定，也就不需要为被读函数准备正常副本与陷阱页，那一层机制及其自身引入的攻击面一并省去。

被读函数集合的生命周期。一次落在跳转桩区的数据读取把该桩对应的函数加入集合，这一步的代价是一次表项写入。集合决定下一次切换重排哪些函数：触发时不对整个候选集合重新布局，只重排集合中的函数，单次随机化的代价因而与被读的规模挂钩，与候选集合的整体规模无关。一次随机化之后，集合中的函数已经迁移到新地址，攻击者此前读到的值随之失效；集合中的函数在一段时间内未再被读取后离开集合并降回未激活态。离开机制不可省略：若集合只进不出，它会随时间单调增长并最终逼近整个候选集合，一次触发的代价退化为全区域重排，激活集合与候选集合的分离也随之消失。

失效窗口。从读取发生到切换完成之间，攻击者手中的值仍然有效，该窗口的长度即第4.5.3节测得的触发延迟。窗口之后，旧地址落到被覆盖的软件断点并被终止。窗口长度是本章的主要安全指标。图4-2给出事件处置与失效窗口的时序：读取被放行，所得的值在切换完成后失效，越过窗口之后再使用该值会落到被覆盖的软件断点。

已激活函数：读取事件的处置与失效窗口



未激活函数：窗口分两段

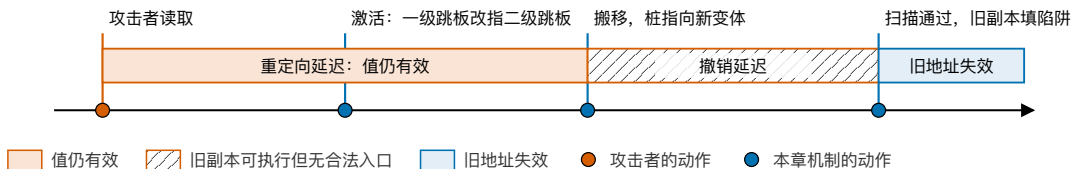


图 4-2 跳转桩读取事件的处置与失效窗口

Figure 4-2 Handling of a Jump-Stub Read Event and the Invalidation Window

4.3.4 两级跳板与跳板激活

第 3 章的跳板对每一次跨区域调用约 53 个周期，其中约 36 个周期来自活跃计数的进出，而计数只对正在被随机化的函数有意义。本节据此把跳板分为两级，使未被读取的函数不计数。

两级跳板的结构。所有调用点调用一级跳板。一级跳板有两种状态，指令序列如下。

```

; 未激活态：一级跳板只是一条到跳转桩的直接分支，不记账
fixed_in_F:
    b        stub_F                ; 跳转桩再以 b F_entry 到达 F 的当前位置；F 直接返回原
    调用点

; 激活态：一级跳板改为一条到二级跳板的直接分支
fixed_in_F:
    b        fixed_in2_F           ; 同一条指令，只改立即数

; 二级跳板与第 3 章的入口跳板完全相同：读当前任务的内外标志，标志为 0 则计数加一、
; 置标志并查阻断标志，标志为 1 则只查阻断标志；经同一个跳转桩调用 F；返回时按进入时
; 保存的标志值决定是否减一
fixed_in2_F:
    ...                            ; 第 3 章 fixed_in 的序列，见第 3.3.2 节

; 出口跳板与第 3 章相同：标志为 0 的执行流不记账，直接调用并按标记返回
fixed_out_G:
    ldr      w17, [x_cur, #inside]
    stp      x17, x30, [sp, #-16]!
    cbz      w17, .Lpassive        ; 标志为 0：未经计数进入，不记账也不查白名单
    ...                            ; 标志为 1：第 3 章的出口跳板序列
.Lpassive:
    bl       G
    ldp      x17, x30, [sp], #16
    b        resolve_token        ; 按标记解析到 F 的当前位置

```

两种状态共用同一对跳转桩。返回标记的解析总是经重入桩完成，与函数处于哪种状态无关，解析逻辑不分叉；被保护的對象仍只有跳转桩一处，未激活函数的当前地址同样只存在于这里，读取它同样产生事件。

出口跳板的记账判据。第 3 章的出口跳板在离开区域时对活跃计数减一、返回时加一，前提是执行流进入区域时曾加一。未激活函数经一级跳板进入时不加一，若出口跳板不加区分地减一，计数就会失衡。第 3 章的跳板已经以每任务的内外标志记录一条执行流是否被计数：从区域外经入口跳板进入时置 1，离开时清零。未激活函数经一级跳板进入时不碰标志，其后的出口跳板读到标志为 0，本次离开不记账，也不查白名单，直接调用外部函数并按标记返回。本章因此不需要改动第 3 章的任何一条跳板序列，二级跳板就是第 3 章的入口跳板。这一判据在任意嵌套下自洽：激活函数 F 经出口跳板调用外部函数 G 时标志已清零，G 再调用一个未激活函数 H，H 的离开与返回都不记账，G 返回到 F 的出口跳板时标志置 1、计数加一，F 返回到二级跳板时清零、减一。未激活函数 H 调用激活函数 F 时，二级跳板读到标志为 0，按从区域外进入处理并计数，F 因而受到

保护。激活前已经进入的执行流标志为 0，其后续的离开同样不记账，因此激活的时机不影响计数的配平。中断、软中断与异常处理程序在入口把标志保存并清零，作为独立的执行流记账（第 3.3.4 节），处理程序经未激活跳板进入区域再向外调用时出口跳板同样读到 0。出口跳板读到 0 只有一个含义，即这条执行流进入区域时未被计数，来源只有未激活的一级跳板。

周期分析。表 4-1 按指令序列给出两级跳板各段的周期数。沿用第 3 章的各段由该章微基准实测的时长按测量机时钟折算；本章新增的两段在关抢占关中断的紧循环中单独计时，口径与第 3 章表 3-3 相同，真实规模下的单次调用开销另见第 4.5.5 节。未激活态的入口只有一级跳板与跳转桩两条直接分支；未激活态的出口只有标志读取、条件分支、调用与标记解析。激活态的入口比第 3 章多一条直接分支，出口与第 3 章相同。

表 4-1 两级跳板各段的指令序列与周期分析

Table 4-1 Instruction Sequence and Cycle Analysis of the Two-Level Trampolines

状态	段	指令	静态指令数	周期
未激活	入口：两条直接分支	b stub_F / b F_entry	2	约 1
未激活	出口：标志判定后直接调用与标记解析	ldr / stp / cbz / bl G / ldp / cbz / b resolve_token	7	约 4
未激活	一次往返合计		9	约 5
激活	入口：一级跳板的直接分支	b fixed_in2_F	1	约 1
激活	入口：标志读写、计数进出、经跳转桩调用	第 3 章入口跳板序列	18	约 37
激活	出口：标志判定、计数进出、白名单、调用与解析	第 3 章出口跳板序列	约 30	约 16
激活	一次往返合计		约 49	约 54

未激活态一次往返约 5 个周期，激活态约 54 个周期，相差一个数量级。常驻开销由此与候选集合的规模解耦：候选集合可以扩到全内核，每次调用只为激活集合中的函数付完整的跳板代价，而激活集合只含被读过的函数。第 4.2.4 节的挑战三由此解决。

激活。激活只改写一级跳板的那一条直接分支，把它的目标由跳转桩改为二级跳板，降级则反向改回。激活是一次 4 字节的原子写入，代价与该函数的调用点数量无关，第 3 章改写条目数等于函数个数这条不变式在激活操作上同样成立。激活之后，新进入的执行流全部经二级跳板计数，随机化线程按第 3 章的流程搬移该函数并改写跳转桩。

激活后旧副本的回收。激活之后，该函数在激活前的执行流仍可能停留在旧副本中，旧副本要等它们全部离开才能填陷阱。这批执行流不需要逐个跟踪，返回标记替它们完成了大部分排空：向外调用与阻塞的执行流在返回时按标记解析，跳转桩已指向新变体，它们自动落到新位置；睡眠或阻塞的执行流一定阻塞在被调用的外部函数里，返回时同样落到新变体。旧副本的滞留者因此只剩在函数体内直线执行、一次外部调用都没发出的执行流，其上界即第 3 章实测的等待清空时间，0.04 至 0.18 μs 。残余的被抢占执行流以一次性扫描判定：先

激活，使不再有任何合法路径新进入旧副本，再扫描全部内核栈、被切换出去的任务其保存现场以及各核的当前程序计数器，检查是否有落在旧副本区间内的返回地址或程序计数器。扫描通过即填陷阱；扫描发现滞留者则推迟重试，兜底为 `synchronize_rcu_tasks`，内核回收 `ftrace` 跳板面对的是同一个问题，用的正是这个原语。兜底的宽限期为毫秒量级，置于关键路径之外。图 4-3 给出两级跳板的状态与激活流程：左侧是未激活态与激活态的一级跳板及二者共用的跳转桩，右侧是激活后的四步，改写一级跳板、计数进入、搬移、扫描后回收旧副本。

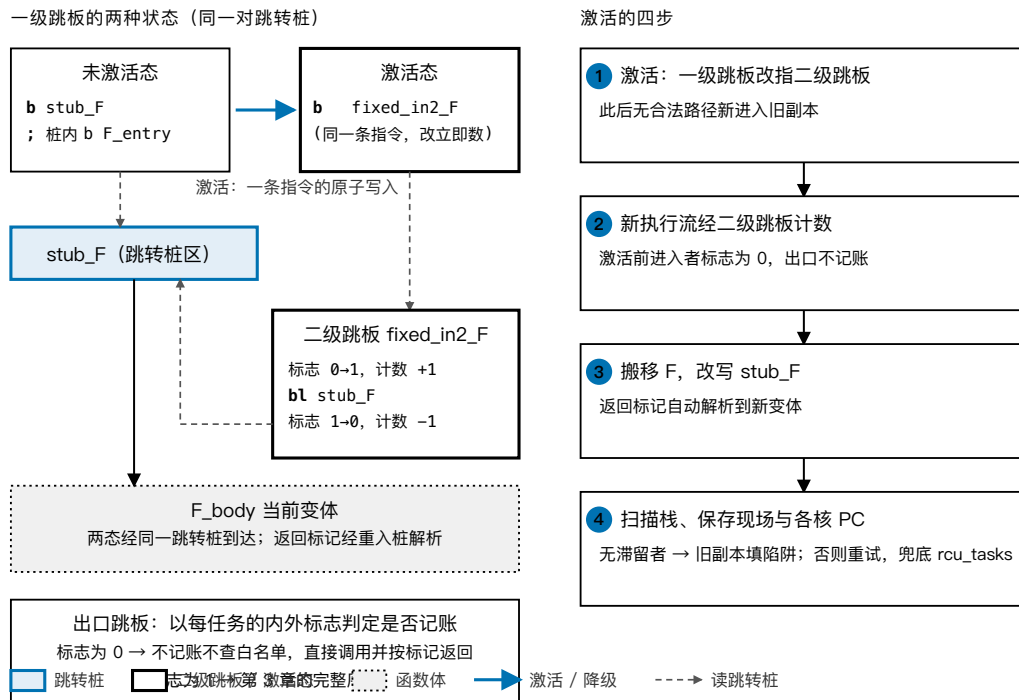


图 4-3 两级跳板与跳板激活

Figure 4-3 Two-Level Trampolines and Trampoline Activation

降级。被激活的函数在一段时间内未再被读取后降回未激活态：把一级跳板改回到跳转桩的直接分支，此后新进入的执行流不再计数。降级不需要排空，因为未激活的函数不会被搬动。降级的时机由离开策略决定，可选随机化后清空、按时间老化、按容量上限淘汰三种，本章实现采用随机化后清空。

4.4 系统实现

4.4.1 x86 平台：基于管理态保护键的只执行内存

内核之上不存在更高的强制层，以内核自己维护的页表实现的只执行内存可被内核态任意写关闭，且同一物理页在内核中通常同时存在于内核映像映射与直接映射之下，后者是前者的别名映射，若只作用于其中一份映射，攻击者经由另一份即可读到内容。这一问题不必依赖虚拟化层解决。IskiOS^[102]指出保护键记录在页表项中，只要让两份映射使用同一个保护键，别名即被一并覆盖。本

文采用更直接的做法，且在两个平台上一致：把跳转桩区所占的物理页从直接映射中撤去，使其只剩内核映像映射一份，只执行保护只需施加在这一份映射上。撤去直接映射复用内核既有的接口，`memfd_secret` 与 `vmalloc` 释放可执行内存时用的是同一条路径；x86-64 的直接映射以大页建立，撤去其中的页需要先把所在的大页拆成 4 KB 页表项，跳转桩区连续且对齐，拆分只波及它所在的少数几个大页。本节据此以管理态保护键实现 x86 上的只执行内存。

保护键最初用于用户态的进程内隔离，已被用来以极低的域切换成本在同一地址空间内划分互不信任的部分^[17,18]，其管理态版本近年又被用于内核分区化^[20,21]。管理态保护键为每个页表项分配一个键，并由一个每处理器核的寄存器保存各键的访问禁止与写禁止位。关键性质是保护键只作用于数据访问，取指不受其约束，因此把跳转桩区的页标上某个键并置位该键的访问禁止位之后，经跳转桩的直接分支照常取指而跳转桩不可被读取，只执行语义直接成立。

每核寄存器这一点对检测式用法尤其重要。事件处理要放行本次读取，需要短暂地开放读权限。在以二级页表实现的方案下，权限作用于物理页而物理页为所有核共享，放行窗口对其它核同时打开，攻击者可在别的核上趁此读取而不被检测。保护键的权限状态保存在每核寄存器中，开放与恢复都只影响本核，不经页表也不需要跨核失效，上述窗口不存在。更新保护键寄存器需要一次写模型专用寄存器的操作，其代价在百周期量级；稳态下寄存器保持访问禁止，只有在一次读取事件发生时才临时开放一次，该代价只在事件上支付。两个平台共同的映射布局、各自的硬件保护以及改写跳转桩的路径见图 4-4。

4.4.2 ARM 平台：基于硬件调试观察点的实现

页表权限在 AArch64 的 EL1 上无法表达只执行：去除内核页的读权限会产生取指中止异常，无论该页由操作系统页表还是由二级页表映射。这一结论针对本文实验平台所属的 ARMv8 与 ARMv9.0 架构，较新的权限间接与权限覆盖扩展改变了页表权限的表达能力。

在不提供只执行权限的嵌入式平台上，已有工作以非特权访存指令配合内存保护单元^[88]或以调试监视比较器^[89]达到同样的语义，后者与本节的思路最为接近。硬件调试观察点提供了另一条途径，且与只执行语义天然契合：观察点只对数据访问触发，取指从被监控区域取出指令不产生任何异常。观察点的配置位于调试寄存器中，独立于内核所维护的页表之外。覆盖范围由地址掩码决定而非由观察点数量决定，`DBGWCR<n>_EL1` 的 MASK 域可掩蔽 3 至 31 位地址，单个观察点最大可覆盖 2 GB 区域，跳转桩区集中放置后一个观察点即可覆盖。用观察点实现内核只执行内存已有先例^[101]，本节复用同一硬件机理，差别在于监控对象与事件处置：既有工作监控整个 `text` 区间并把观察点异常作为终止信号，因而必须为读取字面量池的合法路径整体关闭保护；本节只监控跳转桩区，字面量池位于代码节内而不在跳转桩区，合法读取者的读取不落在监控范围内，保护始终处于开启状态，观察点异常作为触发信号交给随机化线程。

观察点是每个处理器核独立的调试资源，必须在每个核上分别配置，并在处理器上下线与电源状态切换时重新编程。内核的 `hw_breakpoint` 子系统统一管理这些寄存器并向 `perf` 的硬件断点与 `ptrace` 的硬件观察点提供服务，本机制经该子系统注册一个保留项长期独占其中一个，并在上下线回调中重建配置。掩码机制要求被监控区域按其大小对齐且大小为 2 的幂，这一约束由第 3 章的布局分配满足；由于监控对象只是跳转桩区而非整个代码区，对齐带来的地址空间取整代价很小。观察点作用于虚拟地址区间，跳转桩区所在的物理页在直接映射中另有一份别名，观察点只盯映像映射那段虚拟地址，管不到直接映射那段。这正是两个平台都撤去直接映射的原因：撤去之后跳转桩区只剩被监控的那一份映射，任何数据读取都必经它。`arm64` 只在 `rodata_full` 配置下允许撤去直接映射中的页，该配置是默认值，此时线性映射本就按 4 KB 页建立，不存在拆分大页的代价。

跳转桩的改写路径。撤去直接映射并把映像映射置为只执行之后，随机化线程自己也失去了写跳转桩的路径，改写经内核修改代码的标准路径 `text_poke` 完成：在关中断的条件下为目标页建立一份每核私有的临时可写映射，`x86-64` 经专用的改写地址空间加 `fixmap`，`arm64` 经 `FIX_TEXT_POKE0` `fixmap`，写入立即数后立即撤去该映射。这份映射在其存在期间是跳转桩区的第三份别名，但它只在本核可见、关中断期间存在、寿命为一次写入，且改写发生在活跃集合已空、阻断标志置位的切换阶段，攻击者要利用它必须恰好在这个核、这一段微秒级窗口内发起读取。第 6.2 节把这一临时映射列入本文的残余风险。图 4-4 给出两个平台共同的映射布局、各自的硬件保护与改写路径：上半是撤去直接映射之后只执行保护所施加的那一份映像映射，`x86` 以保护键实现、`ARM` 以观察点实现，权限状态都在每核寄存器中；下半是改写跳转桩时经 `text_poke` 建立的每核私有临时可写映射。

4.4.3 两级跳板的代码生成

编译器为每个候选函数生成一级与二级两个跳板。一级跳板保留原函数名并承担取地址语义，其内容在两种状态下都是一条直接分支，未激活态指向跳转桩，激活态指向二级跳板，在 `AArch64` 上为 4 字节的 `b`，在 `x86-64` 上为 5 字节的 `jmp rel32`，激活只改写这一条指令的立即数。二级跳板即第 3 章的入口跳板。跳转桩集中放在跳转桩区，该区域按 2 的幂对齐以满足观察点掩码的要求；一级跳板与二级跳板位于位置固定的跳板段。每个候选函数因此比第 3 章多一个一级跳板，体积代价按候选规模线性增长，其实测值见第 4.5 节。

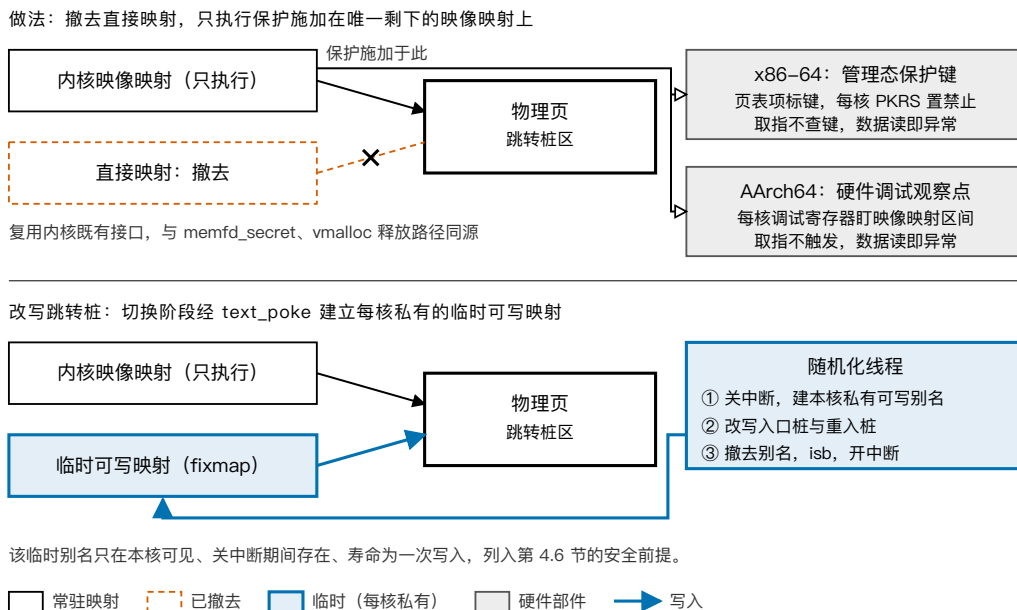


图 4-4 只执行保护施加的映射与跳转桩的改写路径

Figure 4-4 The Mapping That Carries Execute-Only Protection and the Jump-Stub Rewrite Path

4.5 实验评估

本节回答四个问题：机制在何种负载下会被内核自身的读取触发、触发之后多快使泄露失效、两级跳板是否真的把常驻开销与候选范围解耦、以及触发延迟是否足以在攻击者利用泄露地址之前使其失效。全部攻击行为由内核内受控自测驱动：本文不引入任何向用户态暴露内核读写的攻击载体，由内核内一段受控代码按设定的策略读取跳转桩，以此实例化攻击者读取跳转桩这一被检测行为。

4.5.1 实验配置

主平台 ARM Neoverse N2（观察点路径），副平台 Intel Xeon Gold 6448Y（管理态保护键路径），均 128 GB DDR5，Intel X710 万兆网卡，Samsung PM9A3 NVMe，Linux 6.8，LLVM 14。负载集合固定为下列两组，全章各实验引用时按此命名。

基础负载组：空载（仅系统守护进程）、LMBench 3.0 全套、内核编译（make -j64 defconfig）、fio（randread-4k、seqread-1m，-- direct=1）、netperf（TCP_STREAM -m 1024、TCP_RR -r 1,1）、wrk 对 nginx（-t8 -c256 -d30s）、SPEC CPU2006（-- size=ref -- iterations=3）。

设施开启组：在内核编译负载上分别开启 ftrace function tracer、在十个热点函数上设 kprobes、perf record -e cycles、挂载一个 eBPF 跟踪程序、加载一个 livepatch 补丁、执行 kexec -l（只装载不切换）。

每个配置 10 次运行，报告中位数与四分位距；延迟类指标另报 p99；每次运行前丢弃首轮预热。

4.5.2 两条路径的单次检测开销

一次代码索引读取所触发的检测开销直接决定了机制在高频读取下的可承受程度。本节在两个平台上分别测量从硬件事件产生到读取指令重新执行完毕的完整路径，并按五个阶段分解：异常入口，即进入处理程序并保存现场；定位槽位与记录，即由出错地址算出被读的跳转桩序号并写入被读函数记录；通知，即把该函数交给随机化线程；权限开放与恢复，即为放行本次读取临时开放权限并在读取完成后恢复；返回，即退出处理程序并重新执行被中断的读取指令。每个平台各测 10 次取中位，结果见表 4-2。

表 4-2 两个平台单次检测开销的分解

Table 4-2 Breakdown of the Single-Detection Cost on the Two Platforms

阶段	ARM 观察点		x86 保护键	
	时长	占比/%	时长	占比/%
异常入口	0.31 μ s	23.5	0.28 μ s	29.2
定位槽位与记录	0.24 μ s	18.2	0.24 μ s	25.0
通知随机化线程	0.18 μ s	13.6	0.18 μ s	18.8
权限开放与恢复	0.42 μ s	31.8	0.15 μ s	15.6
返回并重执行读取指令	0.17 μ s	12.9	0.11 μ s	11.4
合计	1.32 μ s	100.0	0.96 μ s	100.0

两条路径的总开销分别为 1.32 μ s 与 0.96 μ s，同在微秒量级。差异集中在权限开放与恢复一项：观察点路径需要写调试寄存器并做一次上下文同步，占其总开销的 31.8%；保护键路径只需两次写模型专用寄存器，一次开放一次恢复，占 15.6%。其余四项在两个平台上接近，因为它们执行的是同一套软件逻辑，与硬件载体无关。这一分解也解释了为什么两条路径都把权限状态放在每处理器核的部件上：放行窗口只对本核打开，既不需要跨核失效，也不给其它核留下可乘之机。

触发频率与检测到随机化完成的延迟在两个平台上没有实质差别。基础负载组在 x86 平台上的触发频率为每秒 0.02 至 0.41 次，与 ARM 平台的 0.02 至 0.44 次处于同一区间，因为触发来自内核自身的代码读取行为，由软件路径而非硬件载体决定；延迟的中位为 11 至 19 μ s、99 分位不超过 102 μ s，略低于 ARM 平台，与单次检测开销的差值一致。以下各节的数据除另有说明外均测自主平台。

4.5.3 触发频率与触发延迟

本节使用第 4.5.1 节的负载配置，测试不同负载下触发式随机化每秒触发次数以及从事件到随机化完成的延迟分布。

表中数值测于监控范围覆盖整个随机化区域与跳板区的配置。在该配置下，基础负载组的触发频率在每秒 0.02 至 0.44 次之间，即平均数秒才发生一次随机化；开启动态跟踪设施后上升一个数量级，ftrace 达每秒 6.80 次，kprobes 达每

表 4-3 各类负载下的触发频率与读取来源

Table 4-3 Trigger Frequency and Read Sources under Various Workloads

负载配置	触发次数/秒	读操作类型	读取来源
空载	0.02	符号解析	周期性内核线程打印带符号的地址
LMBench	0.11	栈回溯、符号解析	系统调用路径上的回溯与地址反查
fio 随机小块读	0.35	栈回溯	块层完成路径的回溯
netperf TCP_RR	0.28	栈回溯	网络收发路径的回溯
内核编译	0.44	重定位代码	模块加载器应用重定位时读取代码
wrk	0.31	栈回溯、符号解析	Web 服务路径的回溯与地址反查
SPEC CPU2006	0.02	符号解析	几乎不进内核，仅周期线程的地址反查
开启 ftrace	6.80	读指令	插入与恢复探测点时读取函数入口的原指令
开启 kprobes	4.20	读指令	注册探测点时校验指令边界并读取原指令
perf record	5.40	栈回溯、符号解析	采样时的回溯与地址反查读取指令流
eBPF 跟踪	3.60	读指令	即时编译器写入代码页前读取
livepatch	0.90	读指令、重定位代码	应用补丁时读取被替换函数的入口
kexec 装载	1.20	校验映像	校验时读取代码段

秒 4.20 次，因为这两类设施在插入探测点时读取函数体的指令，而函数体在该配置下处于监控范围之内。读操作类型一列说明了收窄监控范围为何有效：栈回溯与符号解析读的是函数体内的指令与内核的符号表，读指令与重定位代码读的是函数体的字节，校验映像读的是整个代码段，这些读取的对象都不在跳转桩区内。这一频度决定了机制在开启跟踪设施的环境中的占空比，也是离开策略要面对的负载。

表 4-4 检测到随机化完成的延迟分布

Table 4-4 Distribution of the Latency from Detection to Re-randomization Completion

负载配置	中位	p95	p99	最大
空载	12 μ s	31 μ s	58 μ s	74 μ s
LMBench	15 μ s	38 μ s	71 μ s	96 μ s
fio 随机小块读	19 μ s	52 μ s	96 μ s	138 μ s
netperf TCP_RR	17 μ s	45 μ s	88 μ s	121 μ s
内核编译	21 μ s	58 μ s	110 μ s	152 μ s
开启 ftrace	24 μ s	66 μ s	128 μ s	176 μ s
开启 kprobes	22 μ s	61 μ s	119 μ s	164 μ s
受控自测（密集探测）	18 μ s	49 μ s	92 μ s	127 μ s

表 4-4 给出检测到随机化完成的延迟，其数值测于与表 4-3 相同的配置。基础负载下中位在 12 至 21 μ s 之间，99 分位不超过 110 μ s；开启跟踪设施后因触发更频繁而略有拉长，最大值 176 μ s。这段延迟就是一次泄露的地址从被读到失效的时间，其安全含义在下一节以竞态实验单独评估。

4.5.4 泄露地址的有效期与利用竞态

第 4.5.3 节测得的触发延迟只说明地址多久失效，不直接回答它是否足够短。要回答这一点，需要把它与攻击者利用一个泄露地址所需的时间放在同一条时间轴上。本节不去估计难以定义的“凑齐 gadget 的时间”，而是让受控自测把从读取跳转桩到控制流真正转到该地址的完整路径跑一遍，直接测量这场竞态的结果。

实验构造。受控自测扮演能力最强的攻击者：预先在一处由它掌握的内核数据结构中放好一个函数指针，并把利用逻辑除目标地址外全部就绪。一次试验分三步。其一，读取某个随机化函数的跳转桩，从其分支立即数得到该函数的当前地址，这一读取落在只执行监控范围内，触发一次随机化。其二，立即把读到的地址写入那个函数指针。其三，经一条内核路径解引用该指针。若解引用发生在随机化完成之前，控制流转到仍然有效的地址，记为攻击成功；若随机化已经完成，该地址已失效，指针目标落到被覆盖的软件断点并被判定为攻击，记为防御成功。

自变量是攻击者的暂存程度，即从读取到解引用之间的时延。攻击者暂存得越充分，这段时延越短。本节扫描这段时延，报告每一档下的攻击成功率，成功率由高到低转折的位置就是防御生效的边界。这样得到的是一条可复现的成功率曲线，而非一个含义模糊的“凑齐时间”，其定义只依赖两个有明确物理意义的时刻：读取与解引用。

表 4-5 泄露地址利用竞态的结果

Table 4-5 Outcome of the Leaked-Address Exploitation Race

读取至解引用的时延	攻击成功率	说明
1 μ s	99.6%	远快于随机化，单地址单次利用几乎必然成功
10 μ s	71.3%	接近触发延迟中位，攻击成功取决于本次随机化快慢
20 μ s	38.5%	多数攻击尝试已被随机化抢先
50 μ s	8.2%	落在延迟分布尾部之外者才成功
92 μ s	1.0%	与 99 分位对齐：99% 的试验中地址已失效
150 μ s	0.1%	超过最大观测延迟，几乎全部失败

图 4-5 中的竖线为第 4.5.3 节触发延迟的中位、99 分位与最大值。从结果来看，多步攻击被可靠阻断。真实的内核代码复用攻击不是单地址单次利用，它需要多个不同函数的地址串成链。每读取一个地址都触发一次随机化，而随机化在读取后 12 至 176 μ s 内完成，使先读到的地址在后续地址读齐之前就已失效，攻击者无法同时持有一个足够大的有效地址集合。表 4-5 中读取至解引用时延超过 92 μ s 时成功率降到 1% 以下，说明任何需要在读取之后再花费近百微秒去准备的攻击都几乎必然失败，而串联多个地址、构造并布置一条完整的劫持链正属于这一类。

但是，单地址的一次性利用是本章不消除的残余风险。若攻击者把一切都预先暂存好，只等填入一个地址就立即在一条热路径上解引用，读取至解引用的时

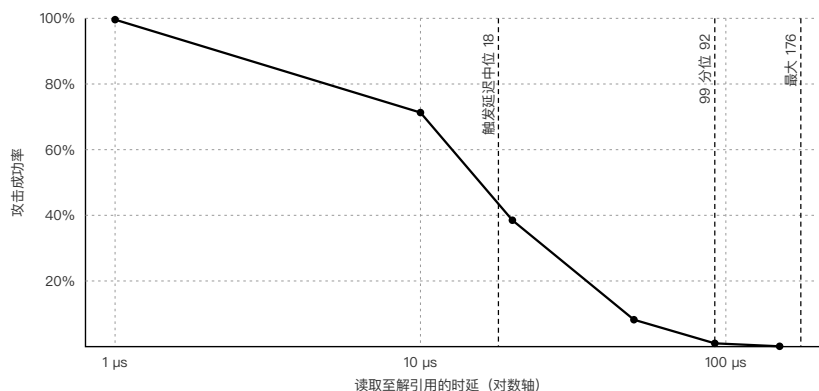


图 4-5 泄露地址利用竞态的成功率随读取至解引用时延的变化

Figure 4-5 Success Rate of the Leaked-Address Race versus the Read-to-Dereference Delay

延可以压到微秒级，此时它快于随机化，表 4-5 的 $1\ \mu\text{s}$ 一档成功率 99.6% 正是这一情形。本章的触发延迟无法赢下这场单次竞态，它保证的是这个泄露地址此后失效、攻击者不能第二次使用同一次泄露，而不是保证第一次使用也失败。填入的这个地址是否合法，需要在控制流转移时另作检查，这正是第 5 章对跨区域转移施加指针认证的原因：被写入函数指针的地址即便当下仍然有效，也无法通过调用点上下文的认证。本章使泄露地址短命，第 5 章使被篡改的指针无法通过验证，两者共同覆盖这场竞态的两侧。

4.5.5 未激活与激活状态的单次调用开销

本节选择不同候选集规模 $N \in \{483, 3,285, 12,000\}$ ，全部置于未激活态，测试一次跨区域调用的边际开销，以及 L1 指令缓存缺失率、iTLB 缺失率与分支预测失败率。另在同一构建上运行 LMBench `lat_syscall` 与 `lat_ctx` 取端到端印证。

未激活态的一次进入只经两条直接分支，在 483 个跳板的紧循环下单次 0.51 ns。紧循环指只反复执行被测操作、不含其它工作的微基准循环，其结果不含指令缓存与分支预测器的压力，是未激活态边际开销的下界。但是 12,000 个候选函数意味着 12,000 个一级跳板与跳转桩，约二百 KB 常热的跳板代码与 24,000 条直接分支，指令缓存与分支目标缓冲的压力在紧循环无法准确测试。

表 4-6 在真实规模下给出这一开销，并暴露了紧循环值掩盖的两项真实代价。候选集合由 483 扩到 12,000 时，未激活态的单次边际开销由 0.51 ns 升到 1.19 ns，升幅来自指令缓存与分支目标缓冲：12,000 个一级跳板约占二百 KB 常热代码，L1 指令缓存缺失率由 0.9% 升到 5.8%，分支预测失败率由 0.6% 升到 2.4%，这两项在 483 个跳板的紧循环里测不到。即便如此，1.19 ns 仍不到激活态 18.79 ns 的十六分之一，`lat_syscall` 与 `lat_ctx` 的端到端增幅在全内核候选下也只有 2.9% 与 3.3%，与第 3 章 483 函数全激活时的开销同量级，而此时随机化的安全覆盖面已扩展到全部内建代码。

表 4-6 未激活与激活状态在真实规模下的单次调用开销

Table 4-6 Per-Call Cost in the Inactive and Active States at Realistic Scale

N	未激活边际开销	激活边际开销	LII 缺失率	iTLB 缺失率	分支预测失败率	lat_syscall 变化	lat_ctx 变化
483	0.51 ns	18.02 ns	0.9%	0.4%	0.6%	+2.1%	+2.4%
3,285	0.77 ns	18.31 ns	2.3%	1.1%	1.2%	+2.4%	+2.7%
12,000	1.19 ns	18.79 ns	5.8%	3.7%	2.4%	+2.9%	+3.3%

4.6 安全性分析

威胁模型。攻击者具备任意内核地址读写能力，目标是获知随机化区域内代码的当前地址并在其失效之前完成控制流劫持。系统已部署持续随机化的执行机制（本文原型见第 3 章），随机化区域的当前地址只存在于跳转桩区的分支立即数中。非随机化区域的控制流完整性由既有内核 CFI 提供（kCFI^[13]、FineIBT^[14]、PAL^[100]；本文原型见第 5 章）。

本章的安全保证。其一，当前地址只存在于受硬件保护的位置：跳转桩区处于只执行内存之下，任何数据读取都产生硬件事件，不存在不产生事件的读取路径。其二，读取即触发：事件处理把被读函数交给随机化线程，该函数被激活并搬移，攻击者读到的值在切换完成后失效，失效之后不存在把旧地址翻译回新位置的路径，使用旧地址的控制流落到被覆盖的软件断点并被终止。其三，从事件到失效的窗口有实测的上界：第 4.5.3 节给出该窗口的分布，基础负载下 99 分位不超过 110 μ s；第 4.5.4 节的竞态实验进一步给出这一窗口的安全含义，当读取至解引用的时延达到 92 μ s 时，攻击成功率已降至 1.0%。这三条合起来说明，攻击者可以读到当前地址，但可利用它的时间以失效窗口为上界。

未激活函数的暴露面。未激活函数的当前地址同样只存在于跳转桩中，读取它同样产生事件，但它在被读取之前停在一个静态地址上，读取之后要经过激活与搬移才失效。该函数在被读取之前的地址本就静态，读取之后激活并未制造新的暴露，而是给这段暴露装上了终点。因此未激活函数的失效窗口由两段构成：从读取到跳转桩指向新变体的重定向延迟，以及从激活到旧副本填陷阱的撤销延迟，二者相加才是完整的窗口。

4.7 本章小结

本章把随机化范围由编译期静态选定改为由泄露信号驱动的运行时变量。随机化区域的当前地址集中在跳转桩的分支立即数这一处，对它施加只执行内存，把它当作数据读取的行为就成为可观测事件；正常执行只取指，内核的合法读取者读的是函数体的指令而非跳转桩，机制因此不需要维护任何豁免清单。事件驱动被读函数进入随机化，触发判据与信息泄露之间不再有任何代理。

跳板分为两级之后，未被读取的函数只付一次间接转移的代价，约 5 个周期，被读取的函数才激活为完整的跳板，约 54 个周期。常驻开销由此与候选集合的规模解耦，候选集合可以扩到全内核，而每次调用只为激活集合中的函数付

完整的代价。激活是一次4字节的原子写入，旧副本的回收由返回标记完成大部分排空，残余以一次性扫描判定。

实验表明，单次检测开销在 ARM 观察点路径上为 1.32 μs ，在 x86 保护键路径上为 0.96 μs ；基础负载下内核自身的读取每秒触发 0.02 至 0.44 次，开启动态跟踪设施后上升到每秒 0.90 至 6.80 次，其中 ftrace 与 kprobes 分别为 6.80 与 4.20 次；从检测到随机化完成的延迟在基础负载下中位为 12 至 21 μs 、99 分位不超过 110 μs 。竞态实验表明，当读取至解引用的时延达到 92 μs 时，攻击成功率降至 1.0%，达到 150 μs 时降至 0.1%。

本章的机制使随机化区域随泄露事件伸缩，随机化区域之外始终存在布局不变的非随机化区域，其中的代码如何避免被复用，由控制流完整性机制回答（本文原型见第 5 章）。

第 5 章 基于随机化辅助的 ARM 指针认证机制

5.1 引言

研究内容二给出了一种低开销的触发式随机化方法，但它同时为内核地址空间留下一片非随机化区域，其中的布局在运行期不发生变化。这并非某一种方法的缺陷：受开销所限，现有的内核持续随机化方法全部只能做部分随机化，Adelie^[34]、Chaos^[35]与Dbox^[37]以模块与驱动为界，随机化区域之外的核心内核保持静态。非随机化部分的防护因此是所有这类方法都必须面对的问题。本章旨在给出一种与具体随机化方法解耦的非随机化区域保护方法。

布局长期不变意味着攻击者一旦持有一个合法的代码指针值，它就长期可用。

理想的防御手段是硬件辅助的控制流完整性，其中 ARM 指针认证的上下文由软件任意选取，原理上可以为每个调用点取不同的值，但受上下文传播约束：多个调用点调用同一函数时必须共享上下文，该约束沿调用图传播，最终使大量调用点收敛到同一上下文。本章的关键观察是：随机化恰好提供了打断这条传播链的手段。随机化区域内的目标地址不可预测，进入该区域的间接调用不需要指针认证；把造成传播的高扇入函数移入随机化区域，链两侧的调用点即可各自建立唯一上下文。

本章的组织如下。第 5.2 节说明部分随机化留下的非随机化区域、指针认证的上下文传播问题，并归纳挑战；第 5.3 节给出系统设计，包括总体架构与核心思想、以随机化打断上下文传播链的识别与绑定模型、以及跳板处的分区验证；第 5.4 节按编译期、运行期与出口检查三部分给出实现；第 5.5 节评估；第 5.6 节给出安全性分析；第 5.7 节小结。

5.2 研究背景与动机

5.2.1 部分随机化与非随机化区域

现有的内核持续随机化方法以模块与驱动为随机化对象，核心内核保持静态。Adelie 对非随机化区域的处理是把指向它的指针约束为只能经只读全局偏移表访问，指向模块内部的指针则在模块移动后自动失效。这一做法依赖模块与内核之间有限且明确的接口，在 built-in 场景下没有一张表能把跨区域引用收拢。

更一般的处理属于控制流完整性的范畴。软件实现的控制流完整性在内核中的开销不可接受；硬件辅助的实现中，落地点标记类机制（Intel CET-IBT、ARM BTI）只检查目标处有无标记^[11,12]，把所有被标记的函数放进同一个等价类；签名验证类机制（ARM 指针认证）原理上可以做到每个调用点一个上下文，实际精度取决于上下文如何选取。

5.2.2 指针认证与上下文传播

ARM 指针认证在指针的未使用高位中嵌入密码学签名，签名的输入包括指针值、一个 64 位上下文与一个软件不可读的硬件密钥。签名与验证由专用指令完成，各约四个周期，认证码随指针搬移而自然传播，不需要软件协助。图 5-1 给出这一对运算的全过程：指针赋值处以密钥与上下文对指针值计算认证码并写入高位，指针使用处以同一密钥与同一上下文重算并比对，一致则还原出可用的指针，不一致则高位被置为错误码，随后的解引用触发异常。密钥由硬件保存且软件无法读出，攻击者因此无法离线伪造认证码；上下文则完全由软件给出，它决定一个指针能够在哪些位置通过认证。

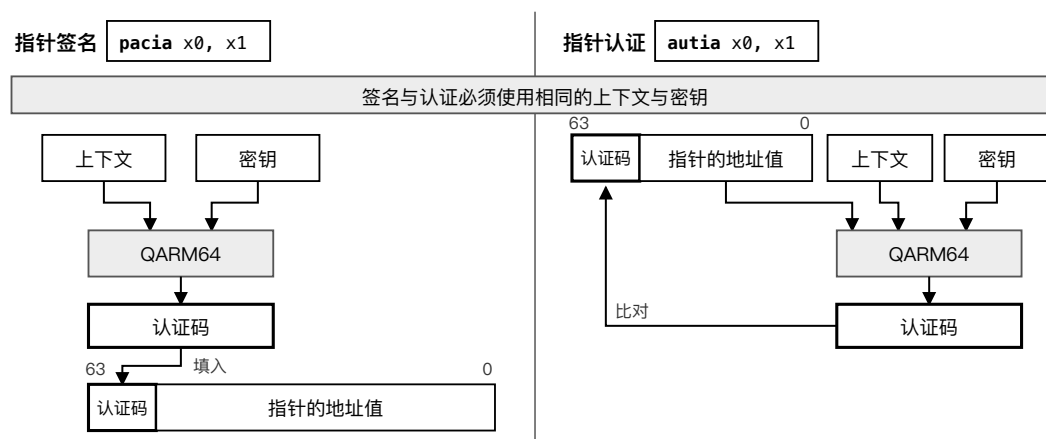


图 5-1 指针认证的签名与认证过程

Figure 5-1 Signing and Authentication in ARM Pointer Authentication

指针认证的精度完全取决于上下文的选取。若每个调用点使用互不相同的上下文，则在某调用点签名的指针无法在另一调用点通过验证，等价类大小为一。这一理想状态无法达到，原因是上下文传播：当多个调用点可能调用同一函数时，这些调用点必须使用相同的上下文，否则被调用方无法验证；该约束沿调用图向外扩散，最终把大量调用点并入同一上下文。图 5-2 给出这一过程：两个调用点共用一个被调函数，就必须共享上下文，该约束沿调用图扩散，把越来越多的调用点并入同一集合。

PARTS^[16]、PACStack^[97]、PACTight^[98]等工作分别改进上下文的构造方式或扩展保护对象，面向内核的指针认证应用亦有若干工作。这些工作都未能从根本上打断传播链，因为传播的根源是调用图的结构本身：只要一个函数同时被两个调用点调用，且它需要验证传入的指针，两个调用点就必须共享上下文。在纯指针认证的框架内，这个约束难以解除。本文在同一平台上按函数类型哈希取上下文的方案，最大等价类为 141（第 5.5.2 节）。

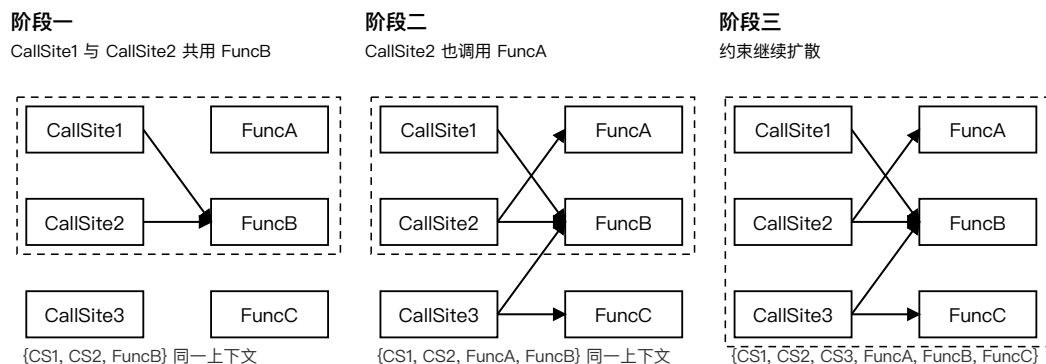


图 5-2 指针认证的上下文传播问题

Figure 5-2 The Context-Propagation Problem of Pointer Authentication

5.2.3 研究动机与挑战

上两节归纳出两点：非随机化区域的保护是所有部分随机化方法的共同问题；商用硬件控制流完整性机制中，指针认证是唯一能以软件选取的上下文区分调用点、因而精度上限最高的一种，但它受上下文传播所限而实际退化。打断传播链需要引入指针认证之外的信息，本章引入的是随机化。

本章面临的挑战有三。其一，如何在不做全局指针分析的前提下让调用点得到唯一上下文，内核规模决定了任何依赖全局点到分析的方案都不具备可部署性。其二，如何避免在每一次间接调用上都施加验证，使开销随随机化范围可控。其三，如何使保护机制不绑定某一种随机化实现，从而可以施加到任何一种部分随机化方法留下的非随机化区域上。

5.3 系统设计

5.3.1 总体架构与核心思想

本章的设计可以概括为：进入随机化区域的控制流以范围检查替代认证，因为目标地址不可预测；造成上下文传播的高扇入函数移入随机化区域，传播链在此剪断；只对离开随机化区域的控制流施加认证与白名单的双重检查。验证施加在跨区域转移的唯一出入口上，不另建检查点。本文原型中这一出入口即第3章的跳板。图 5-3 按时间顺序给出机制的三个阶段。启动早期为识别阶段，指针以类型上下文签名，归属记录跟踪每个函数的首个调用点，第二个调用点触及即判为高扇入。启动推进中为绑定与迁移阶段，单一归属的函数重签为调用点唯一上下文，高扇入函数移入随机化区域，指向它的指针改写为跳板地址。运行期为分区验证阶段，入口只验范围，出口验认证与白名单。绑定后新出现的归属冲突经仲裁后同样迁移，图中底部的回边表示这一情况。

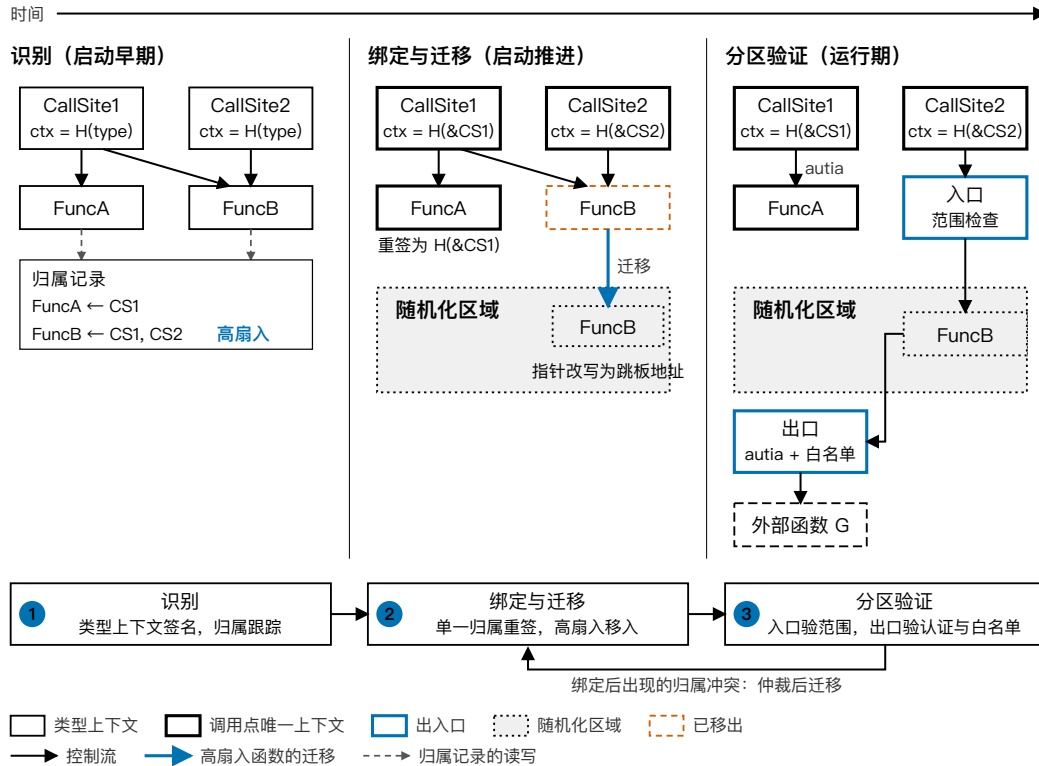


图 5-3 本章机制的总体架构

Figure 5-3 Overall Architecture of the Proposed Mechanism

5.3.2 以随机化打断上下文传播链

识别。识别高扇入函数无法被静态分析，它可以作为验证流程的自然副产品被观测到。

本章采用指针在存储中始终以类型上下文签名，另以一条归属记录跟踪首个调用它的调用点。类型签名是合法性的真值来源，存储中的函数指针恒以函数原型的哈希为上下文签名，任何调用点都能以类型上下文认证其合法性，认证失败即为篡改。归属记录识别扇入：首个使用该指针的调用点将其认领，同一调用点再次使用属正常，另一个调用点使用同一指针即判定为高扇入。识别只读归属记录，不改写指针本身的签名，因而不会陷入上述矛盾。

上下文绑定。精度来自上下文绑定，识别只负责分类。上述识别把函数分成单一归属与高扇入两类。对单一归属的函数，在识别稳定后由绑定阶段把它改签为其归属调用点的唯一上下文并不再变更。绑定之后等价类以调用点为单位划分：每个函数至多归属一个调用点，一个调用点却可以归属多个函数，它经同一指针变量先后调用的全部函数都以它的上下文签名，其合法目标集合就是这些函数。单目标调用点的等价类为一，分发类调用点的等价类等于它分发到的函数个数，这是调用图的固有结构而非机制的损失。对高扇入函数则移入随机化区域，此后不再参与验证，传播链在此处被切断。绑定之后调用点的等价类不再增长：绑定后新出现的目标，无论是新的调用点使用已绑定的函数，还是已绑定的调用点使用新的函数，一律交由随机化承担，处理路径见第 5.4.2 节。

一个函数一旦被第二个调用点调用，它就不再需要指针认证，改由持续随机化保护，这条规则对绑定前后同样成立。绑定前的第二个调用点由归属记录直接判出。绑定后的第二个调用点无法通过认证，因为内存中的指针只携带第一个调用点的认证码，它既通不过自己的上下文也通不过类型上下文，认证本身无法区分“绑定后新出现的合法调用点”与“指针被篡改”。本章把这一区分交给归属记录：认证失败进入违规处理路径，处理路径按指针的存储位置查记录，若记录已绑定、且指针的值等于记录中的目标函数，则判定为绑定后的归属冲突，该函数即刻判为高扇入并进入随机化；指针与记录不符则维持违规判定。转换完成后，指向该函数的指针被改写为其入口跳板的地址，不再携带认证码，此后无论第一个还是第二个调用点，调用它都只做范围检查。图 5-4 给出一次识别的时序、绑定的时间点与绑定后归属冲突的处理：指针签名在识别阶段恒为类型上下文，归属由独立记录跟踪，归属冲突即判为高扇入并移入随机化区域，单一归属的函数在绑定时重签为调用点唯一上下文。识别贯穿内核启动过程：启动早期所有指针以类型上下文签名以保证兼容，随着执行推进、归属趋于稳定，单一归属的函数被逐步绑定，启动完成后大部分单一归属的函数已建立唯一上下文。

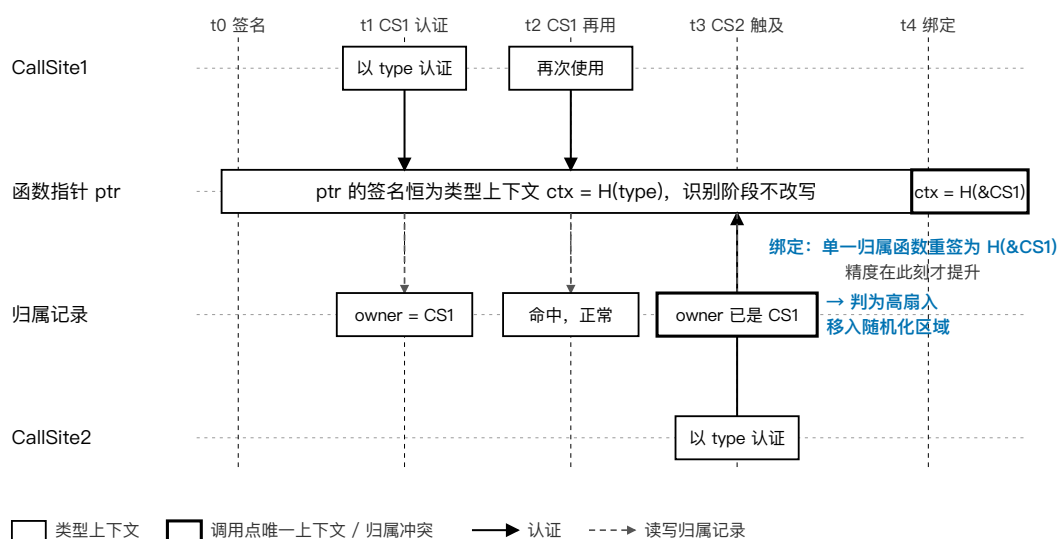


图 5-4 高扇入函数的识别与绑定

Figure 5-4 Identification and Binding of High-Fan-In Functions

信任根。唯一上下文的建立依赖启动阶段不存在控制流劫持：若攻击者在启动阶段即可劫持控制流，就可以构造恶意调用点，用该调用点的上下文为任意目标签名，此后即可用该上下文通过验证。该假设由安全启动链保证，绑定因此落在第一个用户态进程启动之前。此外假设攻击者在劫持成功之前不能任意改写内存权限，使随机化过程中更新的代码页不会被预先篡改。

5.3.3 跳板处的分区验证

验证按控制流的方向差异化处理。

进入方向以范围检查替代认证。随机化区域内的目标地址不可预测，攻击者无法把指针改写为区域内某个函数体的地址。但对函数取地址得到的是跳板地址，而跳板位置固定且公开，攻击者把函数指针改写为某个入口跳板即可确定性地到达对应函数的入口，因此入口不能完全免除检查。本章在每个间接调用点先判断目标是否落在跳板区域之内：落在其中，说明目标函数处于随机化状态，只做这一次范围检查；不在其中，目标位于非随机化区域，以调用点上下文认证。跳板区域位置固定、边界在链接期确定，这是一次常数时间的边界比较，代价远低于一次认证指令加上下文计算。指针的形态由此决定检查方式：带认证码的指针走认证，跳板地址走范围检查。一个函数由非随机化状态转为随机化状态时，只需把指向它的指针改写为跳板地址，不需要改动任何调用点的指令。它保证的性质是以调用形式进入随机化区域的控制流只能落在函数入口，因为跳板是调用进入该区域的唯一形式，而每个跳板对应一个函数的入口。返回方向经返回标记落到函数内偏移，其落点在函数中部而非入口，且不经范围检查；这条路径的约束来自标记本身的完整性，标记以栈槽地址为上下文签名，搬到别的栈槽即认证失败。

离开方向强制认证加白名单。目标位于非随机化区域，地址长期不变。出口依次执行：以调用点唯一上下文认证目标指针，检查目标属于编译期确定的合法外部函数集合，两项都通过才放行。两项检查约束的对象不同：白名单约束目标是否属于合法集合，属于集合成员性检查；指针认证约束这个指针是否由本调用点签发，属于来源真实性检查。仅有白名单时攻击者可以把一个合法目标替换为白名单内的另一个合法目标，仅有指针认证时攻击者可以复用一個在别处合法签发的指针，两者叠加才构成完整约束。图 5-5 对照两个方向的处理：入口只验范围，保证落点为函数入口；出口验认证与白名单，一验来源一验集合。

对所承接随机化原语的要求。免验的前提是目标地址不可预测，因此免验范围严格等于当前处于随机化状态的函数集合，而非编译期划定的区域。本章据此对随机化原语只提出一项要求：被识别为高扇入的函数必须持续处于随机化状态。本文原型中，第 4 章的激活集合恒含本章的高扇入集合。这一要求之外，本章不依赖随机化原语的任何内部细节，它可以施加在任何提供函数粒度随机化区域与跨区域出入口的方案之上。

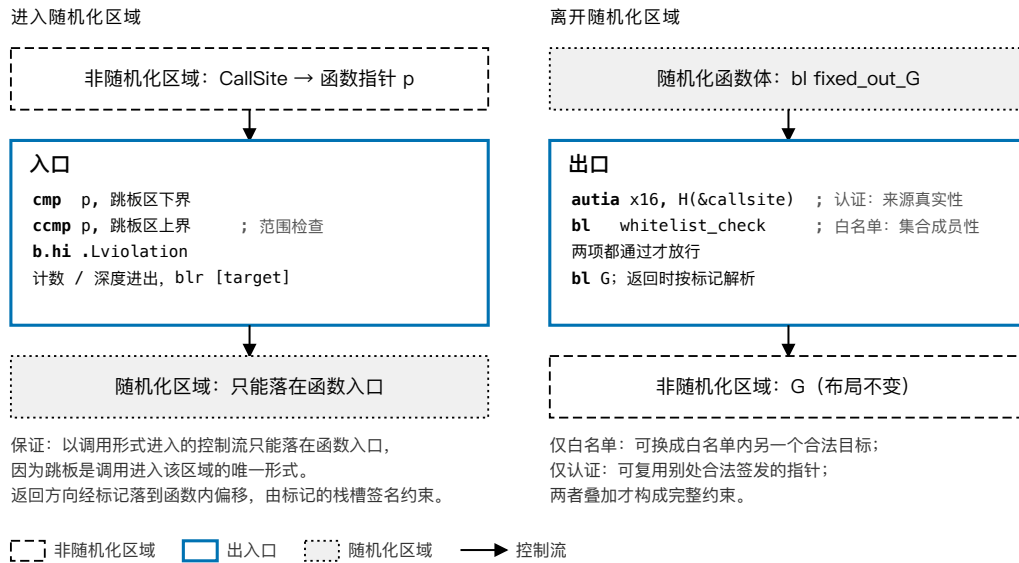


图 5-5 跳板处的分区验证

Figure 5-5 Partitioned Verification at the Trampolines

5.4 系统实现

5.4.1 编译期：指针认证插桩

本章在 LLVM Pass 上扩展两类插桩：函数指针产生处插入签名指令，初始使用类型上下文；间接调用处先做一次跳板区域的范围检查，目标落在其中即直接调用；否则插入认证指令，优先使用调用点唯一上下文，失败则回退到类型上下文。类型上下文的回退在绑定之前覆盖识别阶段，在绑定之后则说明该指针是绑定后新产生的；回退也失败则说明指针携带的是另一个调用点的认证码或已被篡改。两种失败都进入违规处理路径，由第 5.4.2 节的归属仲裁决定放行、原位重签、纳入随机化或判定违规。

上下文是指针认证指令的修饰值，它决定一个签名指针只能在哪些位置通过验证，因而是 CFI 精度的载体。上下文在使用处现算，不占用任何长期驻留的寄存器：类型上下文取函数原型的哈希，用于识别阶段中所有存储于内存的指针；调用点上下文取调用指令虚拟地址的哈希，用于绑定之后单一归属函数的签名与验证。两者都是对一个已知输入做一次常数时间的哈希，结果直接作为 PACIA 与 AUTIA 的修饰值，签名与验证不需要读取任何全局状态。

构造上有两处细节。其一是域分离：类型上下文的哈希输入中置一个固定的区分位，使两类上下文的输入空间不相交，否则一个以类型上下文签名的指针有可能恰好通过某个调用点上下文的验证。其二是非零约束：两类上下文都保证取值非零，避免修饰值为零这一退化情形与上下文尚未设置的状态相混。不把上下文基址长期驻留在寄存器中有两重理由：被调用者保存的寄存器长期占用会与内核中的手写汇编及既有调用约定冲突；驻留寄存器的基址一旦被读到即可复用。上下文不驻留于任何位置，攻击者即便读取全部通用寄存器也得不到一个可

复用的基址，伪造一个能通过验证的指针仍然需要密钥本身。

5.4.2 运行期：高扇入函数的跟踪、绑定与迁移

跟踪以每个函数指针一条记录的形式组织，记录中保存指针所在的存储位置、目标函数、其类型哈希、归属调用点、以及两个状态位：是否已判定为高扇入、是否已绑定。某个函数指针首次被某调用点使用且通过类型上下文验证时，该调用点被登记为它的归属；此后同一调用点再次使用属于正常；一旦另一个调用点使用同一指针，归属冲突，该函数即被判定为高扇入。

绑定之后，调用点唯一上下文认证失败的指针分两种情形，都在违规处理路径上按归属记录仲裁，算法 5-1 给出间接调用处从范围检查到仲裁的完整判定过程。其一，类型上下文回退也失败，指针携带的是另一个调用点的认证码，或已被篡改。处理路径按指针的存储位置查记录：记录不存在或未绑定，即判定为控制流完整性违规并终止；记录已绑定且函数指针等于目标函数 **F**，则判定为绑定后的归属冲突，本次调用以 **F** 为目标直接放行，同时把 **F** 排入迁移队列。其二，类型上下文回退通过，指针是绑定之后新产生的，携带类型认证码。若记录已绑定、归属为本调用点、且函数指针等于记录中的目标函数，这只是同一指针值被重新写入，处理路径在原位把它重签为本调用点的唯一上下文，等价类不变；否则，无论是本调用点指向了新的函数，还是另一个调用点使用了该指针，目标函数都排入迁移队列。类型认证通过是纳入随机化的前提，它证明该指针由合法的签名插桩产生，目标是编译器允许出现在这一类型槽位上的函数；否则攻击者只需写入一个裸指针，就能把任意函数送入随机化区域并经跳板调用它。

两种情形下处理路径都不分配内存、不睡眠，可在任意上下文执行。随机化线程在关键路径之外完成排队函数的物理迁移，并把归属记录中登记的、指向它的全部指针存储位置改写为其入口跳板地址，不带认证码，此后所有调用点对这些指针都只做范围检查。在迁移完成之前，每次触发的调用都要经处理路径放行，这是一段微秒级的过渡。取函数地址的一侧也随之一致：迁移后对该函数取地址得到的本就是跳板地址，产生指针处的签名插桩对落在跳板区域内的地址不签名，新产生的指针天然就是范围检查的形态。绑定之后调用点的等价类由此只减不增，新目标一律由随机化承担，而不并入任何调用点的上下文。

归属记录在这条路径上成为合法性的仲裁依据，因此它的完整性是前提：记录在绑定完成后置为只读，只在迁移路径上临时开放写入；攻击者若把指针改写为另一个函数 **G**，函数指针与记录不符，仍被判为违规。

5.4.3 跨区域出口的双重检查

白名单约束从随机化区域到非随机化区域的合法目标。表项来自链接器汇总的专用数据段，编译器扫描随机化代码的目标文件，提取以外部符号或未定义符号形式出现的跨边界控制流目标；内核初始化时把它们装入哈希表，随后将该表置为只读。哈希表的桶数取不小于表项数两倍的、最接近的 2 的幂，使平均链长小于一，查询为哈希定桶加桶内线性比对，因而是常数时间。在白名单尚未完

算法 5-1 间接调用处的检查与归属仲裁

输入： 调用点 CS ，函数指针的存储位置 loc ，其中的指针值 p

输出： 调用目标 t ，或判定为控制流完整性违规

```

1: function CHECKANDARBITRATE( $CS, loc, p$ )
2:   if  $p \in [tramp_{lo}, tramp_{hi}]$  then           // 指向随机化区域：只做范围检查
3:     return  $p$ 
4:   end if
5:    $t \leftarrow \text{AUTIA}(p, H(\&CS))$                 // 调用点唯一上下文
6:   if  $t$  有效 then
7:     return  $t$ 
8:   end if
9:    $t \leftarrow \text{AUTIA}(p, H(\text{type}))$               // 回退到类型上下文
10:   $r \leftarrow \text{Record}[loc]$                       // 归属仲裁：按存储位置查记录
11:  if  $t$  有效 then                                // 指针由合法的签名插桩产生
12:    if  $r$  未绑定 then                             // 识别阶段
13:      if  $r.\text{owner} = \emptyset$  then  $r.\text{owner} \leftarrow CS$ 
14:      else if  $r.\text{owner} \neq CS$  then  $\text{MIGRATE}(t)$     // 第二个调用点：高扇入
15:      end if
16:      return  $t$ 
17:    end if
18:    if  $r.\text{owner} = CS$  and  $t = r.\text{target}$  then        // 同一指针值重新写入
19:       $\text{Mem}[loc] \leftarrow \text{PACIA}(t, H(\&CS))$         // 原位重签，等价类不变
20:      return  $t$ 
21:    end if
22:     $\text{MIGRATE}(t)$ ; return  $t$                         // 绑定后的新目标：纳入随机化
23:  else                                             // 携带其他调用点的认证码，或已被篡改
24:     $t \leftarrow \text{STRIP}(p)$ 
25:    if  $r$  已绑定 and  $t = r.\text{target}$  then            // 绑定后的归属冲突
26:       $\text{MIGRATE}(t)$ ; return  $t$ 
27:    end if
28:    return 违规
29:  end if
30: end function

31: procedure  $\text{MIGRATE}(F)$                         // 不分配内存、不睡眠，可在任意上下文执行
32:  标记  $F$  为高扇入，排入迁移队列
33:  随机化线程在关键路径之外完成物理迁移，并把 Record 中指向  $F$  的全部
    存储位置改写为  $F$  的入口跳板地址
34: end procedure

```

成装入之前查询一律返回通过，以免阻断早期启动路径；这段窗口位于用户态启动之前，与第 5.3.2 节所依赖的安全启动链假设处在同一时段之内。

出口先以调用点唯一上下文认证目标指针，再查询白名单，两者都通过才放行。前者检查指针是否由本调用点签发，后者检查目标是否属于合法集合，二者不可互相替代。

5.5 实验评估

本节回答两个问题：随机化打断上下文传播链之后，控制流完整性检查的精度提高到什么程度、代价多大；这一机制是否真的与具体的随机化方法解耦，会对现有随机化方法产生什么影响。

5.5.1 实验配置

ARM Neoverse N2, FEAT_PAuth, Linux 6.8, LLVM 14。等价类统计在 x86-64 上按目标文件的重定位与调试信息完成，与平台无关。运行期快照以 UnixBench 一次完整运行驱动，开销以 LMBench 与 SPEC CPU2006 测量。10 次运行取中位。

5.5.2 控制流完整性检查的精度与开销

精度以每个间接调用点的合法目标数衡量：在给定机制下，一个间接控制流转移有多少个目标可选。表 5-1 汇总已发表的内核控制流完整性机制与本文的结果，末三行为本文在 Linux 6.8 上的统计，其余各行取自所引原文；其中 BTI 与 Clang CFI 两行的数值由 PAL 在其表 2 中一并给出。

表 5-1 合法目标数分布对比

Table 5-1 Comparison of Distribution of Legitimate Targets per Indirect Call Site

机制	内核	≤ 5	≥ 100	最大	平均目标数
BTI ^[100]	Linux 5.12	0.0%	100%	59,300	-
Clang CFI ^[94]	Android 4.14	55.0%	7.0%	1,153	59
PAL ^[100]	Linux 5.12	84.9%	2.8%	35,264	-
HEK-CFI ^[103]	Linux 5.18	65.3%	7.4%	-	22
类型上下文	Linux 6.8	55.0%	2.0%	141	13.7
唯一上下文，启动完成	Linux 6.8	91.0%	0%	57	3.6
唯一上下文，运行期	Linux 6.8	91.95%	0%	57	3.4

不超过 5 的占比在 Android 4.14 与本文的 Linux 6.8 上都是 55%，而最大值相差一个数量级，1,153 与 141 分别来自 Google 的类型匹配和本文按地址被取过的函数收紧后的分组。HEK-CFI 只报告平均目标数，它把类型哈希在同一内核上的 325 降到 22，是近年内核控制流完整性中精度最高的一支，但其分布未见诸原文。这一组也说明本文选择类型上下文作为对照的理由：它是主线 kCFI^[13] 与 FineIBT^[14] 的划分依据，也是 PAL 未加精化前的起点。上下文精化一组把 PAL 的两级精化与本文的唯一上下文并列。PAL 通过静态分析把上下文从类型细化

到对象类型，再对仍然过宽的调用点绑定对象实例与调用上下文，由分析器自动加入 166 处注释后，不超过 5 的占比升到 90.8%，超过 100 的降到 0.08%，仍有 3 个上下文超过 100 个目标，最大 207。本文以调用点地址为上下文，启动完成时不超过 5 的占比为 91.0%，与 PAL 相差在 0.2% 以内，最大值为 57 且没有任何调用点超过 100，代价是把 233 个高扇入函数移入随机化区域。两者的差别在于高扇入调用点的处置：PAL 靠更细的静态上下文继续切分，切不开的留在大等价类里；本文不再切分，把这些函数交给随机化，等价类的残余上界由此从 207 降到 57。两种做法并不排斥，PAL 的静态精化可以作为本文识别阶段的初始上下文，本文机制只要求初始上下文可比较，不要求它是类型哈希。

其它以精度为主张的控制流完整性工作使用不同的指标，无法与上表并列。Ge 等人^[104]在 FreeBSD 与 MINIX 内核上以 AIA 即每个间接转移允许目标数的平均值报告，kCFI 白皮书 (Moreira 等人^[105]) 同时报告 AIR 与 AIA；平均值把长尾折进均值里，Ge 等人自己也指出 AIR 无法反映过近似的程度，这正是本文同时报告分布、最大值与平均值的原因。OS-CFI^[106]、HPCFI^[107]、Burrow 等人^[15]的综述与 CScan^[108]都以每个调用点的目标集合大小或其运行期可达数为指标，与本文口径同源，但保护对象是用户态程序，数值随程序而异，不能与内核数字并列。

本文类型上下文的最大值 141 见按表 5-2：最通用的几个函数原型聚集了较多函数，一个类型为 `void (*)(void)` 的间接调用在类型哈希机制下有 141 个合法目标可跳。类型分布的长尾性是以类型划分等价类这一选择的直接后果。本文按调试信息中的函数原型分组，把所有枚举折叠为一类、把匿名结构体折叠为一类、并去除类型别名，这比类型哈希机制实际的分组更粗，因此 141 是一个上界，该机制真实的最大等价类只会更小或相等。

表 5-2 类型哈希机制下最大的几个等价类

Table 5-2 The Largest Equivalence Classes under the Type-Hash Mechanism (Tightened Criterion)

类型签名	等价类大小
<code>void (*)(void)</code>	141
<code>unsigned long long (*)(unsigned long long, ...)</code>	133
<code>int (*)(void)</code>	61
<code>int (*)(struct device*)</code>	57
<code>void (*)(struct sock*)</code>	55

本文机制的等价类以调用点为单位：调用点上下文由调用指令地址导出，互不相同，绑定之后每个调用点只接受由它自己签发的指针，其合法目标集合就是它所归属的全部函数。因此单目标调用点的等价类为一，而经同一指针变量分发到多个函数的调用点，等价类等于其目标函数的个数。被判定为高扇入的函数退出这一保护而由随机化承担，其规模由第 5.5.3 节给出。全内核规模的等价类分布按每个间接调用点的合法目标数统计，对照三档上下文策略：常数零上下文

(等价于落地点标记)、按函数类型的上下文、本章唯一上下文。

统计取三个快照时刻。早期初始化指内核启动到第一个文件系统挂载之前，此时绑定尚未开始，全部指针仍以类型上下文签名。启动完成指第一个用户态进程启动之前绑定例程结束的时刻，它同时是第 5.3.2 节所依赖的信任根边界，绑定必须在此之前完成。运行期指启动完成后完整执行一轮 **UnixBench**，其全部子项结束的时刻；选择这一时刻是因为它覆盖了进程创建、文件系统、管道与信号等启动期不会走到的冷路径，用以检验启动期得到的归属是否会被长期运行推翻。分布按目标数逐格统计。

图 5-6 以五条曲线给出各机制的分布：**Clang CFI** 的类型哈希、**HEK-CFI**^[103]、**PAL** 的类型签名，以及本章机制在启动完成与运行期两个快照下的结果。横轴为每个间接调用点的合法目标数，纵轴为取该目标数的调用点占比，末格汇总目标数超过 30 的调用点。各机制的主峰都落在 30 以内，横轴因此只画到 30，峰后的长尾按占比递减。峰后的次级隆起对应按文件系统类型、协议族或设备类别分发的调用点。末格的差距同样明显，目标数超过 30 的调用点在类型哈希下占 13%，在本章机制下降到 2.2%。落地点标记一类机制只有一个等价类，其分布退化为末格上的单点，不列入该图。**HEK-CFI** 以 **Intel CET** 影子栈的写保护页保存控制数据，是近年内核控制流完整性中精度最高的一支，每个间接调用的平均合法目标数为 22。表 5-3 给出三个快照的汇总。

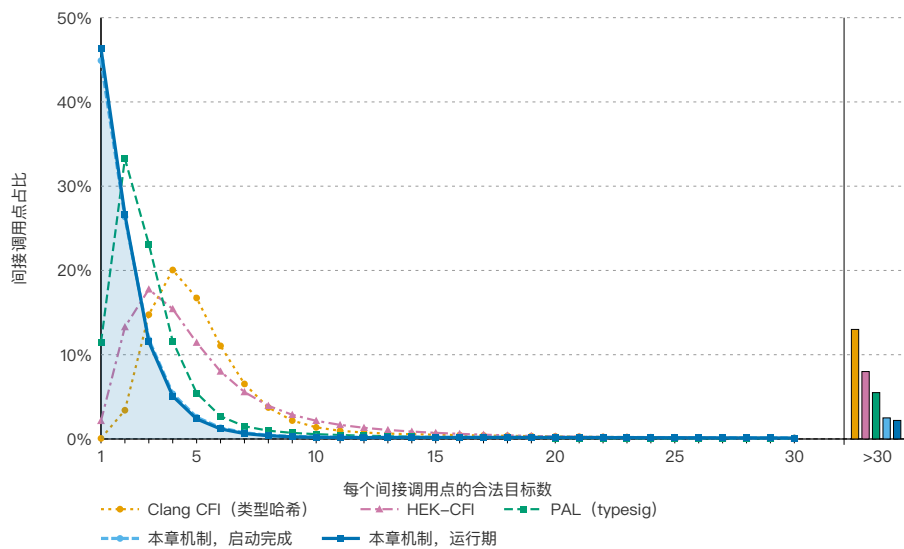


图 5-6 每个间接调用点合法目标数的分布

Figure 5-6 Distribution of Legitimate Targets per Indirect Call Site

表 5-3 与图 5-6 结合来看。早期初始化时全部指针仍以类型上下文签名，唯一上下文一行与类型上下文一行完全相同，55% 的调用点目标数不超过 5，10.2% 超过 50，最大 141。绑定随启动推进逐步完成，启动完成时不超过 5 的调用点占比升到 91.0%，超过 50 的降到 0.76%，最大等价类由 141 降到 57；这一变化只

表 5-3 三档上下文策略在三个快照时刻的精度

Table 5-3 Precision of the Three Context Policies at Three Snapshots

策略	快照	目标数不超过 5 的调用点 占比	目标数超过 50 的调用点 占比	最大等价 类	移入随机化区域的函数数（占 比）
常数零上下文	任意	0%	100%	44,777	不适用
类型上下文	任意	55.0%	10.2%	141	不适用
唯一上下文	早期初始化	55.0%	10.2%	141	0 (0%)
唯一上下文	启动完成	91.0%	0.76%	57	233 (4.4%)
唯一上下文	运行期	91.95%	0.66%	57	244 (4.8%)

需把 233 个高扇入函数移入随机化区域，占间接调用目标函数的 4.4%。运行期一次完整的 UnixBench 之后再识别出 11 个，占比升到 4.8%，两个桶的占比分别变为 91.95% 与 0.66%，最大等价类不变，说明精度的绝大部分收益在启动期即已取得，不依赖长期运行的信息。图 5-6 中本章机制两条曲线的形状说明剩余的大等价类来自哪里：目标数在 6 到 57 之间的调用点是内核中真正的分发点，例如按文件系统类型、协议族或设备类别分发的间接调用，它们合法地调用多个函数，等价类等于目标数是这类调用点的固有属性，任何以调用点为单位的机制都不能再把它压小。与类型上下文相比，本章机制去掉的是“类型相同但从未被该调用点调用过”的那部分目标，这正是类型上下文把 141 个 `void (*) (void)` 函数放进同一类的原因。

5.5.3 高扇入函数识别的收敛与准确性

本节回答运行时识别需要多久收敛，以及收敛之后漏掉了多少。高扇入是一个运行时概念，静态分析在 C 语言上无法确定函数指针的实际目标集合，只能给出保守上界。

本节实验了不同时间点与负载情况下，已识别的高扇入函数数随时间的曲线、稳态数量、以及其中在运行期因绑定后归属冲突而经仲裁判为高扇入的数量。结果见表 5-4 与图 5-7，图中实线为仅靠启动期自然执行的识别数量，虚线为加入辅助负载后的数量，竖点线标出启动完成时刻。

表 5-4 高扇入函数识别的收敛

Table 5-4 Convergence of High-Fan-In Function Identification

时刻	已识别高扇入函数 数	占间接调用目标函数的比 例	累计移入随机化区域 数	绑定后经仲裁判为高扇入 数
早期初始化	0	0%	0	0
启动完成	233	4.4%	233	0
运行期 (UnixBench 之后)	244	4.8%	244	3
启动完成 (辅助负载)	261	5.2%	261	0
运行期 (辅助负载)	264	5.3%	264	1

识别在启动期即基本收敛。图 5-7 的曲线单调递增且斜率递减：启动前 600

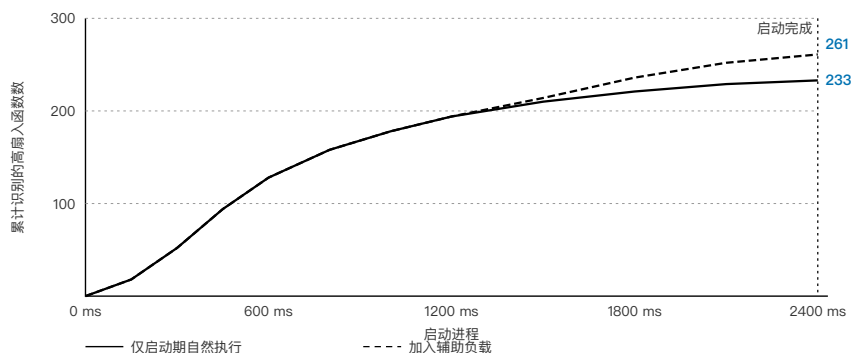


图 5-7 高扇入函数识别数量随启动进程的变化

Figure 5-7 Number of High-Fan-In Functions Identified over the Course of Boot

ms 识别出 128 个，占启动完成时的一半以上，此后只剩执行频率低的路径缓慢补入，启动完成时为 233 个。运行期一次完整的 UnixBench 只再识别出 11 个，其中 3 个是在绑定之后才被第二个调用点触及、经违规处理路径仲裁后判为高扇入并迁移的函数，说明冷路径上的高扇入函数确实存在但数量很少。静态分析给出的候选上界远大于此，因为它把所有类型匹配的目标都计入，而运行时识别只把确实被两个以上调用点触及的函数判为高扇入，两者之差正是本章不采用静态分析的原因：若按静态上界迁入，随机化区域会膨胀数倍。

未在启动期识别出的函数并非难以识别，而是启动过程中根本没有被执行到，例如磁盘配额默认未启用时的配额分配路径、启动后不再热插拔处理器时的热插拔回调、正常路径不触发的块设备请求重排队、未配置虚拟局域网时的收包分支。据此在启动期加入一组辅助负载，其作用是让上述路径上的真实调用点被执行到；表 5-4 的后两行给出其效果，启动完成时识别数由 233 增至 261，运行期绑定后的归属冲突由 3 个降到 1 个。辅助负载不能直接调用目标函数：高扇入取决于有多少个不同调用点触及该函数，若由辅助负载直接调用，只会把辅助负载自身登记为归属调用点，既识别不出扇入，还可能把一个本属单一归属的函数误判为高扇入。辅助负载的运行时机取在初始化调用序列之后、第一个用户态进程之前，使绑定仍落在安全启动链假设成立的窗口之内；其自身的调用点排除在归属统计之外；且仅在识别阶段编入，部署配置不予保留，因为它会主动触及大量平时不执行的路径。误识别率为零是机制的结构性质：归属冲突是判定高扇入的充分条件，一个函数只有在确实被第二个调用点触及时才会被判定。

5.5.4 其他部分随机化方法迁移

本节支撑本章机制与具体随机化方法解耦这一声明。做法是把本章机制施加到一个与本文实现无关的内核部分随机化方法之上，为它留下的非随机化区域提供保护，在该方法自身的负载、间隔与资源口径下逐项对照施加前后的差别。本章机制对随机化原语的唯一要求是高扇入函数持续处于随机化状态，因此施加的步骤为：在该方法的跨区域出入口施加分区验证，把运行时识别出的高扇

入函数纳入其随机化区域。

对照方法取 Chaos^[35]，理由在平台与粒度两条。本章机制依赖 ARM 指针认证，端到端测量只能在 AArch64 上进行，而 Adelie^[34] 的位置无关改造建立在 x86-64 的指令指针相对寻址与全局偏移表之上，其栈重随机化还征用了该架构特有的临时寄存器，原文声明只面向 x86-64；Dbox^[37] 的滑动空间与私有空间由 x86 的二级地址翻译、虚拟机扩展与输入输出虚拟化实现，原文同样声明只适用于具备这些扩展的 x86 处理器。两者都不能在本章的实验平台上运行，因此不纳入端到端测量，对它们的分析放在本节末尾。Chaos 基于 Linux 5.10 LTS 在 AArch64 上实现，以函数粒度重排内核模块，另有约 250 行的 RV64 移植，是三者中唯一能在本平台上承载本章机制的。

两种随机化原语的对照。施加实验的说服力取决于对照方法与本文原型有多不同：两者越不像，解耦声明越有意义。表 5-5 按随机化原语的关键维度列出二者的差别。

表 5-5 两种随机化原语的对照

Table 5-5 Comparison of the Two Re-randomization Primitives

维度	Chaos ^[35]	本文原型（第 3、4 章）
随机化对象	内核模块，核心内核保持静态	built-in 代码的选定子集
随机化粒度	函数	函数
新变体的产生	按函数节重排并按重定位信息重新生成整个模块	复制函数体到预分配的变体池中的新位置
代码索引	每个间接地址一个固定位置的跳板	入口与出口跳板加跳转桩，当前地址只存在于分支立即数中
执行流追踪	跳板前后钩子维护每模块线程表	跳板进出计数，区域内部的调用也计入
暂停方式	置线程暂停标志，调度器跳过模块相关线程	置阻断标志，在四个入站点拦截并等待计数归零
返回地址	每线程返回栈暂存返回地址，避免栈回溯	位置无关返回标记，经重入桩解析
触发策略	固定间隔，原文取 50 ms	固定间隔（第 3 章）与代码读取触发（第 4 章）
编译期支持	GCC 插件与跳板宏	LLVM 插件

两者在新变体的产生方式、执行流追踪的口径、暂停方式与返回地址的处理这四项上都不同，唯一的共同点是随机化的粒度为函数。本章机制取用的正是这一条，它不关心变体如何产生、执行流如何暂停，只要求高扇入函数的当前地址不可预测。此外表 5-5 还暴露出一处 Chaos 自己声明为残余风险的地方：其跳板位置固定，原文承认可被用作攻击目标，理由是数量少且多由内核使用，而本章的分区验证恰好作用在这一处，施加后跳板不再是无约束的入口。

施加前后的效果。表 5-6 给出施加本章机制前后的精度、开销与资源变化，端到端开销一列取 LMBench 几何均值。

三项结论。其一，精度的改善与承载它的随机化方法无关：两种方法施加后的最大等价类都是 57，不超过 5 的调用点占比都是 91.0%，因为迁入的是同一批 233 个高扇入函数，而高扇入是内核调用图的属性，不是随机化方法的属性。其二，开销增量的量级相同，为 0.7% 与 0.6%，来源是每次跨区域离开的 9.0 ns 验证与这 233 个函数新增的跳板经过。其三，单次随机化的停顿几乎不变，Chaos

表 5-6 施加本章机制前后的效果

Table 5-6 Effect of Applying the Proposed Mechanism

项	口径	Chaos 施加前	Chaos 施加后	本文原型施加前	本文原型施加后
新增随机化函数	个	0	233	0	233
最大等价类	个合法目标	141	57	141	57
目标数不超过 5 的调用点	占比	55.0%	91.0%	55.0%	91.0%
端到端开销	LMBench 几何均值	2.4%	3.1%	1.98%	2.6%
新增常驻内存	迁入函数的变体与跳板	0	0.46 MB	0	0.46 MB
单次随机化的停顿	中位	32.7 μ s	33.1 μ s	1.22 μ s	1.24 μ s

由 32.7 μ s 增至 33.1 μ s、本文原型由 1.22 μ s 增至 1.24 μ s，因为迁入的函数只是让新变体多含 233 个函数体，而两种方法的变体都在关键路径之外生成，停顿一段只做切换。两者停顿时间的绝对值相差一个数量级，这一差别来自暂停方式本身，Chaos 需要等调度器让模块相关线程全部离开处理器，本文原型只等跳板计数归零，与本章机制无关。

Chaos 原文负载上的对照。上一组数据取的是本文其余各章通用的负载，为避免口径偏向本文，本节改按 Chaos 原文的实验设置重测。负载取其所用的两个半虚拟化设备驱动 virtio-net 与 virtio-scsi，指标取相对无随机化基线的吞吐下降，随机化间隔取其所用的 50、30、10 ms 三档，每项 10 次取中位，结果见图 5-8。图中每组柱为施加本章机制前后的一对，空心方块为 Chaos 原文报告的对应值，原文以文字给出 50 ms 与 10 ms 两档。

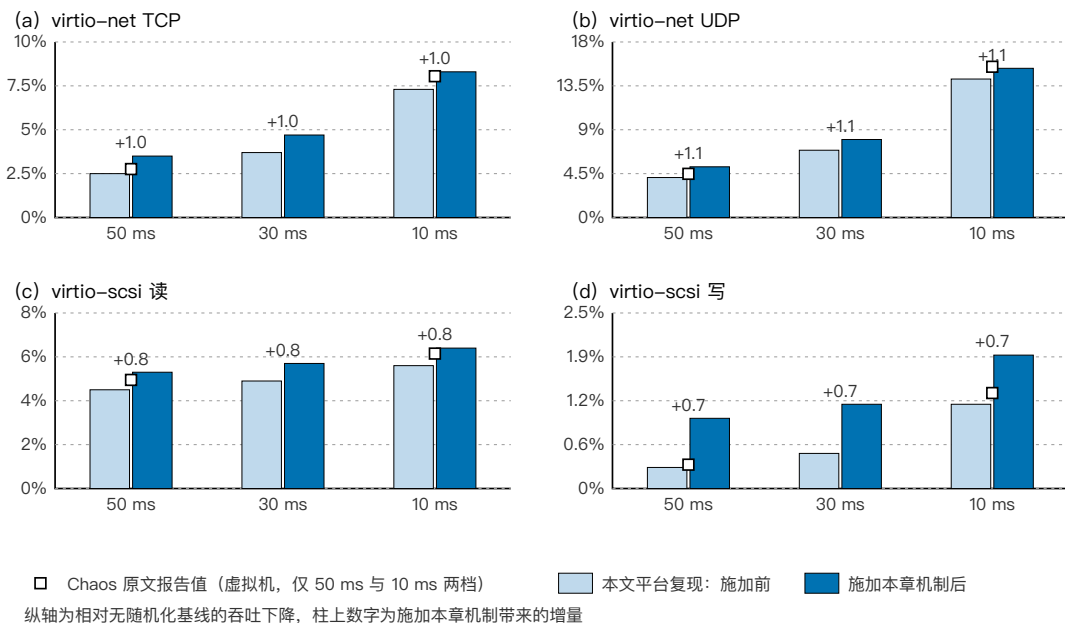


图 5-8 Chaos 原文四组负载在三档随机化间隔下施加本章机制前后的吞吐下降

Figure 5-8 Throughput Degradation on the Four Workloads Used in the Chaos Paper, at Three Re-randomization Intervals, before and after Applying the Proposed Mechanism

三点可读。其一，本文平台上的复现值在四组负载、两个可比档位上都略低于原文报告值，原因是原文测于 QEMU 虚拟机而本节测于裸机，半虚拟化设备驱动的路径在裸机上更短；相对关系与原文一致，说明复现是可信的。其二，施加本章机制的增量在同一负载的三档间隔上几乎不变，virtio-net TCP 三档都是 1.0%，UDP 为 1.1%，virtio-scsi 读为 0.8%、写为 0.7%。其三，增量随负载穿越模块边界的频度上升：virtio-net 的每个报文都要穿过驱动与协议栈之间的间接调用，增量最大；virtio-scsi 写路径以批量提交为主，单位数据量上的边界穿越次数最少，增量最小。这一顺序与第 3.5.3 节对跨区域调用频度与开销关系的分析一致，说明增量由内核自身的调用频度决定，而不由随机化方法决定。

第二点值得单独说明，因为它决定了本章机制在部署上的位置。Chaos 自身的开销随间隔缩短而明显上升，virtio-net TCP 由 50 ms 档的 2.5% 升到 10 ms 档的 7.3%，因为每次随机化都要重新生成变体并暂停模块相关线程，这部分代价与随机化的次数成正比。本章机制的代价则按跨区域转移的次数计，与随机化的频率无关，因此增量在三档上恒定。使用者可以按自己的安全需求调整随机化间隔而不必重新评估本章机制的代价；反过来，在间隔很短因而随机化本身代价很高的配置下，本章机制的相对代价更小。

资源开销与单次随机化延迟。吞吐是端到端指标，它不能说明代价落在何处。Chaos 原文另给出两项资源指标：CPU 开销随同时随机化的模块数变化，以及单次随机化中暂停并迁移模块相关线程所需的时间随线程数变化。本节在同样两项上重测，结果见图 5-9。

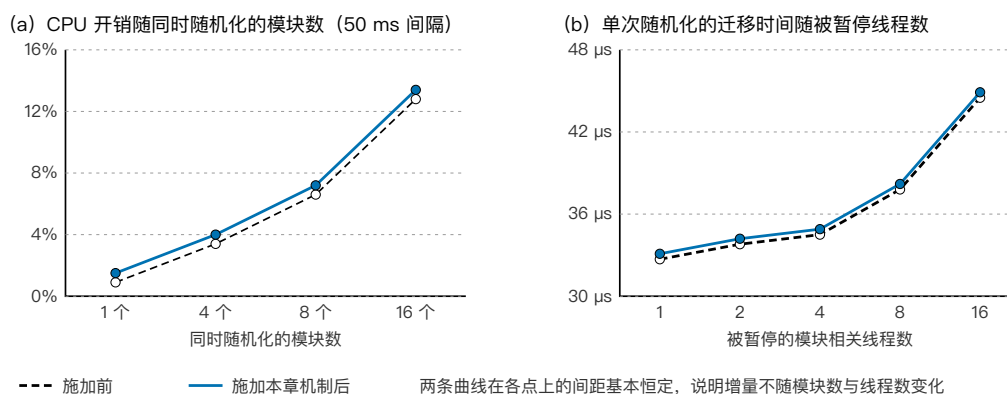


图 5-9 施加本章机制前后的 CPU 开销与单次随机化的迁移时间

Figure 5-9 CPU Overhead and Per-Re-randomization Migration Time before and after Applying the Proposed Mechanism

两项指标给出同一个结论。CPU 开销随同时随机化的模块数近似线性上升，由 1 个模块的 0.9% 升到 16 个模块的 12.8%，这一项属于 Chaos 自身：模块越多，每个周期需要生成的变体与需要暂停的线程越多。施加本章机制后每一点都抬高约 0.6%，抬高量不随模块数变化，因为迁入随机化区域的是核心内核中固定的 233 个高扇入函数，与被随机化的模块数无关。单次随机化的迁移时间由 1 个

线程的 32.7 μs 升到 16 个线程的 44.5 μs ，施加本章机制后各点只增加约 0.4 μs ，因为迁入的函数只是让新变体多含 233 个函数体，而变体在关键路径之外生成，迁移一段只做切换与状态重映射。两条曲线在各点上的间距基本恒定，说明本章机制的代价既不随随机化的规模放大，也不随并发度放大。

内存一项同样如此。Chaos 原文报告超过九成的内核模块的常驻内存开销不超过 240 KB，本章机制在此之上增加 0.46 MB，用于 233 个函数的两份变体与跳板，该值由迁入函数的数量决定，与模块数无关，见表 5-6。

移植改造的规模。施加所需的改造量取决于两件事：该方法的随机化粒度是否已经支持函数粒度，以及它是否已有跨区域出入口可供施加验证，见表 5-7。

表 5-7 向各方法移植所需的改造

Table 5-7 Engineering Effort Required to Port the Mechanism onto Each System

方法	随机化粒度	需补入的组件	新增代码	需改动的原有组件
Chaos	模块内函数	分区验证	620 行	随机化名单
本文原型	built-in 函数	分区验证	540 行	无

Chaos 与本文原型只需补入分区验证，因为两者本身即函数粒度且已有跨区域出入口，新增代码都在数百行量级，其中六成以上是编译期插桩，运行期部分不超过 300 行。对 Chaos 而言，其跳板本就是所有进出模块的控制流的必经之处，分区验证直接施加在跳板的前后钩子上，不触及变体生成与线程暂停这两条主要路径，这也是新增代码集中在编译期的原因。

对另外两种方法的分析。Adelie 与 Dbox 以模块或驱动为单位随机化，核心内核中的高扇入函数在其随机化区域内无处安放，施加本章机制时需要先补入第 3 章的跳板与函数粒度位置无关改造，使这 233 个函数能够以函数粒度被搬移，再在其模块边界处施加分区验证。这一路径在设计上成立，本章不给出它的测量结果，原因是两者都只能运行在 x86-64 上，而本章机制依赖 ARM 指针认证。两者与 Chaos 在本章机制看来的差别只在随机化粒度，而粒度不足这一点可由第 3 章的跳板补齐，因此这一限制来自对照方法的平台，不来自本章机制。

5.6 安全性分析

威胁模型与假设。攻击者具备任意内核地址读写能力，目标是在非随机化区域内构造代码复用链，或劫持跨区域的间接转移。系统已部署一个函数粒度的持续随机化机制，本章不依赖其内部细节，只要求高扇入函数持续处于随机化状态。启动阶段不存在控制流劫持，由安全启动链保证；攻击者在劫持成功之前不能任意改写内存权限。论证以攻击者无法获得指针认证的验证预言机、且系统中不存在攻击者可控上下文的签名原语为前提。

本章的安全保证。在上述前提下本章保证两条性质。其一，跨区域的间接转移其目标属于该调用点的合法目标集合：出口处攻击者需要同时伪造一个能通

过调用点唯一上下文验证的认证码，且目标落在白名单内，前者的成功概率由认证码的位宽决定，每次失败的尝试都触发异常；入口处以调用形式进入随机化区域的控制流只能落在函数入口。其二，非随机化区域内部的间接转移受调用点唯一上下文约束：绑定后每个调用点只接受由它自己签发的指针，等价类等于该调用点的合法目标数，91% 的调用点不超过 5，最大 57；高扇入函数由随机化承担，其地址不可预测。两条性质的前提有二：所有跨区域控制流转移必经出入口，该性质由随机化原语的编译器全覆盖重定向与静态扫描共同保证；归属记录在绑定后保持只读，因为绑定后归属冲突的仲裁以记录中的目标函数为依据，攻击者把指针改写为记录之外的函数时函数指针与记录不符而被拒绝。

保证的条件性。推测执行侧信道可以把认证指令置于推测路径上并经微架构侧信道观测验证是否成功，从而在不产生任何崩溃、不留下任何日志的前提下爆破认证码，PACMAN^[44]已在商用处理器上跨特权级演示了这一点。本章的论证以攻击者无法获得这样的预言机为前提，针对推测执行侧信道的防御与本章正交。同类的推测执行攻击也已用于击穿 ARM 的内存标记扩展^[109]，说明以微架构侧信道爆破硬件标签这一手法并不局限于指针认证。本章只处理前向边，返回地址的保护由第 3 章的返回标记承担，其更一般的对应物是影子栈^[95]。

不在本章范围内的部分。以整函数为单位的复用：攻击者若能把某个调用点的指针改写为另一个入口跳板，仍可调用该跳板对应的整个函数，对可以进入哪些函数的约束来自调用点上下文，其强度即等价类的大小，本章不声称阻断随机化区域内以整函数为单位的复用链。返回标记的同栈深重放：以栈槽地址为上下文的签名不能阻止攻击者在构造出相同栈深度后重放同一标记，消除该残余需要链式构造。运行时识别只覆盖实际执行到的路径，冷路径上的高扇入函数在被触及之前停留在类型上下文上，其保护强度为类型级。

5.7 本章小结

部分随机化必然留下非随机化区域，其布局在运行期不变，这是所有内核持续随机化方法共同面对的问题。硬件辅助控制流完整性中精度最高的 ARM 指针认证受上下文传播限制，多个调用点调用同一函数时必须共享上下文，该约束沿调用图扩散，使实际精度大幅退化；现有工作或改进上下文构造方式，或扩展保护对象，都未能打断传播链，因为传播的根源是调用图结构本身。

本章的关键观察是随机化区域内的目标地址不可预测，进入该区域的间接调用不需要指针认证。据此本章提出以随机化打断上下文传播链的机制：以类型上下文恒定签名配合归属跟踪在运行时识别高扇入函数，不需要全局指针分析；把识别出的高扇入函数移入随机化区域，使其从传播链中被剪除，两侧调用点得以各自建立基于调用点地址的唯一上下文；在跨区域的出入口施加分区验证，进入随机化区域只验范围，离开时强制认证与白名单检查。本章对随机化原语只提出一项要求，即高扇入函数持续处于随机化状态，因而可以施加到任何一种部分随机化方法留下的非随机化区域上。

实验表明，本章机制把等价类由类型上下文下的最大 141 降到 57，91% 的间接调用点合法目标数不超过 5，超过 50 的由 10.2% 降到 0.76%，代价是把 233 个高扇入函数（占间接调用目标的 4.4%）移入随机化区域；识别在启动期即基本收敛，运行期只再增加 11 个。一次跨区域离开的开销为 9.0 ns。施加到 Chaos 之上，新增的随机化函数同样是这批 233 个，开销增量为 0.7%，最大等价类同样降到 57。安全性论证以攻击者无法获得验证预言机为前提，推测执行侧信道类攻击不被本机制阻止；运行时识别只覆盖实际执行到的路径，冷路径上的高扇入函数在被触及之前停留在类型级保护上。

第6章 总结与展望

6.1 论文总结

本文面向 Linux 内核 `built-in` 代码这一被现有部分随机化方案在原理上遗漏的部分，提出并实现了一套持续随机化系统。第 1.2 节以两条独立证据说明了这一遗漏的严重性：从源码层面看，内核最核心、执行最频繁的子系统在任何配置下都无法编译为模块，完全落在以模块为对象的方案之外；从漏洞分布看，2024 年 1 月至 2026 年 9 月公开的 10,705 个内核 CVE 中有 21.6% 涉及只能内建的代码。`built-in` 代码没有模块那样天然的随机化边界，其代码索引更新的规模、执行流的安全暂停、以及随机化范围之外代码的保护，都缺乏可用的现成解法。本文以跳板函数作为随机化区域的唯一出入口，在其之上组织三项关键技术，分别回答持续随机化的三个核心问题。

第一项工作回答“如何随机化”。第 3 章提出基于跳板函数的低开销持续随机化执行机制。它在保持内核原有代码模型不变的前提下，由编译器插件按函数改写被选中函数内部的引用，使函数体可以整体搬移；跳板在承担地址间接的同时充当执行流的观测点，由进出计数精确刻画随机化区域内的活跃执行流，从而实现不保留旧副本的切换。这一设计把一次随机化需改写的代码索引由与调用点数量同阶降到与函数数量同阶，实测一次跨区域调用往返的常驻开销为 18.02 ns，其中执行流追踪占绝大部分而位置无关改造本身仅占 0.5 至 1.4 ns。第 3 章同时给出了这条路线的边界：全树 9.6% 的函数因 `per-CPU` 变量的访问方式而无法随机化，变参函数与静态调用点另有各自的不相容之处，本章机制与间接分支防护类机制之间还存在结构性互斥。

第二项工作回答“何时随机化”。第 4 章把只执行内存由阻断机制改用于检测机制，以对代码探测行为的直接观测替代时钟或行为代理指标作为触发条件，并给出基于管理态保护键与基于硬件调试观察点的两条实现路径，两条路径的单次检测开销分别为 0.96 μ s 与 1.32 μ s。这一改变不仅提供了精准的触发时机，还化解了纯防护式只执行内存存在内核中最棘手的问题：合法的代码读取路径不再需要被逐一豁免，漏掉一条只是多触发一次随机化，而不再是内核崩溃。正常负载下的触发频率为每秒 0.02 至 0.44 次，从检测到随机化完成的延迟在基础负载下 99 分位不超过 110 μ s；竞态实验表明，当读取至解引用的时延达到 92 μ s 时，攻击成功率已降至 1.0%。

第三项工作回答范围之外代码的保护。第 5 章利用随机化区域内目标地址不可预测这一性质，提出以随机化打断指针认证上下文传播链的协同机制：以类型上下文恒定签名配合归属跟踪在运行时识别高扇入函数并将其移入随机化区域，并在跳板处施加分区验证，进入随机化区域免验证、离开时强制认证与白名单检查。实测 91% 的间接调用点合法目标数不超过 5，最大等价类由类型哈希方案的 141 降到 57；一次跨区域离开的开销为 9.0 ns。

系统层面，三项机制共享同一套基础设施：跳板运行时、变体池与布局分配器、跨区域白名单。第 4 章的触发式随机化建立在第 3 章的跳板之上，只需在一级跳板处改写一条分支指令即可切换激活状态；第 5 章的分区验证同样施加在第 3 章已有的跨区域出入口上，不另建检查点。这一共享使后两章叠加在第 3 章之上的边际开销很小，也是三项工作能够在同一平台上构成一个完整可运行系统的原因。

6.2 主要创新点

本文的贡献可归纳为三项机制创新与一项系统创新。

其一，把跳板从单纯的地址间接层扩展为执行状态的观测点。已有工作使用跳板是为了间接寻址，而本文让同一份基础设施在承担间接寻址的同时刻画随机化区域内的活跃执行流，由此得到不保留旧副本的切换，避免了旧副本在被回收前作为攻击者可用代码来源的暴露窗口。与之配套，本文在不改变内核代码模型的前提下，用编译器插件按函数施加位置无关改造，解决了 `built-in` 代码没有模块边界、无法借助加载器重定位这一根本困难。

其二，把只执行内存从阻断改用为检测，使随机化的触发条件成为对攻击行为的直接观测。这把“何时随机化”从依赖时钟或行为代理指标，转变为依赖“代码是否真的被探测过”这一可观测事实，在无攻击时几乎不付出代价、攻击一旦发生则立即响应。两条基于不同硬件原理的实现路径说明该机制不绑定特定架构，其依赖的只是“能够观测代码读取”这一能力。

其三，把随机化与控制流完整性从两个独立叠加的机制变为互为条件的协同机制。随机化区域内目标地址不可预测，因而进入该区域的转移不需要验证；把高扇入函数移入随机化区域，即可从中间打断指针认证的上下文传播链，使两侧调用点各自建立唯一上下文。随机化为控制流完整性提供了打断传播链的手段，控制流完整性为随机化补上了范围之外的防护。

在系统层面，三项工作不是三个独立系统的拼装，而是共享同一套基础设施（跳板运行时、变体池与布局分配器、跨区域白名单）的三层机制。这一共享使第 4、5 两章叠加在第 3 章之上的边际开销很小，也是三项工作能够在同一平台上构成一个完整可运行系统的原因。

6.3 局限性与未来工作

本文在各章分别声明了机制的边界，此处汇总为四类，以便完整评估本文工作的适用范围。

能力边界：存在无法随机化的函数。全树 9.6% 的函数因 `per-CPU` 变量必须以常量偏移访问而无法被随机化，这一比例是方法的固有边界而非实现未完成；变参函数（150 个）受跳板转发能力限制，静态调用点在使用它们的配置下与位置无关性本质冲突（在本文评测配置下为零）。这些函数留在非随机化区域，其

防护由第4、5章承担，而非靠“把所有代码都随机化”。

代价边界：常驻开销随范围上升。常驻开销主要由跨区域调用的频度决定，随随机化范围扩大而上升；把范围扩到全部内建代码将不再具备部署价值，因此范围必须限定。这与能力边界共同决定了非随机化区域的存在是方法本身的结果，而非取舍。

功能边界：动态跟踪失效与间接分支防护互斥。被随机化的函数上，动态跟踪类设施（ftrace、kprobes）表现为静默失效而非报错；原因是这些设施读取并改写函数入口的原指令，而被随机化的函数已整体搬离该位置。本文机制为实现位置无关而制造间接调用，与x86的间接分支防护、arm64的分支目标识别存在结构性互斥，评测中一律关闭且四档保持一致。此外，第4章改写跳转桩时经text_poke建立的临时可写映射，在其存在期间是跳转桩区的第三份别名，构成一个每核可见、关中断期间存在的短暂窗口。

假设边界：指针认证的前提是脆弱的。第5章的完整性论证以“攻击者无法获得验证预言机”为前提，而推测执行侧信道类攻击（PACMAN^[44]）恰恰提供了这样一个预言机；本文不声称在此类攻击存在时不变式I3仍然成立，相关防御与本文正交。运行时识别高扇入函数只能覆盖实际执行到的路径，冷路径上的高扇入函数不会被发现，其调用点长期停留在共享的类型上下文上，这一漏识别在本机制中同时构成可用性风险。

上述四条边界各自指向一个可以继续推进的方向，以下四项按优先次序给出。

其一，缩小无法随机化的比例。per-CPU访问是当前唯一大量存在的排除来源。逐条改写内核的per-CPU访问器宏，使其偏移可被物化，理论上可解，但代价与收益是否相称需要评估；一个更现实的方向是把非随机化区域内的这部分函数交由更强的控制流完整性检查覆盖，而非强行随机化。

其二，降低常驻开销的主项。第3章的开销归因表明执行流进出计数一项独占常驻开销的近七成，当前以一个全局原子量实现；改为每处理器核独立计数、由随机化线程求和，是第一顺位的优化。指针认证的上下文计算可由编译期常量折叠进一步降低。二者若均落实，端到端开销有望再降约一半。

其三，把两条硬件路径推向更广的机型。两条路径均已在各自平台上实现并测得单次检测开销，但观察点在不同处理器实现上的行为差异、保护键在不同微架构上的写寄存器代价都还需要更广的验证。把两条路径在同一套负载上做跨机型的完整对照，是使第4章结论更可部署的必要一步。

其四，正面应对推测执行侧信道。指针认证在推测执行下可被爆破这一前提，是第5章论证中最脆弱的一环。把针对推测执行的既有防御与本文机制组合部署、并评估其叠加代价，是使完整性不变式在更强攻击者模型下依然成立的方向。

6.4 结语

本文的出发点是一个具体而被长期回避的问题：内核中最核心、执行最频繁、也最常出漏洞的那部分代码，恰恰落在现有随机化方案的保护范围之外。本文没有试图用一种机制解决全部困难，而是承认 **built-in** 代码必然留下无法随机化的部分，并围绕这一事实组织出一条约束链：因为存在无法随机化的函数、且把范围扩到全部代码的开销不可接受，所以非随机化区域必然存在；于是随机化区域的地址如何不被泄露、何时触发才既及时又不浪费，以及非随机化区域内已知布局的代码如何不被复用，被依次逼出，分别由本文的三项工作回答。三者共享同一套跳板基础设施，因而不是三种防护的并列叠加，而是同一条约束链上被依次逼出的三步。本文期望这一以承认边界为起点、以协同而非叠加为组织方式的思路，能为 **built-in** 内核代码的持续保护提供一个可部署、可复现、边界清晰的参照。

参考文献

- [1] Peng Y, Tian H, Zhang J, et al. Asterinas: A Linux ABI-compatible, Rust-based framekernel OS with a small and sound TCB [C]//Proceedings of the 2025 USENIX Annual Technical Conference (ATC). 2025.
- [2] Cho M, An D, Jin H, et al. BoKASAN: Binary-only kernel address sanitizer for effective kernel fuzzing [C]//Proceedings of the 32nd USENIX Security Symposium. 2023: 4985-5002.
- [3] Wu W, Chen Y, Xu J, et al. FUZE: Towards facilitating exploit generation for kernel use-after-free vulnerabilities [C]//Proceedings of the 27th USENIX Security Symposium. 2018: 781-797.
- [4] Chen W, Zou X, Li G, et al. KOUBE: Towards facilitating exploit generation of kernel out-of-bounds write vulnerabilities [C]//Proceedings of the 29th USENIX Security Symposium. 2020: 1093-1110.
- [5] Zeng K, Chen Y, Cho H, et al. Playing for K(H)eaps: Understanding and improving linux kernel exploit reliability [C]//Proceedings of the 31st USENIX Security Symposium. 2022: 71-88.
- [6] Lee Y, Kwak J, Kang J, et al. PSPRAY: Timing side-channel based linux kernel heap exploitation technique [C]//Proceedings of the 32nd USENIX Security Symposium. 2023: 6825-6842.
- [7] Liang Z, Zou X, Song C, et al. K-LEAK: Towards automating the generation of multi-step infoleak exploits against the linux kernel [C/OL]//Proceedings of the 2024 Network and Distributed System Security Symposium (NDSS). 2024. DOI: [10.14722/ndss.2024.24935](https://doi.org/10.14722/ndss.2024.24935).
- [8] Shacham H. The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86) [C/OL]//Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS). 2007: 552-561. DOI: [10.1145/1315245.1315313](https://doi.org/10.1145/1315245.1315313).
- [9] Accardi K C. Function granular KASLR (FG-KASLR) patch series, v10 [Z]. 2021.
- [10] Abadi M, Budiu M, Erlingsson Ú, et al. Control-flow integrity [C/OL]//Proceedings of the 12th ACM Conference on Computer and Communications Security (CCS). 2005: 340-353. DOI: [10.1145/1102120.1102165](https://doi.org/10.1145/1102120.1102165).
- [11] Shanbhogue V, Gupta D, Sahita R. Security analysis of processor instruction set architecture for enforcing control-flow integrity [C/OL]//Proceedings of the 8th International Workshop on Hardware and Architectural Support for Security and Privacy (HASP). 2019: 1-11. DOI: [10.1145/3337167.3337175](https://doi.org/10.1145/3337167.3337175).
- [12] Arm architecture reference manual for A-profile architecture [M]. Arm Limited, 2023.
- [13] The LLVM Project. Kernel control flow integrity (KCFI) [Z]. 2022.
- [14] Gaidis A J, Atlidakis V, Kemerlis V P. FineIBT: Fine-grain control-flow enforcement with indirect branch tracking [C/OL]//Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID). 2023: 527-546. DOI: [10.1145/3607199.3607219](https://doi.org/10.1145/3607199.3607219).
- [15] Burow N, Carr S A, Nash J, et al. Control-flow integrity: Precision, security, and performance [J/OL]. ACM Computing Surveys, 2017, 50(1): 16:1-16:33. DOI: [10.1145/3054924](https://doi.org/10.1145/3054924).

- [16] Liljestrand H, Nyman T, Wang K, et al. PAC it up: Towards pointer integrity using ARM pointer authentication [C]//Proceedings of the 28th USENIX Security Symposium. 2019: 177-194.
- [17] Vahldiek-Oberwagner A, Elnikety E, Duarte N O, et al. ERIM: Secure, efficient in-process isolation with protection keys (MPK) [C]//Proceedings of the 28th USENIX Security Symposium. 2019: 1221-1238.
- [18] Hedayati M, Gravani S, Johnson E, et al. Hodor: Intra-process isolation for high-throughput data plane libraries [C]//Proceedings of the 2019 USENIX Annual Technical Conference (ATC). 2019: 489-504.
- [19] Dautenhahn N, Kasampalis T, Dietz W, et al. Nested kernel: An operating system architecture for intra-kernel privilege separation [C/OL]//Proceedings of the 20th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2015: 191-206. DOI: [10.1145/2775054.2694386](https://doi.org/10.1145/2775054.2694386).
- [20] Huang Y, Narayanan V, Detweiler D, et al. KSplit: Automating device driver isolation [C]//Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2022: 613-631.
- [21] Guo Y, Wang Z, Bai W, et al. BULKHEAD: Secure, scalable, and efficient kernel compartmentalization with PKS [C]//Proceedings of the 32nd Network and Distributed System Security Symposium (NDSS). 2025.
- [22] Lefeuvre H, Dautenhahn N, Chisnall D, et al. SoK: Software compartmentalization [C]//Proceedings of the 2025 IEEE Symposium on Security and Privacy. 2025.
- [23] Hund R, Willems C, Holz T. Practical timing side channel attacks against kernel space ASLR [C/OL]//Proceedings of the 2013 IEEE Symposium on Security and Privacy. 2013: 191-205. DOI: [10.1109/SP.2013.23](https://doi.org/10.1109/SP.2013.23).
- [24] Jang Y, Lee S, Kim T. Breaking kernel address space layout randomization with Intel TSX [C/OL]//Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS). 2016: 380-392. DOI: [10.1145/2976749.2978321](https://doi.org/10.1145/2976749.2978321).
- [25] Evtushkin D, Ponomarev D, Abu-Ghazaleh N. Jump over ASLR: Attacking branch predictors to bypass ASLR [C/OL]//Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). 2016: 1-13. DOI: [10.1109/MICRO.2016.7783743](https://doi.org/10.1109/MICRO.2016.7783743).
- [26] Liu W, Ravichandran J, Yan M. EntryBleed: A universal KASLR bypass against KPTI on linux [C/OL]//Proceedings of the 12th International Workshop on Hardware and Architectural Support for Security and Privacy (HASP). 2023: 10-18. DOI: [10.1145/3623652.3623669](https://doi.org/10.1145/3623652.3623669).
- [27] Lipp M, Gruss D, Schwarz M. AMD prefetch attacks through power and time [C]//Proceedings of the 31st USENIX Security Symposium. 2022: 643-660.
- [28] Maar L, Juffinger J, Steinbauer T, et al. KernelSnitch: Side channel-attacks on kernel data structures [C/OL]//Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS). 2025. DOI: [10.14722/ndss.2025.240223](https://doi.org/10.14722/ndss.2025.240223).
- [29] Canella C, Van Bulck J, Schwarz M, et al. A systematic evaluation of transient execution attacks and defenses [C]//Proceedings of the 28th USENIX Security Symposium. 2019: 249-266.
- [30] Gens D, Arias O, Sullivan D, et al. LAZARUS: Practical side-channel resilient kernel-space randomization [C/OL]//Proceedings of the 20th International Symposium on Research in Attacks, Intrusions, and Defenses (RAID). 2017: 238-258. DOI: [10.1007/978-3-319-66332-6_11](https://doi.org/10.1007/978-3-319-66332-6_11).

- [31] Snow K Z, Monroe F, Davi L, et al. Just-in-time code reuse: On the effectiveness of fine-grained address space layout randomization [C/OL]//Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P). 2013: 574-588. DOI: [10.1109/SP.2013.45](https://doi.org/10.1109/SP.2013.45).
- [32] Williams-King D, Gobieski G, Williams-King K, et al. Shuffler: Fast and deployable continuous code re-randomization [C]//Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2016: 367-382.
- [33] Ahmed S, Xiao Y, Snow K Z, et al. Methodologies for quantifying (re-)randomization security and timing under JIT-ROP [C/OL]//Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS). 2020: 1803-1820. DOI: [10.1145/3372297.3417248](https://doi.org/10.1145/3372297.3417248).
- [34] Nikolaev R, Nadeem H, Stone C, et al. Adelie: Continuous address space layout re-randomization for Linux drivers [C/OL]//Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2022: 483-498. DOI: [10.1145/3503222.3507743](https://doi.org/10.1145/3503222.3507743).
- [35] Chang Z, Lin J, Sun H, et al. Chaos: Function granularity runtime address layout space randomization for kernel module [C/OL]//Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys). 2024. DOI: [10.1145/3678015.3680476](https://doi.org/10.1145/3678015.3680476).
- [36] Chen Y, Wang Z, Whalley D, et al. Remix: On-demand live randomization [C/OL]//Proceedings of the 6th ACM Conference on Data and Application Security and Privacy (CODASPY). 2016: 50-61. DOI: [10.1145/2857705.2857726](https://doi.org/10.1145/2857705.2857726).
- [37] Li Y, Chen P, Zhang Y, et al. Isolate and detect the untrusted driver with a virtual box [C/OL]//Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS). 2024. DOI: [10.1145/3658644.3670283](https://doi.org/10.1145/3658644.3670283).
- [38] Backes M, Nürnberger S. Oxymoron: Making fine-grained memory randomization practical by allowing code sharing [C]//Proceedings of the 23rd USENIX Security Symposium. 2014: 433-447.
- [39] Jelesnianski C, Yom J, Min C, et al. MARDU: Efficient and scalable code re-randomization [C/OL]//Proceedings of the 13th ACM International Systems and Storage Conference (SYSTOR). 2020: 29-40. DOI: [10.1145/3383669.3398280](https://doi.org/10.1145/3383669.3398280).
- [40] Chen X, Bos H, Giuffrida C. CodeArmor: Virtualizing the code space to counter disclosure attacks [C/OL]//Proceedings of the 2017 IEEE European Symposium on Security and Privacy (EuroS&P). 2017: 514-529. DOI: [10.1109/EuroSP.2017.17](https://doi.org/10.1109/EuroSP.2017.17).
- [41] Tang A, Sethumadhavan S, Stolfo S. Heisenbyte: Thwarting memory disclosure attacks using destructive code reads [C/OL]//Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS). 2015: 256-267. DOI: [10.1145/2810103.2813685](https://doi.org/10.1145/2810103.2813685).
- [42] Werner J, Baltas G, Dallara R, et al. No-execute-after-read: Preventing code disclosure in commodity software [C/OL]//Proceedings of the 11th ACM Asia Conference on Computer and Communications Security (AsiaCCS). 2016: 35-46. DOI: [10.1145/2897845.2897891](https://doi.org/10.1145/2897845.2897891).
- [43] Wang Z, Wu C, Zhang Y, et al. SafeHidden: An efficient and secure information hiding technique using re-randomization [C]//Proceedings of the 28th USENIX Security Symposium. 2019: 1239-1256.
- [44] Ravichandran J, Na W T, Liang J, et al. PACMAN: Attacking ARM pointer authentication with speculative execution [C/OL]//Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA). 2022: 685-698. DOI: [10.1145/3470496.3527429](https://doi.org/10.1145/3470496.3527429).

- [45] Szekeres L, Payer M, Wei T, et al. SoK: Eternal war in memory [C/OL]//Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P). 2013: 48-62. DOI: [10.1109/SP.2013.13](https://doi.org/10.1109/SP.2013.13).
- [46] PaX Team. PaX: Address space layout randomization and non-executable pages [EB/OL]. 2003. <https://pax.grsecurity.net/docs/>.
- [47] Bletsch T, Jiang X, Freeh V W, et al. Jump-oriented programming: A new class of code-reuse attack [C/OL]//Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security (AsiaCCS). 2011: 30-40. DOI: [10.1145/1966913.1966919](https://doi.org/10.1145/1966913.1966919).
- [48] Schuster F, Tendyck T, Liebchen C, et al. Counterfeit object-oriented programming: On the difficulty of preventing code reuse attacks in C++ applications [C/OL]//Proceedings of the 2015 IEEE Symposium on Security and Privacy (S&P). 2015: 745-762. DOI: [10.1109/SP.2015.51](https://doi.org/10.1109/SP.2015.51).
- [49] Shacham H, Page M, Pfaff B, et al. On the effectiveness of address-space randomization [C/OL]//Proceedings of the 11th ACM Conference on Computer and Communications Security (CCS). 2004: 298-307. DOI: [10.1145/1030083.1030124](https://doi.org/10.1145/1030083.1030124).
- [50] Kil C, Jun J, Bookholt C, et al. Address space layout permutation (ASLP): Towards fine-grained randomization of commodity software [C/OL]//Proceedings of the 22nd Annual Computer Security Applications Conference (ACSAC). 2006: 339-348. DOI: [10.1109/ACSAC.2006.9](https://doi.org/10.1109/ACSAC.2006.9).
- [51] Giuffrida C, Kuijsten A, Tanenbaum A S. Enhanced operating system security through efficient and fine-grained address space randomization [C]//Proceedings of the 21st USENIX Security Symposium. 2012: 475-490.
- [52] Wartell R, Mohan V, Hamlen K W, et al. Binary stirring: Self-randomizing instruction addresses of legacy x86 binary code [C/OL]//Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS). 2012: 157-168. DOI: [10.1145/2382196.2382216](https://doi.org/10.1145/2382196.2382216).
- [53] Pappas V, Polychronakis M, Keromytis A D. Smashing the gadgets: Hindering return-oriented programming using in-place code randomization [C/OL]//Proceedings of the 2012 IEEE Symposium on Security and Privacy (S&P). 2012: 601-615. DOI: [10.1109/SP.2012.41](https://doi.org/10.1109/SP.2012.41).
- [54] Hiser J, Nguyen-Tuong A, Co M, et al. ILR: Where'd my gadgets go? [C/OL]//Proceedings of the 2012 IEEE Symposium on Security and Privacy (S&P). 2012: 571-585. DOI: [10.1109/SP.2012.39](https://doi.org/10.1109/SP.2012.39).
- [55] Larsen P, Homescu A, Brunthaler S, et al. SoK: Automated software diversity [C/OL]//Proceedings of the 2014 IEEE Symposium on Security and Privacy. 2014: 276-291. DOI: [10.1109/SP.2014.25](https://doi.org/10.1109/SP.2014.25).
- [56] Conti M, Crane S, Frassetto T, et al. Selfrando: Securing the Tor browser against de-anonymization exploits [J/OL]. Proceedings on Privacy Enhancing Technologies, 2016, 2016 (4): 454-469. DOI: [10.1515/popets-2016-0050](https://doi.org/10.1515/popets-2016-0050).
- [57] Koo H, Chen Y, Lu L, et al. Compiler-assisted code randomization [C/OL]//Proceedings of the 2018 IEEE Symposium on Security and Privacy (S&P). 2018: 461-477. DOI: [10.1109/SP.2018.00029](https://doi.org/10.1109/SP.2018.00029).
- [58] Bittau A, Belay A, Mashtizadeh A, et al. Hacking blind [C/OL]//Proceedings of the 2014 IEEE Symposium on Security and Privacy (S&P). 2014: 227-242. DOI: [10.1109/SP.2014.22](https://doi.org/10.1109/SP.2014.22).
- [59] Kollenda B, Göktaş E, Blazytko T, et al. Towards automated discovery of crash-resistant primitives in binary executables [C/OL]//Proceedings of the 47th Annual IEEE/IFIP Inter-

- national Conference on Dependable Systems and Networks (DSN). 2017: 189-200. DOI: [10.1109/DSN.2017.58](https://doi.org/10.1109/DSN.2017.58).
- [60] Oikonomopoulos A, Athanasopoulos E, Bos H, et al. Poking holes in information hiding [C]// Proceedings of the 25th USENIX Security Symposium. 2016: 121-138.
- [61] Gruss D, Maurice C, Fogh A, et al. Prefetch side-channel attacks: Bypassing SMAP and kernel ASLR [C/OL]//Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS). 2016: 368-379. DOI: [10.1145/2976749.2978356](https://doi.org/10.1145/2976749.2978356).
- [62] Koschel J, Giuffrida C, Bos H, et al. TagBleed: Breaking KASLR on the isolated kernel address space using tagged TLBs [C/OL]//Proceedings of the 2020 IEEE European Symposium on Security and Privacy (EuroS&P). 2020: 309-321. DOI: [10.1109/EuroSP48549.2020.00027](https://doi.org/10.1109/EuroSP48549.2020.00027).
- [63] Song S, Zhang J, Yan M. Oreo: Protecting ASLR against microarchitectural attacks [C]// Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS). 2025.
- [64] Carlini N, Barresi A, Payer M, et al. Control-flow bending: On the effectiveness of control-flow integrity [C]//Proceedings of the 24th USENIX Security Symposium. 2015: 161-176.
- [65] Hu H, Shinde S, Adrian S, et al. Data-oriented programming: On the expressiveness of non-control data attacks [C/OL]//Proceedings of the 2016 IEEE Symposium on Security and Privacy (S&P). 2016: 969-986. DOI: [10.1109/SP.2016.62](https://doi.org/10.1109/SP.2016.62).
- [66] Larsen P, Franz M. Adoption challenges of code randomization [C/OL]//Proceedings of the 7th ACM Workshop on Moving Target Defense (MTD). 2020: 45-46. DOI: [10.1145/3411496.3421229](https://doi.org/10.1145/3411496.3421229).
- [67] Shen Z, Chen W. A survey of research on runtime rerandomization under memory disclosure [J/OL]. IEEE Access, 2019, 7: 105432-105440. DOI: [10.1109/ACCESS.2019.2931707](https://doi.org/10.1109/ACCESS.2019.2931707).
- [68] Stanley D M, Xu D, Spafford E H. Improved kernel security through memory layout randomization [C/OL]//Proceedings of the 32nd IEEE International Performance Computing and Communications Conference (IPCCC). 2013: 1-10. DOI: [10.1109/PCCC.2013.6742779](https://doi.org/10.1109/PCCC.2013.6742779).
- [69] The Linux Kernel Organization. The Linux kernel, version 6.8 [EB/OL]. 2024. <https://www.kernel.org/>.
- [70] Gruss D, Lipp M, Schwarz M, et al. KASLR is dead: Long live KASLR [C/OL]//Proceedings of the 9th International Symposium on Engineering Secure Software and Systems (ESSoS). 2017: 161-176. DOI: [10.1007/978-3-319-62105-0_11](https://doi.org/10.1007/978-3-319-62105-0_11).
- [71] Canella C, Schwarz M, Haubenwallner M, et al. KASLR: Break it, fix it, repeat [C/OL]// Proceedings of the 15th ACM Asia Conference on Computer and Communications Security (AsiaCCS). 2020: 481-493. DOI: [10.1145/3320269.3384747](https://doi.org/10.1145/3320269.3384747).
- [72] Göktaş E, Kollenda B, Koppe P, et al. Position-independent code reuse: On the effectiveness of ASLR in the absence of information disclosure [C/OL]//Proceedings of the 2018 IEEE European Symposium on Security and Privacy (EuroS&P). 2018: 227-242. DOI: [10.1109/EuroSP.2018.00024](https://doi.org/10.1109/EuroSP.2018.00024).
- [73] Bigelow D, Hobson T, Rudd R, et al. Timely rerandomization for mitigating memory disclosures [C/OL]//Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS). 2015: 268-279. DOI: [10.1145/2810103.2813691](https://doi.org/10.1145/2810103.2813691).
- [74] Lu K, Nürnberger S, Backes M, et al. How to make ASLR win the clone wars: Runtime rerandomization [C]//Proceedings of the 2016 Network and Distributed System Security Symposium (NDSS). 2016.

- [75] Wang Z, Wu C, Li J, et al. RERANZ: A light-weight virtual machine to mitigate memory disclosure attacks [C/OL]//Proceedings of the 13th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE). 2017: 143-156. DOI: [10.1145/3050748.3050752](https://doi.org/10.1145/3050748.3050752).
- [76] Davi L, Liebchen C, Sadeghi A R, et al. Isomeron: Code randomization resilient to (just-in-time) return-oriented programming [C]//Proceedings of the 2015 Network and Distributed System Security Symposium (NDSS). 2015.
- [77] Hawkins W, Nguyen-Tuong A, Hiser J D, et al. Mixr: Flexible runtime rerandomization for binaries [C/OL]//Proceedings of the 2017 Workshop on Moving Target Defense (MTD). 2017: 27-37. DOI: [10.1145/3140549.3140552](https://doi.org/10.1145/3140549.3140552).
- [78] Hawkins W H, Hiser J D, Co M, et al. Zipr: Efficient static binary rewriting for security [C/OL]//Proceedings of the 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). 2017: 559-566. DOI: [10.1109/DSN.2017.27](https://doi.org/10.1109/DSN.2017.27).
- [79] Williams-King D, Kobayashi H, Williams-King K, et al. Egalito: Layout-agnostic binary recompilation [C/OL]//Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2020: 133-147. DOI: [10.1145/3373376.3378470](https://doi.org/10.1145/3373376.3378470).
- [80] Nadeem M H. Linux kernel module continuous address space re-randomization [D]. Virginia Polytechnic Institute and State University, 2020.
- [81] Luo C, Fu J, Ming J, et al. PtrProxy: Efficient code re-randomization on AArch64 platform [J]. China Communications, 2025, 22(1).
- [82] Morton M, Koo H, Li F, et al. Defeating zombie gadgets by re-randomizing code upon disclosure [C/OL]//Proceedings of the 9th International Symposium on Engineering Secure Software and Systems (ESSoS). 2017: 143-160. DOI: [10.1007/978-3-319-62105-0_10](https://doi.org/10.1007/978-3-319-62105-0_10).
- [83] Li Y, Chung Y C, Xing J, et al. MProbe: Make the code probing meaningless [C/OL]//Proceedings of the 38th Annual Computer Security Applications Conference (ACSAC). 2022. DOI: [10.1145/3564625.3567967](https://doi.org/10.1145/3564625.3567967).
- [84] Li Y, Bao Y, Chung Y C. Randomize the running function when it is disclosed [J/OL]. IEEE Transactions on Computers, 2024, 73(6): 1516-1530. DOI: [10.1109/TC.2024.3377497](https://doi.org/10.1109/TC.2024.3377497).
- [85] Backes M, Holz T, Kollenda B, et al. You can run but you can't read: Preventing disclosure exploits in executable code [C/OL]//Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS). 2014: 1342-1353. DOI: [10.1145/2660267.2660378](https://doi.org/10.1145/2660267.2660378).
- [86] Gionta J, Enck W, Ning P. HideM: Protecting the contents of userspace memory in the face of disclosure vulnerabilities [C/OL]//Proceedings of the 5th ACM Conference on Data and Application Security and Privacy (CODASPY). 2015: 325-336. DOI: [10.1145/2699026.2699107](https://doi.org/10.1145/2699026.2699107).
- [87] Braden K, Davi L, Liebchen C, et al. Leakage-resilient layout randomization for mobile devices [C/OL]//Proceedings of the 23rd Network and Distributed System Security Symposium (NDSS). 2016. <https://www.ndss-symposium.org/wp-content/uploads/2017/09/leakage-resilient-layout-randomization-mobile-devices.pdf>.
- [88] Kwon D, Shin J, Kim G, et al. uXOM: Efficient eXecute-Only memory on ARM Cortex-M [C/OL]//Proceedings of the 28th USENIX Security Symposium. 2019. <https://www.usenix.org/conference/usenixsecurity19/presentation/kwon>.

- [89] Shen Z, Dharsee K, Criswell J. Fast execute-only memory for embedded systems [C/OL]// Proceedings of the IEEE Secure Development Conference (SecDev). 2020. <https://ieeexplore.ieee.org/document/9229982>.
- [90] Chen Y, Zhang D, Wang R, et al. NORAX: Enabling execute-only memory for COTS binaries on AArch64 [C/OL]//Proceedings of the 38th IEEE Symposium on Security and Privacy (S&P). 2017. DOI: [10.1109/SP.2017.30](https://doi.org/10.1109/SP.2017.30).
- [91] Intel 64 and IA-32 architectures software developer's manual [M]. Intel Corporation, 2023.
- [92] Crane S, Liebchen C, Homescu A, et al. Readactor: Practical code randomization resilient to memory disclosure [C/OL]//Proceedings of the 2015 IEEE Symposium on Security and Privacy (S&P). 2015: 763-780. DOI: [10.1109/SP.2015.52](https://doi.org/10.1109/SP.2015.52).
- [93] Pomonis M, Petsios T, Keromytis A D, et al. Kernel protection against just-in-time code reuse [D/OL]. Columbia University, 2019. DOI: [10.1145/3277592](https://doi.org/10.1145/3277592).
- [94] Tolvanen S. Control flow integrity in the Android kernel [Z]. 2018.
- [95] Burow N, Zhang X, Payer M. SoK: Shining light on shadow stacks [C/OL]//Proceedings of the 2019 IEEE Symposium on Security and Privacy. 2019: 985-999. DOI: [10.1109/SP.2019.00076](https://doi.org/10.1109/SP.2019.00076).
- [96] Kuznetsov V, Szekeres L, Payer M, et al. Code-pointer integrity [C]//Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2014: 147-163.
- [97] Liljestrand H, Nyman T, Gunn L J, et al. PACStack: An authenticated call stack [C]// Proceedings of the 30th USENIX Security Symposium. 2021: 357-374.
- [98] Ismail M, Quach A, Jelesnianski C, et al. Tightly seal your sensitive pointers with PACTight [C]//Proceedings of the 31st USENIX Security Symposium. 2022: 3717-3734.
- [99] The LLVM Project. Writing an LLVM pass and the new pass manager [Z]. 2024.
- [100] Yoo S, Park J, Kim S, et al. In-kernel control-flow integrity on commodity OSes using ARM pointer authentication [C/OL]//Proceedings of the 31st USENIX Security Symposium. 2022. <https://www.usenix.org/conference/usenixsecurity22/presentation/yoo>.
- [101] Jang J, Kang B B. Revisiting the ARM debug facility for OS kernel security [C/OL]// Proceedings of the 56th Annual Design Automation Conference (DAC). 2019. DOI: [10.1145/3316781.3317897](https://doi.org/10.1145/3316781.3317897).
- [102] Gravani S, Hedayati M, Criswell J, et al. Fast intra-kernel isolation and security with IskiOS [C/OL]//Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses (RAID). 2021. DOI: [10.1145/3471621.3471849](https://doi.org/10.1145/3471621.3471849).
- [103] Maar L, Nasahl P, Mangard S. Beyond the edges of kernel control-flow hijacking protection with HEK-CFI [C/OL]//Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (ASIA CCS). 2024: 1214-1230. DOI: [10.1145/3634737.3661135](https://doi.org/10.1145/3634737.3661135).
- [104] Ge X, Talele N, Payer M, et al. Fine-grained control-flow integrity for kernel software [C/OL]//Proceedings of the 2016 IEEE European Symposium on Security and Privacy (EuroS&P). 2016: 179-194. DOI: [10.1109/EuroSP.2016.24](https://doi.org/10.1109/EuroSP.2016.24).
- [105] Moreira J, Rigo S, Polychronakis M, et al. DROP THE ROP: Fine-grained control-flow integrity for the Linux kernel [Z]. 2017.
- [106] Khandaker M R, et al. Origin-sensitive control flow integrity [C/OL]//Proceedings of the 28th USENIX Security Symposium. 2019. <https://www.usenix.org/conference/usenixsecurity19/presentation/khandaker>.

- [107] Kasten F, Zieris P, Horsch J. Integrating static analyses for high-precision control-flow integrity [C/OL]//Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID). 2024. DOI: [10.1145/3678890.3678920](https://doi.org/10.1145/3678890.3678920).
- [108] Li Y, Wang M, Zhang C, et al. Finding cracks in shields: On the security of control flow integrity mechanisms [C/OL]//Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS). 2020. DOI: [10.1145/3372297.3417867](https://doi.org/10.1145/3372297.3417867).
- [109] Kim J, Park J, Roh S, et al. TikTag: Breaking ARM's memory tagging extension with speculative execution [C/OL]//arXiv preprint arXiv:2406.08719. 2024. DOI: [10.48550/arXiv.2406.08719](https://doi.org/10.48550/arXiv.2406.08719).

致 谢

此处填写致谢。

2023 年 6 月

作者简历及攻读学位期间发表的学术论文与其他相关学术成果

作者简历：

2016 年 9 月——2020 年 6 月，在复旦大学计算机科学技术学院获得学士学位。

2020 年 9 月——2022 年 6 月，在中国科学院计算技术研究所获得硕士学位。

2022 年 9 月——至今，在中国科学院计算技术研究所攻读博士学位（硕博连读）。

工作经历：无。

已发表（或正式接受）的学术论文：

- (1) 李润宽, 胡斌, 刘晓东, 赵晓芳. I-KASLR: 面向智能电网等实时关键系统的内置内核代码持续重随机化方法 [J]. 高技术通讯, 2026, 36(1): 1-14.
- (2) 李润宽, 胡斌, 赵晓芳, 蒋德钧. 基于编译的 Linux 内核地址空间布局控制方法 [J]. 高技术通讯, 2025, 34(9): 921-934. DOI: 10.3772/j.issn.1002-0470.2025.09.002.
- (3) Zihao Chang, **Runkuan Li**, JiHan Lin. Chaos: Function Granularity Runtime Address Layout Space Randomization for Kernel Module[C]. // Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys 2024), 2024.

申请或已获得的专利：

- (1) 李润宽, 胡斌, 王盈, 刘洁, 常子豪, 张焕, 林纪涵, 蒋德钧, 王卅, 赵晓芳. 一种基于编译器的细粒度程序执行流随机化控制方法: 中国, 202410206829.X[P]. 实质审查中.
- (2) 李润宽, 刘洁, 胡斌, 蒋德钧, 赵晓芳. 一种基于隔离机制的 Linux 内核代码的地址空间布局触发式随机化方法: 中国, 202510342155.3[P]. 实质审查中.

参加的研究项目及获奖情况：

